

Response Letter for Paper 344 $\Rightarrow$ 1528

Dear Meta Reviewer and Referees:

We have substantially revised the paper following the Referees' suggestions. For the convenience of Referees, changes for addressing the concerns of R1, R2 and R4 are color-coded in blue, orange and purple, respectively. We would like to thank the Referees for insightful comments and for supporting this work!

Below please find our responses to the reviews.

Response to the comments of the Meta Reviewer.

[Meta summary & recommendation] *The reviewers unanimously agree that the author's revision plan is reasonable and do not have any further suggestions beyond that. Still, please carefully reread the reviews and comments on all concerns in the revision letter.*

[A] Many thanks! We have followed the revision plan and addressed all comments of the Referees in the revision.

Response to the comments of Referee #1.

[R101] *The transition to some sections is, to some extent, abrupt (for example, the transition to the preliminary section). The writing is very notation-heavy. Although the authors provide examples after each section, the paper is still difficult to follow, as keeping track of all the notation at once is challenging. Some parts could be explained in plain text rather than relying heavily on formal notation. Especially in section 4.1, even the provided example is full of notation. The conveyed message seems concrete, but it is very hard to follow.*

[A] Thanks! As suggested, we have revised the presentation to improve the transitions between sections and added intuitive explanations for the notations. First, in *Contributions & organization*, we have clarified that Section 2 introduces the basic concepts and notation required for the paper, and outlined that Section 3 presents the hypergraph representation, Section 4 defines hyper simulation, and Section 5 presents HyperMatch for computing hyper simulation (p. 2). We have also revised the opening of Sections 3–5 to outline the purpose and structure of each section. Second, we have added Table 1 to summarize the key notation used throughout the paper (p. 3). Moreover, we have revised Section 4.1, which was stressed by the Referee. Before presenting the definitions, we now intuitively explain the meaning of hyper simulation, and clarify the roles of its three components: h_v checks whether two vertices are consistent, $HC^{\sigma,b}$ identifies comparable hyperedge clusters, and $\mathcal{D}^\delta$ determines the vertex pairs that must be compared between the hyperedge clusters. We have also rewritten Example 4 to explain how two hyperedge clusters are paired, how their vertices are aligned and compared, and how a conflicting vertex pair is identified (pp. 5–6). Finally, in Sections 5.1–5.2, we have added intuitive explanations of the inputs, outputs, and main steps of HCCalc, the $\mathcal{D}$ -match computation, and HyperMatch before presenting detailed formulas and pseudocode (pp. 7–9).

[R102] *The claim that the approach is LLM-free needs justification. If leveraging LLM can ease some of the process, the authors should consider using it. In fact, the framework uses some pre-trained embeddings, NER, and NLI, which are cheaper than LLM, but if replacing some component with LLM improves the current results, then being*

strictly LLM-free should not be presented as a goal in itself, given the recent paradigm. Table 4 shows that LLM-based approaches appear to be slow; however, to motivate against using LLMs, the authors could consider providing a trade-off experiment comparing different variants of HyperMatch (for example, replacing a component with an LLM), which would better justify the claim.

[A] Thanks! We agree that avoiding LLMs should not be treated as a goal. Thus, in Section 1 (p. 2), we have removed the positioning of HyperMatch as LLM-free and clarified that HyperMatch does not require LLM invocations by default, while the ML-models $\mathcal{M}_1$ in HCCalc and $\mathcal{M}_2$ in $\mathcal{D}$ -match are modular and can be replaced with LLMs. The lightweight pretrained models used by default are efficient choices for latency-sensitive consistency checking, rather than a restriction of the framework. Beyond the end-to-end comparison with LLM-based methods, we have added controlled component-replacement experiments in Exp-5 (p. 12). Specifically, we have replaced $\mathcal{M}_1$ in HCCalc and $\mathcal{M}_2$ in $\mathcal{D}$ -match with Qwen3.5-9B, while keeping all other components unchanged. We have also compared lightweight non-LLM alternatives: Qwen3-EMB-0.6B and MiniLM-RR for $\mathcal{M}_1$, and DeBERTa-NLI and MiniLM-RR for $\mathcal{M}_2$. The reported Answer F1 and Runtime have been averaged over HotpotQA and MuSiQue. Replacing $\mathcal{M}_1$ with Qwen3.5-9B improves Answer F1 from 69.21% to 71.29%, but increases Runtime from 1.086 s to 105.20 s, i.e., by more than 96 $\times$. In contrast, replacing $\mathcal{M}_2$ with Qwen3.5-9B obtains 64.94% Answer F1 in 101.02 s, compared with 69.21% in 1.086 s for the default configuration. Thus, LLM substitution provides a modest Answer F1 gain for $\mathcal{M}_1$, but not for $\mathcal{M}_2$, while introducing substantial latency in both cases. These results show that HyperMatch does not exclude LLMs, rather, its default configuration provides a more practical effectiveness–efficiency balance for low-latency consistency checking.

[R103] *RAG experiment (exp-1): I understand that once they have retrieved results, then HyperMatch enforces that each vertex match must be consistent with all its neighboring relationships. However, the discussion seems to be very short. I was wondering what fraction of the retrieved information was inconsistent, and how much effort they required to distinguish between consistent and inconsistent.*

[A] Thanks! As suggested, we have expanded the RAG analysis to quantify both the prevalence of inconsistent retrievals and the effort required to identify and explain them. First, we have revised document marking in Section 4.2 (p. 6) to enhance downstream RAG. Each retrieved document is marked based on the behavior and results of the hyper simulation between the query and the documents, and each annotation consists of a category and an reason. A document is marked as *consistent*, *useless*, *conflict*, or *irrelevant*. The latter three categories are counted as inconsistent retrievals, while only the *conflict* category denotes a contradiction with the query. Moreover, to quantify the fraction of inconsistent retrievals, we have introduced GIR@ k , the proportion of the top- k documents manually confirmed as inconsistent with the query. We also report IDR@ k , the recall achieved by HyperMatch in identifying these documents (p. 9). Table 4 reports both metrics on HotpotQA and MuSiQue. Across the evaluated retrieval settings, GIR@ k ranges from 82.6% to 99.8%, showing that inconsistent documents constitute a large fraction of the retrieved results, while IDR@ k remains

between 94.9% and 99.4%. Appendix E.4 (Table 11, p. 30, in [1]) shows that the document marking time, already included in Table 6, accounts for only 0.902% and 0.542% of the end-to-end runtime on HotpotQA and MuSiQue, respectively. Finally, Exp-6 traces a HotpotQA instance in which HyperMatch marks two documents as *consistent* and one as *conflict* (pp. 12–13). The first two documents provide the “interview” and “team–league” relations needed to derive the answer “Adrian Lewis Peterson”. For the third document, HC matches the relation that “Ricky Williams played for the Miami Dolphins” with the query relation that “the requested running back played for the New Orleans Saints”. The teams “Miami Dolphins” and “New Orleans Saints” are inconsistent, as computed by $h_v(\cdot, \cdot) = \text{false}$. HyperMatch therefore marks the document as *conflict* and reports these two entities as the reason. In contrast, Vanilla RAG passes the original top- k documents, including this conflicting document, downstream without such a mark.

[R104] *The experimental analysis only includes a sensitivity study of some hyperparameters, which seems inadequate. The design choices of their framework has not been justified. Including an ablation study that isolates individual components (e.g., removing CR or hyperedge clustering) would strengthen the paper by demonstrating how each part affects performance. Without these, it’s very hard to justify the motivation behind their framework design.*

[A] Thanks! As suggested, we have added an ablation study in Exp-5 of Section 6 (Table 7, p. 12), which evaluates the key design choices of HyperMatch in terms of Answer F1 and Runtime, averaged over HotpotQA and MuSiQue. The study isolates three groups of designs: (1) replacing the hypergraph representation with a standard graph or removing coreference resolution; (2) removing hyperedge cluster and directly comparing individual hyperedges; and (3) removing $\mathcal{D}$ -match by treating its checks as satisfied by default. Table 7 also reports variants that replace $\mathcal{M}_1$ in HCCalc and $\mathcal{M}_2$ in $\mathcal{D}$ -match with alternative models. Please refer to our response to **[R102]** for their accuracy–runtime trade-offs. Relative to Full HyperMatch (69.21%, 1.086 s), replacing the hypergraph with a graph, removing coreference resolution, removing hyperedge cluster, and removing $\mathcal{D}$ -match reduce Answer F1 by 27.25%, 14.89%, 14.17%, and 35.54%, respectively. Their runtimes are 1.091 s, 1.031 s, 0.825 s, and 0.799 s, respectively. These results show that the runtime reductions obtained by simplifying the framework come with substantial accuracy losses: the hypergraph preserves multi-way relations, coreference resolution connects entities expressed through different mentions, hyperedge clusters enable multiple relations involving the same entity to be compared jointly, and $\mathcal{D}$ -match enforces the required vertex correspondences between them.

Response to the comments of Referee #2.

[R201] *The proposed method may depend heavily on the performance of existing techniques for entity extraction and resolution, since it requires entity extraction and hypergraph construction as preprocessing steps. The quality of these steps may be crucial to the overall method. As a result, the reported performance gains may largely stem from entity recognition and coreference resolution techniques that have already been extensively studied in the literature. Therefore, I wonder whether the benefits of the proposed method come from these existing*

techniques or from the hypergraph representation itself.

[A] Thanks! We agree that entity extraction and coreference resolution affect the quality of HyperMatch, but they play a different role from the hypergraph representation. In Sections 3.1–3.2 (pp. 3–4), we have clarified that rule-based extraction and NER identify the text spans represented as vertices, and coreference resolution merges mentions referring to the same entity. The hypergraph representation then organizes these vertices into multi-way relations, which HyperMatch jointly and recursively checks. To separate the contribution of the representation from that preprocessing, we have added an ablation study in Exp-5 of Section 6 (Table 7, p. 12), with Answer F1 averaged over HotpotQA and MuSiQue. Replacing only the hypergraph representation with a standard graph, reduces Answer F1 from 69.21% to 41.96%. The no-coreference resolution variant retains the hypergraph but removes coreference resolution, reducing Answer F1 to 54.32%. These results show that the hypergraph representation is central to HyperMatch: replacing it with a standard graph under unchanged preprocessing reduces Answer F1 by 27.25%, while existing preprocessing such as coreference resolution also contributes, with its removal causing a 14.89% loss. Please refer to our response to **[R104]** for the ablation settings and results.

[R202] *The technical contributions are not clearly explained. In Section 3.2, the construction of the hypergraph appears analogous to the construction of a knowledge graph. Why not simply use a knowledge graph for this purpose? The justification for using a hypergraph is not well articulated, nor are the subsequent definitions of hyper-simulation. Moreover, the construction process in Section 3.2 seems quite straightforward, and I do not see what technical challenges are being addressed. In addition, Algorithm 2 appears to use a brute-force approach to find adjacent hyperedges.*

[A] Thanks! Our responses to the three questions are as follows.

(1) We have revised the contributions of the work in Section 1 (p. 2) to clarify that document-to-hypergraph construction provides the input representation, while our main technical contributions are hyper simulation and HyperMatch, which extend graph simulation to hypergraphs for consistency checking and do not increase computational complexity. In Section 3.1 (p. 3), we have further explained why a hypergraph rather than a standard graph is used: the hypergraph preserves each multi-way relation as a single unit, allowing all participating entities to be checked jointly. This choice is supported by the controlled graph variant in Exp-5 (Table 7, p. 12): replacing the hypergraph representation with a standard graph reduces Answer F1 from 69.21% to 41.96%, showing that decomposing multi-way relations into edges loses information required for joint consistency checking. Please refer to **[R104]** for the complete ablation results. We have also added an intuitive explanation in Section 4.1 (p. 5) of why graph simulation cannot be directly applied to hypergraphs. Graph simulation follows adjacent binary edges. In contrast, hyper simulation needs to determine which relations between two hypergraphs should be compared, how the entities involved in these relations are matched, and how inconsistencies among these matches are recursively propagated.

(2) We have stated the two challenges of constructing a hypergraph from a document, at the beginning of Section 3.2 (p. 4): identifying the text spans that should form vertices and merging mentions of

the same entity, and determining which groups of vertices express a relation without enumerating all possible combinations. We address the former using rule-based extraction, NER, and coreference resolution, and the latter by grouping vertices through paths over E_{dep} obtained from dependency parsing to construct hyperedges.

(3) In Section 5.1 (p. 7), we have clarified that Algorithm 2 does not enumerate arbitrary combinations of adjacent hyperedges. The 2-section property and the maximum query hyperedge size Δ_E restrict candidate generation to vertex pairs induced by query hyperedges, b bounds the candidate data vertices, and m bounds the constructed hyperedge cluster size. For each selected vertex pair, HCCalc constructs one hyperedge cluster along a shortest path rather than enumerating alternative paths and their combinations. Under these bounds, Theorem 2 establishes the complexity $O(|E_Q|(|V| + |E|))$.

[R2O3] *The datasets used in the experiments are very small, as shown in Table 1. In the efficiency experiments, the preprocessing time should also be compared across methods. In addition, the performance shown in Table 3 for conflict detection task is not particularly strong. Finally, why are there 2 separate sets of experiments evaluating b and δ ?*

[A] Thanks! We address the four concerns as follows.

(1) We acknowledge that some individual benchmarks contain relatively few evaluated instances. However, they are established public benchmarks rather than ad hoc small samples. The selected datasets are representative and widely adopted for real-world RAG and multi-hop QA [B1,B2] and for knowledge-conflict and document-level contradiction detection [B3,B4]; their scale and difficulty have also been examined in recent benchmark studies [B5,B6]. In the Experimental Setting of Section 6 (p. 9), we have now clarified this benchmark selection. Our evaluation covers 17,094 instances from 11 public datasets, including five RAG and six conflict-detection datasets. Table 2 (p. 10) further reports, for every dataset, the number of instances, the average query and data hypergraph sizes in terms of vertices and hyperedges, and the data type.

[B1] Gan et al., “Retrieval Augmented Generation Evaluation in the Era of LLMs: A Comprehensive Survey,” arXiv 2025.

[B2] Mavi et al., “A Survey on Multi-hop Question Answering and Generation,” arXiv 2022.

[B3] Xu et al., “Knowledge Conflicts for LLMs: A Survey,” EMNLP 2024.

[B4] Li et al., “ContraDoc: Understanding Self-Contradictions in Documents with Large Language Models,” NAACL 2024.

[B5] Lee et al., “GRADE: Generating Multi-hop QA and Fine-grained Difficulty Matrix for RAG Evaluation,” EMNLP 2025.

[B6] Chen et al., “Think Wider, Detect Sharper: Reinforced Reference Coverage for Document-Level Self-Contradiction Detection,” EMNLP 2025.

(2) In Exp-3 of Section 6 (Table 6, p. 11), we have revised the efficiency comparison to report both execution and preprocessing time wherever a separate preprocessing phase is required. On MuSiQue and HotpotQA, the preprocessing times are 0.450 s and 0.270 s for HyperMatch, 0.463 s and 0.459 s for BSIM, and 0.262 s and 0.274 s for ParaSIM, respectively. The speedups are computed using the sum of execution and preprocessing time. Under this end-to-end accounting, HyperMatch takes 1.247 s and 0.925 s per query and is $4.18\times$ – $34.38\times$ faster than the LLM-based methods, $1.48\times$ – $1.97\times$

faster than BSIM, and $12.11\times$ – $30.65\times$ faster than ParaSIM.

(3) We agree that the improvements on some conflict-detection datasets are moderate. In Exp-1 (Table 5, p. 10), we have expanded the evaluation by Precision, Recall, and F1 on six datasets. HyperMatch achieves the highest F1 on five datasets and ranks second on SNLI by 0.19%; its improvements include 0.04% on ContractNLI, 2.80% on ECON, and 1.59% on ConTROL. Beyond aggregate performance, an advantage of simulation-based consistency checking is its ability to explain inconsistent entities, relations, and recursively unsupported matches. Exp-6 (pp. 12–13) traces these operations and localizes a conflict to the “New Orleans Saints”–“Miami Dolphins” vertex pair, while Appendix G (pp. 30–34, in [1]) provides four additional cases with fine-grained reasons. Please also refer to our response to **[R1O3]**.

(4) The two parameter studies evaluate different outcomes. Exp-2 (pp. 10–11) studies how σ , b , and δ affect task effectiveness, measured by Answer F1 for RAG and F1 for conflict detection. Exp-4 (pp. 11–12) instead studies how document and hypergraph sizes and these parameters affect Runtime and scalability. We have revised the experiment titles and discussions to make this distinction explicit.

[R2O4] *The proposed method appears to have high time complexity, seemingly quadratic in the graph size, as indicated in Theorem 3. This may be computationally expensive.*

[A] Thanks! We have clarified that the apparent quadratic term is inherited from graph simulation, which also examines candidate vertex pairs between the graphs. HyperMatch runs in $O(|V_Q||V| + |E_Q|(|V| + |E|))$, whereas graph simulation runs in $O((|V_Q| + |E_Q|)(|V| + |E|))$. This shows that extending graph simulation from graphs to hypergraphs does not increase the complexity. We added this comparison after Theorem 3 in Section 5.2 (p. 9) and clarified the same complexity positioning in Sections 1 and 7 (pp. 3 and 13). The empirical results further show that the empirical runtime remains practical. Including preprocessing, HyperMatch takes 0.925–1.247 s per query and is $1.48\times$ – $30.65\times$ faster than the graph simulation baselines in Exp-3 (Table 6, p. 11). Exp-4 (pp. 11–12) additionally verifies predictable scaling with document length and on synthetic data hypergraphs containing up to one million vertices and 400K hyperedges. Please refer to **[R2O3]** for the complete efficiency results.

[R2O5] *In Section 2, the paper mentions existing graph simulation methods. What is the difference between the proposed approach and the graph simulation method? In particular, does extending graph simulation to hypergraphs require significantly different designs?*

[A] Thanks! Hyper simulation is not a direct application of graph simulation to another data model. Although it retains the fixed-point principle of graph simulation, extending the matching object from binary edges to multi-entity relations requires new designs for both the hyper simulation definition and HyperMatch algorithm. For the representation-level motivation for using hypergraphs, please refer to **[R2O2]**. At the definition level, graph simulation checks whether each adjacent binary query edge is supported by a data edge whose endpoint remains matched. Hyper simulation instead determines which relations between two hypergraphs should be compared, which entities within those relations should be compared, and how inconsistencies among these entity

pairs are propagated recursively (p. 5). We have therefore introduced hyperedge clusters and $\mathcal{D}$ -match in Section 4.1 (pp. 5–6). A hyperedge cluster groups relations containing common entities, while a $\mathcal{D}$ -match identifies the entity pairs to be compared between two hyperedge clusters. To evaluate a candidate vertex pair, hyper simulation Π considers every hyperedge cluster pair used to compare relations containing those vertices. For each hyperedge cluster pair, the corresponding $\mathcal{D}$ -match must contain the candidate pair, and every pair specified by that $\mathcal{D}$ -match must also belong to Π . At the algorithm level, Sections 5.1–5.2 (pp. 7–9) introduce HCCalc to calculate hyperedge cluster pairs, a procedure for computing $\mathcal{D}$ -matches, and HyperMatch that propagate invalid vertex pairs through the hyperedge cluster pairs. Exp-5 further validates the necessity of these designs (Table 7, p. 12): removing hyperedge cluster and $\mathcal{D}$ -match reduces Answer F1 from 69.21% to 55.04% and 33.67%, respectively. Section 7 (p. 13) summarizes this distinction. Please refer to [R2O4] for the complexity comparison.

[R2O6] *There are too many parameters to be tuned, including θ , σ , b , and δ , as shown in Section 4.2. This creates a considerable burden.*

[A] Thanks! To reduce the tuning burden, we added a parameter configuration guide in Appendix F (p. 30, in [1]). It distinguishes θ as a cutoff controlled by the external retrieval module rather than a parameter tuned within HyperMatch, and summarizes the roles, default settings, and tuning requirements of the remaining parameters. In practice, HyperMatch can use the recommended stable configuration. Only optional one-at-a-time local adjustment is needed for specific accuracy–efficiency preferences. The corresponding sensitivity results are reported in Exp-2 and Exp-4 (pp. 10–12).

[R2O7] *In the introduction, the authors argue that existing methods are limited because they assume a specific data model, "e.g., graphs, where corresponding values can be compared for conflicts." However, this work also relies on a specific graph-based data model, namely a hypergraph. Therefore, I do not think this is a convincing argument against existing methods.*

[A] Thanks! We clarified that the limitation is not the use of a data model itself, since our method also uses a hypergraph. Rather, existing consistency constraints assume structured or semi-structured inputs in which entities and relations are already explicit, whereas documents returned by vector retrieval are unstructured. Their entities and relations must therefore be explicitly represented before consistency checking. We revised Section 1 (p. 2) to make this distinction clear and Sections 3, 7 (pp. 3–4, 13) to explain that the hypergraph captures multi-way relations as the input to hyper simulation.

Response to the comments of Referee #4.

[R4O1] *Novelty was not entirely clear to me after reading the introduction. There is existing work on converting documents to graphs, there is existing work on graph simulation, etc. Is there existing work on converting documents to hypergraphs, or hypergraph simulation? And if not, what are the new technical challenges?*

[A] Thanks! Prior work has indeed studied hypergraph representations and document-to-hypergraph construction. However, these studies do not define a simulation relation over hypergraphs or develop an algorithm for computing such a relation. To the best of

our knowledge, there is no prior hypergraph simulation method for simulation-based consistency checking. We therefore clarified that our novelty lies in hyper simulation and HyperMatch, rather than document-to-hypergraph conversion alone. Please refer to our response to [R2O2] for the motivation and construction challenges, to [R2O5] for the technical differences from graph simulation, and to [R2O4] for the complexity comparison. Accordingly, we revised Sections 1 and 7 (pp. 2 and 13) to clarify the novelty and review prior hypergraph-construction methods.

[R4O2] *Additional baselines could be: 1. State of the art RAG methods such as HyDE (<https://arxiv.org/pdf/2212.10496>). 2. Trying to solve the problem upstream by benchmarking the quality of embeddings, using MTEB or similar approaches (<https://arxiv.org/abs/2210.07316>).*

[A] Thanks! We incorporated both suggestions. We added HyDE to the RAG effectiveness comparison in Exp-1 (Table 3, p. 10) and the efficiency comparison in Exp-3 (Table 6, p. 11). Following the suggestion to evaluate upstream embedding quality, we also added an *impact of embedding quality* study in Exp-1 (Table 4, p. 10). We compare four embedding models on HotpotQA and MuSiQue under multiple top- k settings. The study analyzes how embedding quality affects retrieved inconsistencies and downstream answer quality. For each configuration, Table 4 reports the fraction of inconsistent retrievals ($\text{GIR}@k$), and downstream Answer F1. Changing the embedding model or k affects the retrieved documents but does not eliminate inconsistency: $\text{GIR}@k$ ranges from 82.6% to 99.8% across all configurations. On MuSiQue, reducing k increases $\text{GIR}@k$ while decreasing Answer F1, indicating that smaller retrieved sets may exclude consistent evidence required to answer the query.

[R4M1] *On page 4, in the description of Algorithm 4, it would be helpful to see some examples of rules instead of being directed to [1].*

[A] Thanks! We clarify that this comment refers to Algorithm 1 for hypergraph construction. As suggested, we revised Example 3 in Section 3.2 (p. 5) to present two representative deduction rules in $\mathcal{R}_1$ using compact transitions over the state (I, R) . The example shows how the ENTITY rule extracts “Scholar Nilsson” as one vertex and how successive applications of the EVENT_{compound} rule expand “seminar” into “ethics seminar” and then “AI ethics seminar”. This example illustrates both the effect of rules and the necessity of composing rules to construct complete vertices from text.

[R4M2] *In the experimental study, I was hoping to see some qualitative examples of improved retrieval by HyperMatch compared to baselines.*

[A] Thanks! As suggested, we added a case study in Exp-6 (pp. 12–13). The case study shows that HyperMatch marks two retrieved documents as consistent and one as conflict, and reports the conflicting vertex pair as the reason, whereas Vanilla RAG forwards the original retrieved documents without such consistency annotations. Please refer to [R1O3] for the detailed analysis. Appendix G (pp. 31–35, in [1]) provides four additional cases.

Availability of artifacts

[R4] *Code is available but some is not commented.*

[A] Thanks! We add comments to improve its readability.

Catching Inconsistencies in Vector Similarity Retrieval

Paper ID: 344 $\Rightarrow$ 1528

Abstract

The rapid adoption of large language models and AI agents has driven the extensive use of vector databases for fundamental tasks such as retrieval-augmented generation, yet the issue of vector data consistency remains largely unaddressed despite significant optimizations in indexing and storage. Current systems rely on distance metrics that are insufficient to express complex logical dependencies, frequently resulting in vectors with high cosine similarity that correspond to semantically conflicting documents. Furthermore, detecting these inconsistencies using large language models incurs substantial computational overhead that conflicts with the stringent latency requirements of vector databases. To address this gap, we propose a fine-grained, efficient, and interpretable method for determining the consistency of vector data. Our approach utilizes a hypergraph representation of the retrieved documents to capture the semantics in natural language. Based on this representation, we formalize vector data consistency through hyper simulation, which extends graph simulation to recursively check the consistency of entities and relations. We further develop HyperMatch to compute the hyper simulation relation. Experimental results demonstrate that our method improves Answer F1 by 3.94% on average (up to 10.78%) in RAG tasks and F1 score by 0.94% on average (up to 2.80%) in conflict detection over the baselines. Additionally, it maintains an average execution time of 1.247 seconds per query, achieving speedups of up to 34.38 $\times$ over LLM-based methods and 30.65 $\times$ over graph simulation approaches.

1 Introduction

The rapid proliferation of large language models (LLMs) and AI agents across various domains, such as healthcare [122], education [127], finance [75], and science [59], has driven increasing research on vector databases that provide vector management services including memory management [54], retrieval-augmented generation (RAG) [7], multimodal retrieval [68], and query caching [8]. Gartner has recognized it as key emerging technology [16], with billions invested recently [2]. These databases provide efficient vector search via distance metrics (e.g., cosine similarity, Euclidean distance) with optimizations of index design [84], storage [69], automatic tuning [120], etc., which retrieve vectors and corresponding raw data (typically documents from knowledge bases [23]) that are semantically similar to the query. However, little attention has been paid to the data quality issue of vector retrievals, especially *the consistency problem*, which jeopardizes the usability of vector databases. Consider the following real-life cases.

Example 1: Figure 1 illustrates four retrievals in a RAG application, where Qwen3-Emb-0.6B [126] is employed to embed each query Q_i and knowledge base documents as vectors, and both vectors similar to the input embedding and their corresponding documents in the knowledge base (implemented by a vector database) are retrieved to an LLM ordered by cosine similarity score [73]. Note that D_j in each case is only an exemplary result for illustration purposes, which does not represent all retrievals for each query. We find the

Q_1	Who was officially crowned Emperor of the Holy Roman Empire by the Pope in Rome on Christmas Day?
D_1	Pope Leo III officially crowned Charlemagne as Emperor of the Holy Roman Empire in Rome on December 1st, 800 AD.
Q_2	The board of directors transferred the core intellectual property to an open-source foundation.
D_2	The board of directors transferred to an offshore shell corporation.
Q_3	Scholar Nilsson delivered a keynote at Stockholmsmässan on August. He also participated in roundtable discussions. That day, the venue hosted an AI ethics seminar, which featured his keynote and discussions.
D_3	At Stockholmsmässan, Lars Nilsson gave a presentation on Aug. 15. He was also engaged in debates. A seminar on quantum computing was held there that same day, which included his presentation and debates.
Q_4	Who is the lead botanist of the Verdant Canopy expedition?
D_4	The Verdant Canopy expedition, focusing on rare epiphytes, was fully funded and organized by the Sylva Research Foundation.
D_5	Dr. Aris Thorne is currently the lead botanist of the Verdant Canopy ecological conservation park.
D_6	Dr. Elara Vance serves as the executive director and lead botanist at the Sylva Research Foundation.

Figure 1: Exemplary retrieval results of a vector database.

inconsistency of the retrievals as follows.

- (1) Although the vector embeddings of Q_1 and D_1 share high cosine similarity of 0.748, “Christmas Day” in Q_1 and “December 1st” in D_1 refer to different dates, which presents a factual mismatch. Thus, there exists an inconsistency between the high-dimensional vector space and the semantics of documents, where temporally contradictory statements are similar in vector retrievals.
- (2) Although the vector cosine similarity of Q_2 and D_2 reaches 0.713 there exist inconsistencies driven by structural and relational disparities. Specifically, while both documents share identical subject and verb, D_2 omits the direct object (“the core intellectual property”) presented in Q_2 , which results in inconsistent subject-verb-object structures. Furthermore, their relational predicates are inconsistent, as the target recipient shifts from “an open-source foundation” in Q_2 to “an offshore shell corporation” in D_2 .
- (3) Despite their vectors sharing a high cosine similarity of 0.759, the query Q_3 and document D_3 diverge in both entity and event semantics. “Scholar Nilsson” in Q_3 and “Lars Nilsson” in D_3 refer to distinct individuals, and the associated events are irrelevant (an AI ethics seminar versus a quantum computing one). This exemplifies an inconsistency in the vector retrieval, where documents discussing completely unrelated entities and events are projected into close proximity in the embedding space.
- (4) Q_4 requires multiple documents to reach the answer, where the candidate documents D_4 and D_6 are semantically consistent and can prompt the LLM to respond correctly. However, as the cosine similarities of Q_4 against D_4 , D_5 , and D_6 are 0.713, 0.716, and 0.575, respectively, only D_4 and D_5 , whose similarity scores are higher than the threshold 0.7, are passed to the LLM, which triggers

a factual contradiction where the “Verdant Canopy expedition” conflicts with the “Verdant Canopy ecological conservation park”. This shows that semantically consistent evidence may rank lower than conflicting information in distance-based vector retrievals. $\square$

These cases demonstrate the inconsistency problem in vector retrievals, where queries and documents sharing high vector similarity may provide semantically conflicting information. Worse yet, previous research demonstrate that these inconsistent retrievals jeopardize the generation quality of LLMs in RAG pipelines [80, 123], which encourages the development of mitigation strategies as follows. On the one hand, the information retrieval (IR) community has proposed a re-ranking phase [47, 51] that employs a specialized model to refine the top- k vector retrieval results. However, these models require re-training when the knowledge base updates [26, 28] and can hardly provide explanations for the re-ranking results [20, 92]. On the other, the natural language processing (NLP) community leverages LLMs to detect contradictions within the retrieved documents [101, 110]. Although empirical results prove this LLM-based approach promising [61, 63], the computational overhead [129], token consumption [119] and inevitable hallucinations [4, 74] of LLMs prohibit its practical use.

Different from previous solutions that employ re-ranking models or LLMs, we propose to address the inconsistency problem in vector retrievals from the data quality perspective. Specifically, an efficient, accurate and explainable detection method is desired to catch the inconsistencies in vector retrievals such that the generation quality of downstream LLMs in the RAG pipeline can be improved [80, 123]. However, it is non-trivial to develop this inconsistency detection method, which faces the following challenges.

First, the vector-based retrieval can hardly detect inconsistencies among documents. This is because that vector databases find documents by measuring the geometric similarity of their corresponding vectors, which merely considers whether two documents are similar in wording, rather than consistent in content. Specifically, as the vector similarity metrics can only capture geometric relationships of proximity, opposition and orthogonality in the high-dimensional embedding space, these metrics cannot express or detect the semantic of inconsistency in the documents. As illustrated in cases (1) and (3) of Example 1, two documents that share high vector similarity can make contradictory factual claims. Moreover, theoretical studies [115] have shown that projecting unbounded real-world entities into a fixed-dimensional continuous space inevitably causes semantic collisions, where distinct concepts are mapped to similar vectors. Therefore, detecting inconsistencies in vector retrievals requires more beyond vector distance metrics only.

Second, existing data consistency constraints are not applicable to vector retrievals. These constraints are defined over structured or semi-structured data, including relational data [6, 32–34, 103], trees [70, 107, 109], and graphs [35, 39, 43], where entities and relations have been clearly defined. Thus, these constraints can hardly adapt to the unstructured documents retrieved by vector search, as the entities along with relations are hidden in the texts. Worse yet, a relationship in documents may involve multiple entities that should be considered together, which cannot be expressed by simple graphs. For example, Q_2 and D_2 in Figure 1 both describe a transfer by the same board, while differ in the transferred object and destination. Therefore, a suitable data model that is able to capture these entities

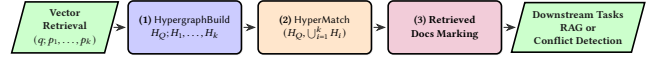


Figure 2: Workflow for inconsistency catching.

and their multi-way relations is desired for consistency checking.

Third, detecting inconsistencies in vector retrievals requires analyzing fine-grained text details and reasoning over entity relationships across documents. At the lexical level, conflicts often reside in details such as temporal-spatial expressions or entity names, as illustrated in cases (1), (3), and (4) of Example 1. At the syntactic level, a contradiction may result from modifying clauses even when the main sentence is consistent. For example, though documents in case (2) share the same subject and verb, the conflict arises in their modifiers. Worse yet, detecting complex contradictions requires reasoning across multiple sentences or documents. Case (4), for instance, requires cross-referencing three documents to determine whether the aggregated evidence remains semantically consistent with the query, or introduces a factual contradiction.

Contributions & organization. In view of these challenges, we propose simulation on hypergraphs to catch inconsistencies in vector retrievals. This extension of simulation from graphs to hypergraphs allows jointly checking entities involved in each multi-way relation, while maintaining the same complexity as graph simulation. Figure 2 illustrates the workflow. Specifically, we represent the retrieved document as a hypergraph, define vector retrieval consistency via hyper simulation, and develop HyperMatch to compute the hyper simulation relation. LLM invocations are optional in our method and fine-grained explanations for inconsistencies are provided. In practice, HyperMatch operates as a post-check on top- k vector candidates, and marks inconsistent documents before passing them to the LLM. Different from re-ranking models, our approach adapts to different domains and requires no retraining.

This paper is organized as follows. Section 2 reviews basics of vector databases, graph simulation, NLP methods and hypergraphs, with notations defined. Section 3 proposes the hypergraph representation of vector data and provides the construction of hypergraphs from the documents. Section 4 introduces hyper simulation to define the consistency of vector retrievals. Section 5 develops HyperMatch to compute the hyper simulation relation. The experimental studies are summarized in Section 6. Our main contributions are as follows.

(1) *Hypergraph Representation* (Section 3). We construct hypergraphs from the retrieved documents, which represent entities as vertices and their multi-way relations as hyperedges. We use dependency parsing, named entity recognition, coreference resolution, and deduction rules to construct these vertices and hyperedges.

(2) *Vector Retrieval Consistency* (Section 4). We propose hyper simulation, which extends graph simulation to hypergraphs and recursively checks the entities involved in multi-way relations. Specifically, the query and the retrieved vector data are consistent if the hyper simulation of their corresponding hypergraphs holds, where hyper simulation is a binary relation $\Pi \subseteq V_Q \times V$ defined over the vertex sets of a query hypergraph $H_Q = (V_Q, E_Q)$ and a document hypergraph $H = (V, E)$. A vertex pair $(u, v) \in \Pi$ indicates that u and v are consistent and can be recursively matched through their neighborhoods. Based on hyper simulation, we define when a query and its retrieved results are consistent.

(3) *Hyper Simulation Algorithm* (Section 5). We develop algorithm HyperMatch to compute the hyper simulation relation. It employs a fixed-point computation with cascaded pruning to iteratively eliminate conflicting vertex pairs until convergence. We prove that the algorithm correctly computes the unique maximum hyper simulation in $O(|V_Q||V| + |E_Q|(|E| + |V|))$ time, **which extends the graph simulation to hypergraphs while retaining the complexity.**

(4) *Experimental Study* (Section 6). Extensive experiments on real-world datasets across retrieval-augmented generation and conflict detection tasks have shown that our method outperforms state-of-the-art baselines in detection accuracy while maintaining tractable computational overhead. Specifically, on RAG tasks where inconsistencies are most prevalent, HyperMatch outperforms the best baseline by up to **10.78%** and **3.94%** on average in Answer F1, while achieving an F1 by up to 94.45% and 81.25% on average on conflict detection, and maintaining a low latency of **1.247** s per query (**34.38×** faster than LLM-based approaches).

2 Preliminaries

We start by reviewing basic concepts with notations provided.

Vector Database. Let $\mathcal{V}^d = \{p_1, p_2, \dots, p_n\}$ denote a vector database comprising n data entries. Each entry p_i in the d -dimensional space is formalized as a tuple (v, D) , where $v \in \mathbb{R}^d$ is the *vector embedding*, and $D \in \Sigma^*$ represents its *corresponding document* over alphabet Σ .

Graph and Path. A *labeled graph* is denoted by $G = (V, E, \llbracket \cdot \rrbracket)$, where V is a finite set of vertices, $E \subseteq V \times V$ represents the set of directed edges, and $\llbracket \cdot \rrbracket : V \rightarrow \Gamma$ is a labeling function assigning a label from an alphabet Γ to each vertex. A *path* ρ in G is a sequence of vertices $(v_1, v_2, \dots, v_k)$ such that $(v_i, v_{i+1}) \in E$ for all $1 \leq i < k$.

Graph Simulation. A graph simulation [57] defines a binary relation $S \subseteq V_Q \times V$ between the vertex sets of graph $G_Q = (V_Q, E_Q, \llbracket \cdot \rrbracket)$ and graph $G = (V, E, \llbracket \cdot \rrbracket)$. For every pair $(u, v) \in S$, it must satisfy: (1) $\llbracket u \rrbracket = \llbracket v \rrbracket$, and (2) for every edge $(u, u') \in E_Q$, there exists an edge $(v, v') \in E$ such that $(u', v') \in S$.

Tokens. Let Υ denote a countably infinite set of *tokens*, serving as the basic semantic units in NLP. The tokenization process is defined as a mapping $\mathcal{T} : \Sigma^* \rightarrow \Upsilon^*$, which parses an input document $D \in \Sigma^*$ into a sequence of tokens $T = \mathcal{T}(D) = (t_1, t_2, \dots, t_m)$, where $t_i \in \Upsilon$. We define a contiguous sequence of tokens as a *token span*.

Universal Dependencies. Universal Dependencies (UDs) [89] provide a framework for consistent grammatical annotation, computed by *dependency parsing* [60, 95]. We represent the UD of a document as a triplet (T, E_{dep}, Pos) . Here, T is the ordered set of tokens acting as vertices, and $E_{dep} \subseteq T \times \text{DEP} \times T$ denotes the set of dependency arcs. These arcs define a labeled dependency on tokens, where each triplet (t, dep, t') indicates a directed edge from a head token t to a dependent token t' with relation $\text{dep} \in \text{DEP}$. The mapping $\text{Pos} : T \rightarrow \text{POS}$ assigns a part-of-speech tag [15] to each token. Figure 3 illustrates an UD fragment over D_2 , where the directed arcs represent the dependency types dep and the gray annotations below the tokens indicate their part-of-speech tags.

Named Entity Recognition. Named Entity Recognition (NER) is a sequence-labeling task that identifies and classifies named entities

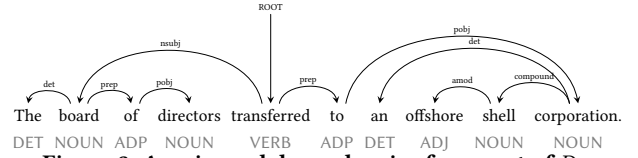


Figure 3: A universal dependencies fragment of D_2 .

within a text. Formally, we define it as a mapping function $\text{Ent} : T^* \rightarrow \text{ENT} \cup \{\perp\}$, where ENT is a predefined set of entity categories.

Coreference Resolution. Coreference resolution identifies expressions in a text that refer to the same entity. Let $M = I_1, \dots, I_k$, where each *mention* I_i is a contiguous span of the token sequence T . Writing $I_i \sim I_j$ when two mentions are coreferential, we define $\text{CR} : M \rightarrow M/\sim$ by $\text{CR}(I_i) = [I_i]_\sim = I_j \in M \mid I_j \sim I_i$, which enables co-referential mentions to be merged into a single vertex during hypergraph construction.

Hypergraph. A *hypergraph* is formally defined as $H = (V, E)$, where V is a finite set of vertices, and $E \subseteq \mathcal{P}(V) \setminus \{\emptyset\}$ is the set of *hyperedges*, with $\mathcal{P}(V)$ denoting the power set of V . Unlike an edge in a standard graph, which is strictly restricted to a binary pair of vertices, a hyperedge $e \in E$ can be an arbitrary non-empty subset of V .

The notations of the paper are summarized in Table 1.

Table 1: Notations.

Notation	Meaning
$p = (v, D); q; I$	vector data entry; query entry; top- k query instance
$T, E_{dep}, \text{Pos}, \text{Ent}, \text{CR}$	tokens, UD arcs, POS, NER and coreference mappings
$H = (V, E, \llbracket \cdot \rrbracket)$	hypergraph representation of vector data
$H_Q, H; V_Q, V; E_Q, E$	query/data hypergraphs; vertex sets; hyperedge sets
$u, v; e; C = (C.V, C.E)$	vertices; hyperedge; hyperedge cluster
$h_v; \text{HC}^{\sigma, b}; \mathcal{D}^\delta$	components of hyper simulation
$\Pi; I \models C_{\theta, \sigma, b, \delta}$	hyper simulation relation; query-level consistency
$\theta, \sigma, b, \delta; m$	similarity thresholds; search and cluster-size bounds

3 Hypergraph Representation of Vector Data

This section presents a hypergraph-based representation that captures consistencies in vector data. Section 3.1 gives the formal definition of a hypergraph on vector data entries, and Section 3.2 describes how to construct such a hypergraph from a document.

3.1 Definition of Hypergraph on Vector Data

Given a vector database entry, we represent its corresponding document D as a hypergraph. **A relation in natural language documents may involve multiple entities that should be jointly considered during consistency checking [79, 116]. For example, the “transfer” relation in Q_2 involves the board, the transferred object, and the destination. Hypergraphs naturally represent such multi-way relations via hyperedges that connect all related entities, which enables the joint comparison of entities.** Let $T = \mathcal{T}(D)$ be the token sequence. We define vertices, hyperedges, and hypergraphs as follows.

Vertex. A vertex u models an entity, predicate or attribute extracted from the corresponding document D . To anchor this vertex to the raw text, we employ the value function $\llbracket u \rrbracket \in \mathcal{P}(T)$, which maps vertex u to a token span within the token sequence T .

Hyperedge. A hyperedge e directly connects the vertices that participate in one relation. We model it as an n -ary relation $e = (u_1, u_2, \dots, u_n)$, where $n \geq 2$. The value function $\llbracket e \rrbracket \in \mathcal{P}(T)$ maps the hyperedge to the token spans that express this relation in D .

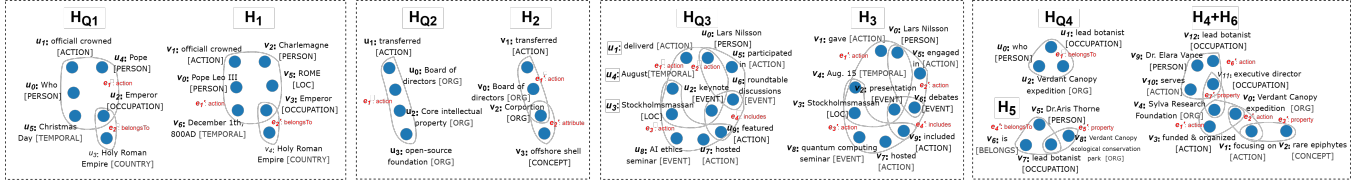


Figure 4: Hypergraph representations of a query Q and four candidate documents D_1 – D_6 .

Hypergraph. Integrating the above components, we define the hypergraph representation of vector data as a triplet $H = (V, E, [\cdot])$, where V denotes the finite set of vertices, E denotes the set of hyperedges connecting the vertices in V , and $[\cdot] : V \cup E \rightarrow \mathcal{P}(T)$ is the value function mapping vertices and hyperedges to their anchoring token spans. The mapped token subset $\llbracket u \rrbracket \in \mathcal{P}(T)$ preserves the relative positional ordering of the sequence T .

Example 2: Figure 4 illustrates the hypergraphs for the vector data entries from Figure 1. Where H_{Q1} – H_{Q4} (resp. H_1 – H_6) represent the hypergraphs for the queries Q_1 – Q_4 (resp. data D_1 – D_6). In the hypergraph, a vertex represents an entity with its type. For instance, the vertices in H_{Q1} include an entity u_2 : “Emperor” of type OCCUPATION, an entity u_4 : “Pope” of type PERSON, and a predicate u_1 : “crowned”. Based on coreference resolution, a vertex can span non-contiguous tokens that refer to the same entity. In H_{Q3} , $CR(\text{“the venue”}) = CR(\text{“Stockholmsmässan”})$ that refer to the same facility and thus represented by a vertex u_3 of type FAC. The hyperedges model n -ary relations among these vertices. For example, the 5-ary hyperedge $e_1 = (u_0, u_1, u_2, u_4, u_5)$ in H_{Q1} groups the queried person, coronation action, title, Pope, and date in one relation, which supports the joint checking of these entities during inconsistency detection. The value function $[\cdot]$ maps these vertices and hyperedges to their corresponding tokens. Specifically, it maps a vertex to its token span (e.g., $\llbracket u_5 \rrbracket = \text{“Christmas Day”}$) and a hyperedge to the token sequence that forms the relationship (e.g., $\llbracket e_1 \rrbracket = \text{“who was officially crowned Emperor by the Pope on Christmas Day”}$). By sharing common vertices across different hyperedges, these components form the complete hypergraph structure. $\square$

3.2 Hypergraph Construction

We introduce the construction of the hypergraph $H = (V, E, [\cdot])$ from a vector data entry (v, D) . There are two main challenges when constructing a hypergraph from a document. First, the document does not specify which text spans should be represented as a vertex or which mentions refer to the same entity. We address this by combining rule-based extraction, NER and coreference resolution to identify vertices. Second, determining hyperedges among the vertices requires finding groups of entities that jointly express a relation, while enumerating all possible vertex combinations is infeasible. We therefore leverage dependency parsing to identify groups of vertices connected through paths over E_{dep} and construct hyperedges from these groups to represent the relations expressed in the document. Given the UD’s combined with NER $(T, E_{dep}, \text{Pos}, \text{Ent})$, Algorithm 1 constructs the hypergraph representation.

Algorithm 1. HypergraphBuild presents the pseudocode of the hypergraph construction process. Given a vector data entry (v, D) , the algorithm computes the hypergraph representation defined in

Algorithm 1: HypergraphBuild

Input: Vector data entry (v, D) , Coreference resolution model CR
Output: Hypergraph $H = (V, E, [\cdot])$

- 1 Get universal dependencies with NER $(T, E_{dep}, \text{Pos}, \text{Ent})$ of D ;
- 2 $V \leftarrow \text{VerticesDeduction}(T, E_{dep}, \text{Pos}, \text{Ent})$;
- 3 $\tilde{V} \leftarrow \{\text{CR}(v) \mid v \in V\}$;
- 4 $\psi \leftarrow \text{LinksMap}(\tilde{V}, E_{dep})$;
- 5 $E \leftarrow \text{HyperedgesConstruction}(\tilde{V}, \psi)$;
- 6 **return** $H = (V, E, [\cdot])$

Function VerticesDeduction $(T, E_{dep}, \text{Pos}, \text{Ent})$ */

- 7 $C \leftarrow (\emptyset, T)$;
- 8 **while** $C = (I, R)$ matches a rule ϕ in $\mathcal{R}_1$ **and** $R \neq \emptyset$ **do**
- 9 $C \leftarrow \phi(C) \text{ /* } \phi : (I, R) \Rightarrow (I', R') \text{ */}$
- 10 Set C_n as $(I_{\text{final}}, \emptyset)$, I_{final} is the final I when the while loop exits;
- 11 **return** $V = \{\text{Vertex}(v) \mid v \in I_{\text{final}}\}$

Function HyperedgesConstruction $(\tilde{V}, \psi)$ */

- 12 $E \leftarrow \emptyset$, not_leafs $\leftarrow \{u \in \tilde{V} \mid \psi(u) \neq \emptyset\}$;
- 13 **foreach** vertex $u \in \text{not_leafs}$ **do**
- 14 Let $\{v_1, \dots, v_k\}$ be the vertices in $\psi(u)$;
- 15 $E \leftarrow E \cup \{(u, v_1, \dots, v_k)\}$;
- 16 **return** E

Section 3.1 as $H = (V, E, [\cdot])$. We now explain the functions and special notations used in the algorithm.

Function VerticesDeduction. Given the universal dependencies and NER annotations $(T, E_{dep}, \text{Pos}, \text{Ent})$ of a document D , this function extracts the vertex set V via a rule-based deduction system. The deduction system operates on a state denoted as $C = (I, R)$, where I represents the set of currently extracted token spans, and $R \subseteq T$ is the set of unresolved tokens. The deduction initiates from an empty state $C_0 = (\emptyset, T)$ and iteratively applies a set of deduction rules $\mathcal{R}_1$ to transition between states (i.e., $C_i \Rightarrow C_{i+1}$). These rules utilize part-of-speech tags, entity types, and universal dependencies to group tokens into spans (see [1]).

Our algorithm considers that a hyperedge $(v_1, \dots, v_k), k > 1$ exists if and only if the following conditions hold: (1) all vertices $v_1, \dots, v_k \in V$ are distinct, forming a set $S = \{v_1, \dots, v_k\}$; (2) for each vertex $v' \in \{v_2, \dots, v_k\}$, there exist tokens $t \in v_1$ and $t' \in v'$ such that there exists a path from t to t' in the universal dependency (T, E_{dep}) ; (3) no other vertex set $S' = \{v'_1, \dots, v'_m\}$ that satisfies both (1) and (2) properly contains S , ensuring S is maximal. We define a *links map* to facilitate the hyperedges construction.

Links Map. The links map $\psi : \tilde{V} \rightarrow \mathcal{P}(\tilde{V})$ is a mapping among the vertices in $\tilde{V}$ based on the dependencies E_{dep} . For each vertex $u \in \tilde{V}$ and $v \in \psi(u)$, it holds that $u \neq v$ and there exists tokens $t \in \llbracket u \rrbracket$ and $t' \in \llbracket v \rrbracket$ such that there exists a path from t to t' in the universal dependency (T, E_{dep}) . We compute the links map ψ by a

breadth-first search (BFS) process on the dependencies E_{dep} among the vertices in $\bar{V}$ (line 4).

Function HyperedgesConstruction. Given the vertices $\bar{V}$ and the links map ψ as input, the function constructs the hyperedges E among the vertices in $\bar{V}$. The function first initializes the hyperedges set E as empty (line 14). It then identifies the non-leaf vertices in $\bar{V}$, namely vertices u with $\psi(u) \neq \emptyset$ (line 15). For each non-leaf vertex u , it forms a hyperedge by concatenating u with all vertices in $\psi(u)$ (lines 16-18). Finally, it returns the hyperedges set E (line 19).

With the above functions and notations explained, we now describe the hypergraph construction process in Algorithm 1. The algorithm first obtains the UD combined with NER ($T, E_{dep}, \text{Pos}, \text{Ent}$) of document D by invoking a dependency parser method (line 1). It then extracts the vertices V by calling the function VerticesDeduction (line 2). After that, coreference resolution is applied to merge the vertices in V referring to the same, forming the set of vertices $\bar{V}$ (line 3). Next, it computes the links map ψ among the vertices in $\bar{V}$ based on the dependencies E_{dep} (line 4), and constructs the hyperedges E among the vertices in $\bar{V}$ by invoking the function HyperedgesConstruction (line 5). Finally, the algorithm returns the constructed hypergraph $H = (V, E, \llbracket \cdot \rrbracket)$ (lines 6).

Complexity. The size of token sequence T extracted from document D is linear in the document size $|D|$, namely $|T| = O(|D|)$. Dependency parsing, NER, and coreference resolution typically run in no more than $O(|T|^2)$ time. The deduction process in VerticesDeduction costs no more than $O(|T|^2)$ time. The computation of links map ψ by BFS on dependencies E_{dep} takes $O(|T| + |E_{dep}|)$ time, which simplifies to $O(|T|)$ since $|E_{dep}|$ is linear in $|T|$. Finally, the function HyperedgesConstruction takes no more than $O(|V|^2)$ time, which is $O(|T|^2)$ since $|V| \leq |T|$. Overall, the hypergraph construction algorithm runs in $O(|T|^2) = O(|D|^2)$.

Example 3: The rules in $\mathcal{R}_1$ construct vertices by identifying their corresponding spans in a document. Here we illustrate two representative rules over a state (I, R) as examples, where I contains the spans extracted so far, R consists of the remaining tokens, and $s \oplus t$ denotes the contiguous extension of span s with token t .

(1) The rule ENTITY: $\text{Ent}(s) \neq \perp \Rightarrow (I, R) \mapsto (I \cup \{s\}, R \setminus s)$ means that if Ent recognizes span s as an entity, s will be added to the extracted spans in I and its token will be removed from R , which constructs an entity vertex. For example, $\text{Ent}(\text{"Scholar Nilsson"}) = \text{PERSON}$ constructs vertex u_0 with $\llbracket u_0 \rrbracket = \text{"Scholar Nilsson"}$.

(2) The rule EVENT_{compound}: $s \in I, t \in R, \exists t' \in s : (t', \text{compound}, t) \in E_{dep} \Rightarrow (I, R) \mapsto ((I \setminus \{s\}) \cup \{s \oplus t\}, R \setminus \{t\})$ means that if an extracted span s contains a token t' connected to a remaining token t by a compound dependency, t will be incorporated into s to construct an event vertex. For "AI ethics seminar", $\mathcal{R}_1$ first extracts "seminar". Then, it applies EVENT_{compound} twice, yield "seminar" $\mapsto$ "ethics seminar" $\mapsto$ "AI ethics seminar", using the compound arcs that connect "seminar", "ethics" and "AI". This merged span constitutes vertex u_8 with $\llbracket u_8 \rrbracket = \text{"AI ethics seminar"}$.

To construct hyperedges, a breadth-first search traverses the dependency arcs E_{dep} to compute the links map ψ . For the predicate $u_1 = \text{"delivered"}$, the search locates its dependent vertices through paths like $u_1 \xrightarrow{\text{nsubj}} u_0$ and $u_1 \xrightarrow{\text{prep}} \text{"at"} \xrightarrow{\text{pobj}} u_3$, resulting in the mapping $\psi(u_1) = \{u_0, u_2, u_3, u_4\}$, that construct

$e_1 = (u_1, u_0, u_2, u_3, u_4)$. Finally, the HyperedgesConstruction function converts each non-empty mapping in ψ into a hyperedge, yielding the hypergraph H_{Q_3} . $\square$

4 Consistency of Vector Data

This section formalizes consistency for vector data. Section 4.1 introduces *hyper simulation*, a recursive relation that captures structural and semantic match between two hypergraphs. Section 4.2 then defines when a query and its retrieved documents are consistent.

4.1 Definitions of Hyper Simulation

Challenge. Graph simulation recursively matches vertices through edges, where a matched vertex pair is verified by checking whether their adjacent vertices are also matched. However, this strategy cannot directly adapt to hypergraphs, since a hyperedge may connect multiple entities involved in one relation, which should be checked together rather than independently. Thus, hypergraph simulation needs to determine which relations between two hypergraphs should be compared, how the entities involved in these relations are matched, and how inconsistencies among these matches are recursively propagated. To address the above challenge, we extend graph simulation to hypergraphs by introducing hyper simulation. Given two hypergraphs $H_1 = (V_1, E_1, \llbracket \cdot \rrbracket)$ and $H_2 = (V_2, E_2, \llbracket \cdot \rrbracket)$, a hyper simulation is a binary relation $\Pi \subseteq V_1 \times V_2$. Intuitively, $(u, v) \in \Pi$ means that u and v do not conflict, and the relations containing them can also be matched via vertex pairs that belong to Π . To determine whether a vertex pair satisfies this condition, we need to check whether the two vertices conflict, identify the related sets of hyperedges that should be compared, and pair the vertices within these sets. We define these three operations below.

Function h_v . Given two vertices $u \in V_1$ and $v \in V_2$, we define a binary function $h_v : V_1 \times V_2 \rightarrow \{\text{true}, \text{false}\}$ to determine whether they are in conflict. The predicate $h_v(u, v) = \text{true}$ means that u and v are not conflicting, and only such pairs may belong to Π .

A relation in a document may be represented by multiple hyperedges. Therefore, the hyper simulation compares sets of hyperedges, called *hyperedge clusters*, rather than individual hyperedges.

Hyperedge Cluster. A hyperedge cluster denotes a set of adjacent hyperedges. Formally, we define a cluster as a tuple $C = (V, E)$, written as $C = [e_1 + e_2 + \dots + e_m]$, where $C.E = \{e_1, e_2, \dots, e_m\} \subseteq E$ constitutes a subset of hyperedges, and $C.V := \{v \in e \mid e \in C.E\}$ represents the union of vertices contained in these hyperedges.

Given a vertex pair (u, v) as input, the function $\text{HC}^{\sigma, b}(u, v)$ returns the similar hyperedge cluster pairs (C_1, C_2) such that $u \in C_1.V$ and $v \in C_2.V$. The threshold σ specifies the lowest similarity between C_1 and C_2 , while b limits the number of candidates examined during the search (see Section 5.1).

We represent the relations encoded by their vertices and hyperedges as relational expressions, rather than using a type label for each cluster. For instance, considering the hyperedge clusters $C_2 = [e_2]$ and $C_3 = [e_3 + e_4]$ within H_{Q_3} of Figure 4, we define their semantic representations as $\llbracket C_2 \rrbracket = \text{participate}(\text{PERSON}, \text{EVENT})$ and $\llbracket C_3 \rrbracket = \text{host}(\text{FAC}, \text{EVENT\#1}, \text{TEMPORAL}) \wedge \text{feature}(\text{EVENT\#1}, \text{EVENT\#2}, \text{EVENT\#3})$, respectively.

$\mathcal{D}$ -match. Given a hyperedge cluster pair (C_u, C_v) , a $\mathcal{D}$ -match pa-

parameterized by δ , denoted by $\mathcal{D}^\delta(C_u, C_v)$, is a partial 1-to-1 relation $\mathcal{D}^\delta(C_u, C_v) \subseteq C_u.V \times C_v.V$. A vertex pair is included only if its similarity score exceeds δ . Intuitively, $\mathcal{D}^\delta(C_u, C_v)$ lists the vertex pairs that should be compared between C_u and C_v .

Example 4: We illustrate the functions h_v , $\text{HC}^{\sigma,b}$, and the $\mathcal{D}$ -match using the hypergraphs H_{Q_3} and H_3 . The function h_v checks whether the values of two vertices conflict. The function $\text{HC}^{\sigma,b}$ identifies hyperedge clusters that express similar relations. It pairs the hyperedge cluster $C_3 = [e_3 + e_4]$ from H_{Q_3} with $C'_3 = [e'_3 + e'_4]$ from H_3 . We define $\llbracket C_3 \rrbracket = \text{host}(\text{FAC}, \text{EVENT\#1}, \text{TEMPORAL}) \wedge \text{include}(\text{EVENT\#1}, \text{EVENT\#2}, \text{EVENT\#3})$. Together with $\llbracket C_3 \rrbracket$ defined above, the representations of $\llbracket C_3 \rrbracket$, $\llbracket C'_3 \rrbracket$ describe a facility hosting a seminar at a specified time, where the seminar contains two other events. Hence, $\text{HC}^{\sigma,b}$ pairs C_3 with C'_3 . For this cluster pair, the $\mathcal{D}$ -match returns the 1-to-1 vertex pairs $\mathcal{D}^\delta(C_3, C'_3) = \{(u_3, v_3), (u_4, v_4), (u_8, v_8), (u_2, v_2), (u_6, v_6)\}$. These pairs compare the venue, time, seminar, keynote/presentation, and discussions/debates in the two documents, respectively. In particular, the $\mathcal{D}$ -match pairs u_8 with v_8 since both vertices represent the seminar in their respective clusters. However, $\llbracket u_8 \rrbracket = \text{"AI ethics seminar"}$ and $\llbracket v_8 \rrbracket = \text{"quantum computing seminar"}$ make inconsistent claims about the seminar topic. Therefore, $h_v(u_8, v_8) = \text{false}$, which exposes the conflict between the two clusters. $\square$

Hyper Simulation. Given h_v , HC , $\mathcal{D}$ and parameters σ, b , and δ , a relation $\Pi \subseteq V_1 \times V_2$ is a *hyper simulation* between H_1 and H_2 if every pair $(u, v) \in \Pi$ satisfies the following conditions:

- (1) $h_v(u, v) = \text{true}$; and
- (2) for each hyperedge cluster pair $(C_u, C_v) \in \text{HC}^{\sigma,b}(u, v)$, there exists a $\mathcal{D}$ -match $\mathcal{D}^\delta(C_u, C_v) \subseteq C_u.V \times C_v.V$ such that:
 - (a) $(u, v) \in \mathcal{D}^\delta(C_u, C_v)$; and
 - (b) for every $(u', v') \in \mathcal{D}^\delta(C_u, C_v)$, we have $(u', v') \in \Pi$.

We say that H_1 is *matched* to H_2 by hyper simulation Π with respect to $h_v, \text{HC}, \mathcal{D}, \sigma, b, \delta$.

Intuitively, $(u, v) \in \Pi$ means that u and v form a consistent match, where their values do not conflict and the entities related to them are also paired consistently. If one of these entity pairs conflicts, the relation containing (u, v) may become inconsistent, which may further invalidate other vertex pairs.

Example 5: We evaluate the hyper simulation Π between H_{Q_3} and H_3 . By Condition (1), initial candidates in Π must satisfy $h_v(u, v) = \text{true}$, which retains (u_i, v_i) for $i \in \{0, \dots, 7, 9\}$. Note that $(u_8, v_8) \notin \Pi$ since $h_v(u_8, v_8) = \text{false}$. By Condition (2), every pair in Π requires a valid $\mathcal{D}$ -match where all mapped vertices also belong to Π . For clusters C_3 and C'_3 , the mapping $\mathcal{D}^\delta(C_3, C'_3)$ contains (u_8, v_8) . Since $(u_8, v_8) \notin \Pi$, this mapping violates Condition (2)(b). Consequently, the remaining pairs (u_2, v_2) , (u_3, v_3) , (u_4, v_4) and (u_6, v_6) in this match, lose their support, fail Condition (2)(b), and are removed from Π . This removal recursively invalidates $\mathcal{D}^\delta([e_1], [e'_1]) = \{(u_0, v_0), (u_1, v_1), (u_2, v_2), (u_3, v_3), (u_4, v_4)\}$, leading to the elimination of (u_0, v_0) and (u_1, v_1) . Similarly, the deletion of (u_0, v_0) and (u_6, v_6) invalidates $\mathcal{D}^\delta([e_2], [e'_2]) = \{(u_0, v_0), (u_5, v_5), (u_6, v_6)\}$, eliminating (u_5, v_5) . Thus, all candidate pairs recursively violate Condition (2), yielding $\Pi = \emptyset$ (see [1]). $\square$

Uniqueness & Maximality. For hypergraphs H_1 and H_2 , a hyper simulation Π is *maximum* if, for any other hyper simulation Π' , it holds

that $\Pi' \subseteq \Pi$. We present the following theorem regarding the existence and uniqueness of the maximum hyper simulation.

Theorem 1: For any hypergraphs H_1 and H_2 , there exists a unique maximum hyper simulation Π with respect to $h_v, \text{HC}, \mathcal{D}, \sigma, b, \delta$. $\square$

Proof sketch: The existence is guaranteed by the finite state space $V_1 \times V_2$, allowing a maximum simulation to be found via fixed-point iterations. Uniqueness is proven by contradiction: since the conditions of hyper simulation are verified point-wise, the union of any two valid simulations remains a valid simulation. Thus, assuming two distinct maximums yields a strictly larger union, contradicting their maximality (see [1]). $\square$

4.2 Consistency in Query

We now formally define vector data consistency within a query process by leveraging the notion of hyper simulation.

Top-k Vector Query. Let $q = (v_0, D_0, H_Q)$ be a query vector data entry enriched with a hypergraph. Consider a vector database $\mathcal{V}^d$ and the set of retrieved top- k vector data entries $\mathcal{V}^d(q, k, \text{sim}) = (p_1 = (v_1, D_1, H_1), \dots, p_k = (v_k, D_k, H_k))$, where $\text{sim} : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ is a similarity metric and each v_i represents one of the top- k most similar vectors to v_0 under sim . We define the tuple $I = (q; p_1, \dots, p_k)$ as an instance of a top- k vector query.

Consistency. Given a top- k vector query instance I , we consider each retrieved data entry $p_i = (v_i, D_i, H_i)$ for $1 \leq i \leq k$. Let $\text{Ids} := \{1 \leq i \leq k \mid \text{sim}(v_0, v_i) > \theta\}$ and $H = \bigcup_{i \in \text{Ids}} H_i$. We define the instance I as *consistent* under parameters $\theta, \sigma, b, \delta$ if there exists a maximum hyper simulation Π with respect to $h_v, \text{HC}, \mathcal{D}, \sigma, b, \delta$ between H_Q and H , and for every vertex $u \in H_Q.V$ there exists a vertex $v \in H.V$ satisfying $(u, v) \in \Pi$. This relationship is denoted by $I \models C_{\theta, \sigma, b, \delta}$. If the query fails to satisfy this condition, we designate the instance I as *inconsistent*, which is denoted by $I \not\models C_{\theta, \sigma, b, \delta}$.

Intuitively, I is consistent if every vertex in $H_Q.V$ is paired by the hyper simulation with a vertex in $H.V$, where H is defined from the retrieved results whose similarities exceed the threshold θ .

Document Marking. In order to enhance the performance of downstream tasks such as RAG, we annotate each retrieved document based on the behavior and results of the hyper simulation between the query and the documents. Specifically, retrieved documents are divided into four categories: (1) a *consistent* document matches entities paired in Π and the relations in the query through hyperedge clusters computed by HC ; (2) a *useless* document matches certain entities in Π but no relation by HC ; (3) a *conflict* document contains a vertex pair such that $h_v(u, v) = \text{true}$, but the pair is subsequently removed during the computation of Π , indicating a contradiction with the query; and (4) an *irrelevant* document does not satisfy the condition for *conflict*, and no entity in the document is matched with an entity in Π . The latter three categories are treated as *inconsistent*. Based on this taxonomy, we annotate documents with its category and a reason, based on the behavior and results of hyper simulation (see Appendix E in [1] for more details).

5 Implementation of Hyper Simulation

This section presents the implementation of hyper simulation. Section 5.1 describes the realization of the key components: the func-

tion h_v , the hyperedge cluster, and $\mathcal{D}$ -match. Section 5.2 then gives the algorithm HyperMatch that computes the hyper simulation relation between two hypergraphs.

5.1 Components of Hyper Simulation

We implement the components introduced in Section 4.1, which include the function h_v , the *hyperedge cluster*, and the $\mathcal{D}$ -match.

Function h_v . Given two vertices $u \in V_1$ and $v \in V_2$, the predicate $h_v(u, v)$ evaluates to true if the textual values of u and v do not exhibit semantic contradictions. The evaluation operates in two phases: (1) We first ascertain whether $\llbracket u \rrbracket$ and $\llbracket v \rrbracket$ belong to compatible entity types, utilizing NER labels enriched by WordBank [12]. (2) If the types of entities are compatible, we employ a model of natural language inference (NLI) [55] to determine if there is a logical contradiction between $\llbracket u \rrbracket$ and $\llbracket v \rrbracket$. The function returns true only if both checks pass; otherwise, it returns false.

Hyperedge Cluster HC. The calculation of hyperedge clusters HC is performed by the HCCalc algorithm (Algorithm 2). The algorithm takes as input a query hypergraph $H_Q = (V_Q, E_Q, \llbracket \cdot \rrbracket)$, a data hypergraph $H = (V, E, \llbracket \cdot \rrbracket)$, a beam bound b , a maximum hyperedge cluster size m , and a semantic similarity threshold σ . It outputs the set of hyperedge cluster pairs $\text{HC}^{\sigma, b}$.

For each hyperedge cluster C_Q , HCCalc examines each distinct pair $u, u' \in C_Q.V$ and selects at most b vertices in $H.V$ for each vertex. For selected pair $v, v' \in H.V$, the algorithm constructs C_D using the shortest path connecting v and v' in H , and retains (C_Q, C_D) when C_D satisfies the size bound m and similarity threshold σ .

The algorithm begins by initializing $\text{HC}^{\sigma, b}$ as empty and generating the set of query hyperedge clusters C_Q from $H_Q.E$ via a predefined rule set $\mathcal{R}_2$ (lines 1-2) (see [1]). It then evaluates each query cluster $C_Q \in C_Q$ by iterating over its distinct vertex pairs u, u' (lines 3-4). To search hyperedge clusters in H that are similar to C_Q , the algorithm picks distinct vertex pairs (v, v') in H that satisfy a metric to measure potential contradictions between similar entities [118]:

$$\text{likely}(\vec{u}, \vec{v}) = \cos(\vec{u}, \vec{v}) + \frac{\alpha}{\sqrt{d} - 1} \left(\sqrt{d} - \frac{\|\vec{u} - \vec{v}\|_1}{\|\vec{u} - \vec{v}\|_2} \right) \quad (1)$$

where $\vec{u}$ and $\vec{v}$ denote the embeddings of $\llbracket u \rrbracket$ and $\llbracket v \rrbracket$ derived from a model based on contrastive learning [118], respectively, and d denotes the embedding dimension. The predicate $\text{likely}^b(u, v) = \text{true}$ indicates that v as one of the top- b higher $\text{likely}(\cdot, \cdot)$ score vertices for u . For each valid pair, a candidate data cluster C_D is constructed via the shortest path between v, v' in H (lines 5-6).

Finally, if $|C_D.E| \leq m$ and the similarity scored by ML model $\mathcal{M}_1$ meets the threshold σ (line 7), the matched pair (C_Q, C_D) is added to $\text{HC}^{\sigma, b}$ (line 8). After evaluating the bounded candidates, the algorithm concludes by returning the populated set $\text{HC}^{\sigma, b}$ (line 9). We implement $\mathcal{M}_1$ with an NLI model [55].

Parameters α and m . Drawing upon empirical evidence, we directly set the parameters α and m . For open-domain knowledge applications, configuring $\alpha = 2.36$ optimally retrieves potential contradictions [118]. Furthermore, inter-document conflicts exhibit locality. Logical contradictions within texts are frequently embedded in combinations of a minimal number of relationships [72, 85]. Therefore,

we set $m = 5$ to concentrate on local document contradictions.

Example 6: Taking $\sigma = 0.7, b = 3$, we trace the execution of HCCalc using H_Q and H_3 , focusing on the derivation of (C_3, C'_3) . First, the algorithm applies the predefined rule $\mathcal{R}_2$ (line 2) to merge the query hyperedges e_3 and e_4 , which are linked by a relative clause dependency (relcl), yielding the cluster $C_3 = [e_3 + e_4]$. Next, by iterating over vertex pairs in C_3 , such as u_3 and u_6 , it identifies their likely counterparts v_3 and v_6 in H_3 that satisfy $\text{likely}^b(\cdot, \cdot) = \text{true}$. The algorithm then extracts the shortest path connecting v_3 and v_6 within the H_3 (line 6). This path traverses $v_3, e'_3, v_9, e'_4, v_6$, instantiating the candidate data cluster $C'_3 = [e'_3 + e'_4]$. Finally, it evaluates the cosine similarity between the clusters $\cos(\llbracket C_3 \rrbracket, \llbracket C'_3 \rrbracket) = 0.987 > \sigma$ (line 7), the pair (C_3, C'_3) is appended to the output set $\text{HC}^{\sigma, b}$. $\square$

Theorem 2: Given hypergraphs $H_Q = (V_Q, E_Q, \llbracket \cdot \rrbracket)$, $H = (V, E, \llbracket \cdot \rrbracket)$, and bounds b, m , algorithm HCCalc computes the set of hyperedge cluster pairs $\text{HC}^{\sigma, b}$ in $O(|E_Q|(|V| + |E|))$ time. Furthermore, the size of the output set $|\text{HC}^{\sigma, b}|$ is asymptotically bounded by $O(|E_Q|)$. $\square$

Proof sketch: Based on the 2-section *viz.*, clique expansion property of hypergraphs [14], enumerating all vertex pairs (u, u') in the query hyperedge clusters operates in $O(|E_Q|\Delta_E^2)$ time, where $\Delta_E = \max_{e \in E_Q} |e|$ is the maximum hyperedge size. Combined with the branch bound b of the function likely^b , the total number of candidate data vertex pairs (v, v') is bounded by $|P| = O(|E_Q|\Delta_E^2 b^2)$. Constructing the corresponding data clusters C_D for these $|P|$ queries via BFS on the data hypergraph H requires $O(|P|(|V| + |E|\Delta_E))$ time. By treating b and Δ_E as constants in the context of natural language processing, the overall computational complexity simplifies to $O(|E_Q|(|V| + |E|))$. Furthermore, since each candidate evaluation yields at most one matching pair, the size of the output set $\text{HC}^{\sigma, b}$ is strictly limited by the total number of candidate queries, concluding that $|\text{HC}^{\sigma, b}| \leq |P| = O(|E_Q|)$ (see [1]). $\square$

HCCalc bounds the construction of hyperedge clusters. Specifically, the 2-section property [14] of hypergraphs restricts candidate generation to vertex pairs within query hyperedges, the branch bound b limits the number of candidate vertices considered for each query vertex, and the parameter m bounds the size of each constructed cluster. Moreover, HCCalc constructs clusters via the shortest path between selected vertices rather than enumerating combinations of paths. Thus, the number of examined clusters is bounded, which leads to the complexity given in Theorem 2.

$\mathcal{D}$ -match. Following the generation of valid hyperedge cluster pairs $(C_Q, C_D) \in \text{HC}^{\sigma, b}$, the algorithm must establish a precise 1-to-1 vertex correspondence, denoted as $\mathcal{D}^\delta(C_Q, C_D)$, between the $C_Q.V$ and $C_D.V$. The $\mathcal{D}$ -match considers vertices of the same entity type and selects the highest-scoring pairs, while enforcing that each vertex appears in at most one pair.

First, to minimize the search space for comparisons, the algorithm applies a filter based on Named Entity Recognition. It constructs an initial candidate relation R containing vertex pairs $(u, v) \in C_Q.V \times C_D.V$. A pair (u, v) is admitted into R if they share identical entity types, formulated as $\text{Ent}(\llbracket u \rrbracket) = \text{Ent}(\llbracket v \rrbracket)$.

Second, for the surviving candidate pairs in R , the algorithm evaluates their contextual alignment to quantify the structural correspondence. Given a predefined threshold δ , a similarity score is

Algorithm 2: HCCalc

Input: Hypergraphs H_Q, H , branch bound b , maximum hyperedge cluster size m , similarity threshold σ

Output: Set of hyperedge cluster pairs $\text{HC}^{\sigma,b}$

```

1  $\text{HC}^{\sigma,b} \leftarrow \emptyset$ ;
2 Calculate  $C_Q$  from  $E_Q$  over rule set  $\mathcal{R}_2$ ;
3 for  $C_Q \in C_Q$  do
4   foreach distinct  $u, u' \in C_Q.V$  do
5     foreach distinct  $(v, v')$  where  $\text{likely}^b(u, v) = \text{true}$  and
6        $\text{likely}^b(u', v') = \text{true}$  do
7        $C_D$  is constructed by the shortest path between  $v$  and
8          $v'$  in  $H$ ;
9       if  $\mathcal{M}_1(\llbracket C_Q \rrbracket, \llbracket C_D \rrbracket) \geq \sigma$  and  $|C_D.E| \leq m$  then
10         $\text{HC}^{\sigma,b} \leftarrow \text{HC}^{\sigma,b} \cup \{(C_Q, C_D)\}$ ;
9 return  $\text{HC}^{\sigma,b}$ ;
```

computed for each pair (u, v) by aggregating the textual similarities of the paths connecting them to their respective neighbors. Formally, the score is defined as:

$$\text{Score}(u, v) = \frac{1}{|R \setminus \{u, v\}|} \sum_{(u', v') \in R \setminus \{u, v\}} \mathcal{M}_2(\llbracket [u, u'] \rrbracket, \llbracket [v, v'] \rrbracket)$$

where $R \setminus \{u, v\} := \{(u', v') \in R \mid u' \neq u, v' \neq v\}$. Here, $\llbracket [u, u'] \rrbracket$ represents the textual span derived from the path between u and u' within the query cluster. To distinguish the two endpoints when comparing their connecting paths, the vertices u and u' are replaced with indexed entity-type markers (e.g., PERSON#1). The same replacement is applied to v and v' in the data cluster. If $\text{Score}(u, v) > \delta$, where $\mathcal{M}_2$ employs an embedding model [126], pair (u, v) is promoted to the refined candidate set for the final matching phase.

Finally, since a valid $\mathcal{D}$ -match strictly requires a 1-to-1 mapping, the algorithm resolves any structural conflicts induced by the scoring phase. If a vertex u is associated with multiple candidate vertices v (or symmetrically, multiple u mapping to a single v), a greedy bipartite resolution strategy is applied. The algorithm retains only the specific pair (u, v) that maximizes $\text{Score}(u, v)$, pruning all mutually exclusive alternatives. The surviving vertex pairs establish the finalized $\mathcal{D}$ -match $\mathcal{D}^\delta(C_Q, C_D)$.

Example 7: Taking $\delta = 0.7$ to illustrate the computation process between H_{Q_3}, H_3 and examine the $\mathcal{D}$ -match $\mathcal{D}^\delta(C_3, C'_3) = \{(u_3, v_3), (u_8, v_8), (u_4, v_4), (u_2, v_2), (u_6, v_6)\}$ between H_{Q_3} and H_3 established in Example 4. For a vertex $u_2 \in C_3.V$, the entity type matching generates the candidate pairs (u_2, v_2) , (u_2, v_6) , and (u_2, v_8) within R . The algorithm evaluates these candidates and yields $\text{Score}(u_2, v_2) = 0.799 > \delta$, $\text{Score}(u_2, v_6) = 0.744 > \delta$, and $\text{Score}(u_2, v_8) = 0.639 < \delta$. Since a valid $\mathcal{D}$ -match enforces a 1-to-1 mapping constraint, the process retains the pair that achieves the maximum score among the candidates that exceed the threshold. Consequently, the algorithm selects (u_2, v_2) and finalizes its inclusion such that $(u_2, v_2) \in \mathcal{D}^\delta(C_3, C'_3)$. $\square$

Parameters σ, b , and δ . We determine σ , b , and δ via random search [10]. For downstream tasks, we identify the optimal parameter configuration by performing a search over a partitioned validation set. Furthermore, we present a parameter sensitivity analysis in Section 6, demonstrating that the optimal intervals for

Algorithm 3: HyperMatch

Input: Query hypergraph $H_Q = (V_Q, E_Q, \llbracket \cdot \rrbracket)$, data hypergraph $H = (V, E, \llbracket \cdot \rrbracket)$, h_v , and thresholds σ, b, δ

Output: Hyper Simulation relation $\Pi \subseteq V_Q \times V$

```

1 Calculate the  $\text{HC}^{\sigma,b}$  by algorithm HCCalc;
2 Calculate the  $\mathcal{D}^\delta$  over  $\text{HC}^{\sigma,b}$ ;
3  $\Pi \leftarrow \{(u, v) \in V_Q \times V \mid h_v(u, v) = \text{true} \wedge \forall (C_u, C_v) \in \text{HC}^{\sigma,b}(u, v), (u, v) \in \mathcal{D}^\delta(C_u, C_v)\}$ ;
4  $A_{\text{cluster}} \leftarrow \bigcup_{(u, v) \in \Pi} \text{HC}^{\sigma,b}(u, v)$ ;
5  $\mathcal{V}_C \leftarrow \{(C_u, C_v) \in A_{\text{cluster}} \mid \mathcal{D}^\delta(C_u, C_v) \subseteq \Pi\}$ ;
6 foreach  $(C_u, C_v) \in A_{\text{cluster}}$  do
7    $D_{\text{cluster}}[(C_u, C_v)] \leftarrow \{(u, v) \in \Pi \mid (C_u, C_v) \in \text{HC}^{\sigma,b}(u, v)\}$ ;
8 foreach  $(u', v')$  do
9    $D_{\text{pair}}[(u', v')] \leftarrow \{(C_u, C_v) \in A_{\text{cluster}} \mid (u', v') \in \mathcal{D}^\delta(C_u, C_v)\}$ ;
10  $Q \leftarrow \{(u, v) \in \Pi \mid \text{HC}^{\sigma,b}(u, v) \not\subseteq \mathcal{V}_C\}$ ;
11  $\Pi \leftarrow \Pi \setminus Q$ ;
12 while  $Q \neq \emptyset$  do
13    $(u', v') \leftarrow \text{pop}(Q)$ ;
14   foreach  $(C_u, C_v) \in D_{\text{pair}}[(u', v')] \cap \mathcal{V}_C$  do
15      $\mathcal{V}_C \leftarrow \mathcal{V}_C \setminus \{(C_u, C_v)\}$ ;
16     foreach  $(u, v) \in D_{\text{cluster}}[(C_u, C_v)] \cap \Pi$  do
17        $\Pi \leftarrow \Pi \setminus \{(u, v)\}$ ;
18     push  $(u, v)$  to  $Q$ ;
19 return  $\Pi$ ;
```

these parameters exhibit consistency across various tasks.

5.2 Hyper Simulation Algorithm

Following the implementation of the components in Section 5.1, we now present the algorithm HyperMatch that computes the hyper simulation relation. The core objective of this phase is to efficiently determine the maximal set of valid vertex matches between two hypergraphs by recursively filtering out conflicting pairs.

Algorithm 3. HyperMatch details the computation of the hyper simulation relation. The algorithm takes as input a query hypergraph $H_Q = (V_Q, E_Q, \llbracket \cdot \rrbracket)$, a data hypergraph $H = (V, E, \llbracket \cdot \rrbracket)$, and the function h_v with parameters σ, b, δ . It outputs the maximum hyper simulation relation $\Pi \subseteq V_Q \times V$.

The algorithm begins with an initialization phase to establish the initial candidate matches and to construct the necessary auxiliary indexing structures. Prior to initializing the core sets, the algorithm pre-computes the hyperedge clusters $\text{HC}^{\sigma,b}$ via function HCCalc and the corresponding $\mathcal{D}$ -matches (lines 1-2). The tentative match relation Π is initialized by selecting all vertex pairs $(u, v) \in V_Q \times V$ that successfully pass the non-conflict evaluation, $h_v(u, v) = \text{true}$, and possess structurally sound $\mathcal{D}$ -matches for all associated hyperedge clusters (line 3).

To facilitate efficient recursive pruning, the algorithm establishes several auxiliary variables. A_{cluster} represents the universe of all hyperedge cluster pairs associated with the initial candidate matches in Π (line 4). The set $\mathcal{V}_C$ maintains the collection of currently valid hyperedge clusters, initialized to contain only those cluster pairs whose required $\mathcal{D}$ -match vertex mappings are fully present within the initial relation Π (line 5). Furthermore, two dictionary struc-

tures are constructed to map dependencies: $D_{cluster}$ maps a specific hyperedge cluster pair (C_u, C_v) to the set of vertex pairs $(u, v) \in \Pi$ that rely on it (lines 6-7), while D_{pair} provides a reverse mapping from a dependent vertex pair (u', v') to the hyperedge clusters that require it to satisfy their $\mathcal{D}$ -match constraints (lines 8-9).

Following the construction of the indices, the algorithm initializes a queue Q to manage the cascade deletion process. The queue Q serves as a buffer for invalid vertex pairs that fail the hyper simulation conditions and must be removed. It is initialized by identifying pairs $(u, v) \in \Pi$ that depend on hyperedge clusters no longer classified as valid *i.e.*, clusters not in $\mathcal{V}_C$. These identified pairs are loaded into Q (line 10) and excised from the working relation Π (line 11).

Finally, the algorithm executes a while loop to perform cascaded deletions, iteratively refining Π until a stable maximum hyper simulation is achieved. While the queue Q is not empty (line 12), the algorithm pops an invalid vertex pair (u', v') (line 13). Utilizing the D_{pair} index, it identifies all currently valid hyperedge clusters $(C_u, C_v) \in \mathcal{V}_C$ that depend on the removed pair (u', v') (line 14). Since a required structural dependency is now missing, these cluster pairs are invalidated and removed from $\mathcal{V}_C$ (line 15). Consequently, the algorithm queries the $D_{cluster}$ index to find any remaining candidate vertex pairs $(u, v) \in \Pi$ that relied on the newly invalidated clusters (line 16). These dependent vertex pairs are sequentially removed from Π (line 17) and pushed onto the queue Q (line 18) to recursively propagate the structural invalidation. Upon the depletion of Q , the cascaded pruning concludes, and the algorithm returns the finalized hyper simulation relation Π (line 19).

Example 8: We trace the execution of HyperMatch using H_{Q_3} and H_3 . During initialization (line 3), since $h_v(u_8, v_8) = \text{false}$, the pair (u_8, v_8) is excluded from the tentative relation Π . Consequently, the mapping for the cluster pair (C_3, C'_3) is incomplete, preventing its inclusion in the valid set $\mathcal{V}_C$. The algorithm then constructs the dependency indices $D_{cluster}$ and D_{pair} . Using these indices, it populates the queue $Q = \{(u_2, v_2), (u_3, v_3), (u_4, v_4), (u_6, v_6)\}$ with pairs depending on the invalid (C_3, C'_3) and removes them from Π . In the while loop (lines 12-18), the algorithm processes Q to refine Π . When an invalid pair like (u_2, v_2) is popped, querying D_{pair} reveals that the valid cluster $([e_1], [e'_1]) \in \mathcal{V}_C$ relies on it. Thus, $([e_1], [e'_1])$ is invalidated and removed from $\mathcal{V}_C$. The algorithm then consults $D_{cluster}([e_1], [e'_1])$ to locate its surviving dependent pairs, (u_0, v_0) and (u_1, v_1) , which are removed from Π and pushed onto Q . Similarly, dequeuing (u_6, v_6) invalidates $([e_2], [e'_2])$, forcing its dependent (u_5, v_5) out of Π into Q . This enqueue-dequeue cascade dismantles the unsupported topology until Q empties, yielding the finalized relation $\Pi = \emptyset$. $\square$

Theorem 3: Given a query hypergraph $H_Q = (V_Q, E_Q, \llbracket \cdot \rrbracket)$ and a data hypergraph $H = (V, E, \llbracket \cdot \rrbracket)$, along with the function h_v , and thresholds σ, b, δ , Algorithm 3 (HyperMatch) correctly computes the unique maximum hyper simulation relation Π in $O(|V_Q||V| + |E_Q|(|E| + |V|))$ time. $\square$

Proof sketch: The correctness of HyperMatch is rooted in fixed-point theory. The algorithm is guaranteed to converge to the maximum hyper simulation by preserving valid vertex pairs and recursively eliminating those that fail to satisfy $\mathcal{D}$ -match constraints or h_v evaluations. Furthermore, the algorithm achieves a time com-

plexity of $O(|V_Q||V| + |E_Q|(|E| + |V|))$ (see [1]). $\square$

Compared with graph simulation [57], which runs in $O((|V_Q| + |E_Q|)(|V| + |E|))$, HyperMatch retains the same time complexity.

6 Experimental Study

In this section, we experimentally evaluate the proposed hyper simulation for the consistency of the vector database using real-world datasets in RAG and conflict detection tasks. Our study aims to empirically verify the framework’s accuracy in mitigating semantic inconsistencies, alongside its computational efficiency and scalability with respect to varying problem sizes.

Experimental Setting. We start with the setting.

Datasets. We evaluate the proposed method on 11 publicly available datasets across two primary tasks: Retrieval-Augmented Generation (RAG) and Conflict Detection (CD). (1) For the RAG task, we utilize HotpotQA [121], ARC [19], MuSiQue [104], MultihopQA [58], and LegalQA [50]. (2) For the CD task, we employ MNLI [117], SNLI [13], ContractNLI [72], ECON [66], ConTRoL [76], and LegalNLI [50].

Baselines. We compare our approach against state-of-the-art baselines for both tasks. (1) RAG tasks: We compare against vanilla RAG, SparseCL [118], SENTLI [100], ContraDoc [74], BSIM [37], ParaSIM [40], CDIT [80], and HyDE [46]. (2) Conflict Detection tasks: We compare against DocInfer [85], DeBERTav3-base [55], RoBERTa-large [77], and ContraDoc [74].

Embedding Models & Substitute Models. In order to investigate the impact of embedding quality, we implement and compare Contriever [64], MPNet [98, 102], E5-base-v2 [112], and GTR-T5-base [87]. As for how the selection of ML model M_1 and M_2 used in HyperMatch influence the performance, we compare DeBERTa-NLI [55], Qwen3-Embedding-0.6B [126] and MiniLM-RR [113].

Metrics. We evaluate the performance of our proposed method on both the retrieval-augmented generation (RAG) and conflict detection tasks using three categories of metrics: (1) *Generation Metrics:* To evaluate the response quality in RAG tasks, we utilize the Answer F1-score. This metric computes the harmonic mean of precision and recall based on the token-level overlap between the generated response and the ground truth. (2) *Classification Metrics:* To assess the effectiveness of conflict detection, we measure Precision ($\text{Prec} = \frac{\text{TP}}{\text{TP} + \text{FP}}$), Recall ($\text{Rec} = \frac{\text{TP}}{\text{TP} + \text{FN}}$), and F1-score ($F_1 = \frac{2 \cdot \text{Prec} \cdot \text{Rec}}{\text{Prec} + \text{Rec}}$), where TP, FP, and FN denote the number of true positives, false positives, and false negatives, respectively. (3) *Retrieval Inconsistency Metrics:* In top- k vector query retrieval, GIR@ k denotes the fraction of the k retrieved documents that are manually confirmed inconsistent with respect to the query, while IDR@ k indicates the recall of those identified inconsistent by HyperMatch.

Implementation & Environment. Algorithm HypergraphBuild is implemented in Python, utilizing spaCy [60] for dependency parsing and NER. Furthermore, coreference resolution is achieved by invoking the lingmess-coref [90] model. Algorithm HyperMatch employs a hybrid implementation. Specifically, we utilize PyTorch to invoke the NLI model [55] and the embedding model [118, 126], and we implement the fixed-point computation in Rust. For downstream RAG tasks, we utilize Contriever [64] to generate vector embeddings and retrieve the top-20 documents from Wikipedia [71] to serve as con-

Table 2: Datasets for Evaluation.

Dataset	N	Avg. $ V_Q $	Avg. $ E_Q $	Avg. $ V $	Avg. $ E $	Type
HotpotQA	3000	8.72	4.18	61.54	182.26	Wikipedia
ARC	1170	18.18	9.97	45.87	113.27	Science
MuSiQue	2417	8.02	4.39	65.52	328.1	Multi-hop
MultihopQA	2500	19.33	11.78	14.23	15.87	Wikipedia/News
Legal-QA	975	31.19	24.16	40.61	30.27	Legal
MNLI	2000	5.12	2.83	7.14	5.23	Multi-Genre
SNLI	2000	5.75	2.41	15.42	7.25	Image Captions
ContractNLI	611	6.56	3.91	373.32	264.62	NDAs
ECON	1500	5.30	2.62	15.88	8.41	Narratives
ConTRoL	805	6.56	3.44	178.50	108.95	News/Wikipedia
Legal-NLI	116	31.62	17.50	148.82	94.38	Legal

Table 3: Answer F1-score comparison on RAG task.

Answer F1 (%)	Vanilla	SparseCL	SENTLI	ContraDoc	BSIM	ParaSIM	CDIT	HyDE	HyperMatch (Ours)
ARC	93.24	88.90	88.82	93.76	89.33	91.87	93.84	94.87	94.18 (↓ 0.69)
LegalQA	96.09	76.05	76.03	96.69	76.75	73.48	96.39	92.94	96.87 (↑ 0.18)
HotpotQA	70.53	66.39	66.86	68.94	69.61	32.00	66.36	69.70	79.46 (↑ 8.93)
MultihopQA	43.56	37.88	36.18	43.89	40.42	39.93	43.10	83.68	84.17 (↑ 0.49)
MuSiQue	38.68	18.07	17.98	37.44	11.83	10.41	35.20	48.18	58.96 (↑ 10.78)

text. During the generation phase, we employ the Qwen3.5-9B [96] model. All experiments are conducted on a machine equipped with a modified NVIDIA RTX 4090 GPU with 48GB VRAM, an Intel Core 13900 CPU with 32 cores, and 80GB RAM.

Exp-1: Accuracy. We evaluate the precision of the proposed approach against the baselines in both RAG and Conflict Detection.

Retrieval-Augmented Generation (RAG). As reported in Table 3, HyperMatch consistently achieves the highest Answer F1 on 4 of the 5 RAG datasets. On datasets with relatively simple single-hop queries (ARC and LegalQA), HyperMatch reaches near-ceiling performance (94.18% and 96.87%), respectively trailing and exceeding their strongest baselines by 0.69% and 0.18%. More notably, on multi-hop datasets where queries require reasoning across multiple entities (HotpotQA, MultihopQA, and MuSiQue), HyperMatch outperforms the strongest baselines by 8.93%, 0.49%, and 10.78%, respectively. This gain stems from recursive matching: HyperMatch requires each vertex match to be consistent with all neighboring hyperedges, thereby identifying documents with correct entities but incorrect relations—a key contradiction in multi-hop queries.

Impact of Embedding Quality. We compare four embedding models on HotpotQA and MuSiQue under top- k retrieval ($k \in \{10, 15, 20\}$), where each embedding produces a different retrieved set. Table 4 reports $GIR@k$, $IDR@k$, and Answer F1, measuring retrieved inconsistency, $GIR@k$ detection recall by HyperMatch, and downstream answer quality, respectively. The results yield two main findings. First, changing the embedding model or k affects retrieval quality but does not eliminate inconsistency. Across all settings, $GIR@k$ remains between 0.826 and 0.998. On MuSiQue, reducing k simultaneously increases $GIR@k$ and decreases Answer F1, indicating that the smaller retrieved sets exclude consistent evidence required for answering the queries. Second, HyperMatch consistently detects ground-truth inconsistencies as the retrieval embedding model and k change. Across all embeddings and k values, $IDR@k$ remains between 94.9% and 99.4%. Thus, changes in the retrieval setting do not prevent HyperMatch from identifying inconsistencies in the retrieved documents.

Conflict Detection (CD). Table 5 reports the accuracy results for the Conflict Detection (CD) task. Across all six datasets, HyperMatch achieves the highest or second-highest F1 in most cases, with

Table 4: Impact of embedding quality.

Embeddings	k	HotpotQA			MuSiQue		
		$GIR@k$	$IDR@k$	Ans. F_1	$GIR@k$	$IDR@k$	Ans. F_1
Contriever	10	0.910	0.971	0.738	0.998	0.984	0.098
	15	0.935	0.976	0.761	0.940	0.976	0.311
	20	0.951	0.979	0.795	0.838	0.994	0.590
MPNet	10	0.908	0.980	0.671	0.982	0.965	0.143
	15	0.931	0.979	0.751	0.936	0.967	0.325
	20	0.936	0.975	0.820	0.864	0.980	0.622
E5	10	0.826	0.949	0.898	0.968	0.957	0.217
	15	0.884	0.965	0.914	0.924	0.970	0.371
	20	0.909	0.974	0.907	0.867	0.988	0.564
GTR-T5	10	0.902	0.971	0.718	0.987	0.974	0.101
	15	0.924	0.974	0.776	0.953	0.972	0.243
	20	0.936	0.975	0.832	0.909	0.977	0.513

Table 5: Performance evaluation of Conflict Detection task.

Dataset	Metric (%)	DocInfer	DeBERTa	RoBERTa	ContraDoc	HyperMatch (Ours)
MNLI	Prec	87.34	89.85	87.08	92.57	93.85
	Rec	87.31	89.84	87.11	88.74	89.90
	F_1	87.32	89.84	87.09	90.61	91.83
SNLI	Prec	91.03	91.86	91.61	96.21	96.87
	Rec	91.04	91.86	91.65	93.12	92.14
	F_1	91.03	91.86	91.63	94.64	94.45
ContractNLI	Prec	43.22	55.36	43.17	87.50	90.21
	Rec	42.87	18.39	20.69	98.24	92.60
	F_1	43.04	27.61	27.97	92.56	92.60
ECON	Prec	33.33	33.33	34.52	69.39	82.51
	Rec	5.98	8.76	18.22	42.86	42.14
	F_1	10.14	13.87	23.85	52.99	55.79
ConTRoL	Prec	31.72	31.16	33.73	61.76	68.51
	Rec	27.14	26.63	30.58	77.83	72.54
	F_1	29.25	28.72	32.08	68.87	70.46
LegalNLI	Prec	47.64	55.36	43.17	85.75	84.62
	Rec	28.16	18.39	20.69	78.94	80.23
	F_1	35.40	27.61	27.97	82.20	82.37

particularly large margins on ContractNLI ($F_1 = 92.60\%$), ECON ($F_1 = 55.79\%$), and ConTRoL ($F_1 = 70.46\%$), outperforming the second-best baseline ContraDoc by 0.04%, 2.80%, and 1.59%, respectively. On MNLI and SNLI, where general-domain NLI models already perform well, HyperMatch still attains the best precision (93.85% and 96.87%) and competitive overall F_1 , demonstrating its ability to surface contradictory document pairs without excessive false alarms. The gains on legal and narrative datasets (ContractNLI, ECON, LegalNLI) underscore the structural advantage of hyper simulation: these domains involve rich multi-way entity relationships that can be captured by hyperedges, making the hypergraph representation more effective than flat sequence-based models for detecting logical inconsistencies in long, structured text.

Exp-2: Sensitivity. We evaluate the impact of parameters σ , b , and δ on the accuracy of the HyperMatch algorithm.

Varying σ and δ . Figures (5a)-(5d) report the task performance (Answer F1 for RAG; F_1 for CD) when varying σ (with $\delta = 0.75$) and δ (with $\sigma = 0.70$) across the range $[0.4, 1]$, while fixing $b = 8$. Both parameters exhibit an inverted-U trend, peaking at moderate values. For σ , a low threshold introduces noise by falsely detecting inconsistencies among irrelevant hyperedges. Conversely, an excessively high σ blocks structural inference, degenerating HyperMatch into vertex-level conflict detection and thus degrading overall accuracy. Similarly, δ controls whether a vertex pair is included in the $\mathcal{D}$ -match based on its similarity. An overly low δ incorrectly aligns irrelevant vertices sharing the same entity type; any local conflict between them generates false positives that cascade through the

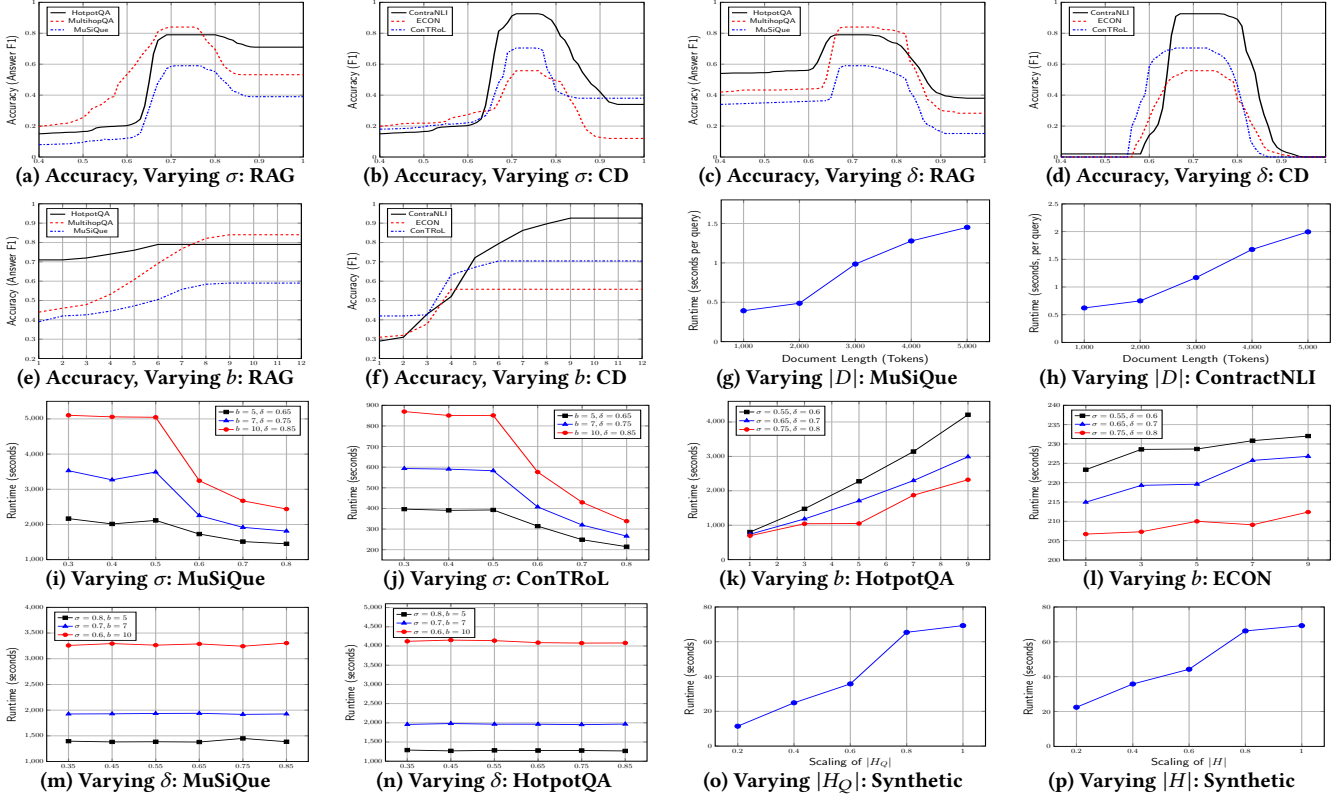


Figure 5: HyperMatch accuracy and efficiency with varying parameters.

Table 6: Efficiency Comparisons.

Method	MuSiQue		HotpotQA	
	Exec. (Pre.) (s)	Speedup	Exec. (Pre.) (s)	Speedup
HyperMatch (Ours)	0.797 (0.450)	-	0.655 (0.270)	-
ContraDoc (LLM)	6.420	5.15×	3.863	4.18×
CDIT (LLM)	7.142	5.73×	4.257	4.60×
HyDE (LLM)	42.866	34.38×	16.582	17.93×
BSIM	1.379 (0.463)	1.48×	1.365 (0.459)	1.97×
ParaSIM	14.839 (0.262)	12.11×	28.078 (0.274)	30.65×

recursive hyper simulation. In contrast, an overly strict δ yields empty $\mathcal{D}$ -matches, violating hyper simulation Condition (2)(a) and misclassifying consistent vertices as inconsistent.

Varying b . Fixing $\sigma = 0.70$ and $\delta = 0.75$, we vary b from 1 to 9 to evaluate its impact on the Answer F1 for the RAG task and the F1 for the Conflict Detection task, as illustrated in Figure (5e) and Figure (5f). These metrics initially increase and gradually stabilize after reaching a peak near $b = 9$. As b increases, HyperMatch searches for additional hyperedge clusters to match with the current hyperedge cluster. Expanding this search space increases the probability of detecting inconsistencies among hyperedge clusters. However, inter-document conflicts are finite, and the final F1 score ceases to improve once the search scope reaches a specific threshold.

Exp-3: Efficiency. We first evaluate the preprocessing cost of hypergraph construction. On MuSiQue and HotpotQA, HypergraphBuild converts the retrieved documents for each query into hypergraphs in 0.450 s and 0.270 s on average, respectively.

With $\sigma = 0.7$, $b = 8$, and $\delta = 0.75$, Table 6 reports the aver-

age per-query runtime. Parenthesized values denote preprocessing overhead, which is included in every speedup calculation. Under this accounting, HyperMatch takes 1.247 s on MuSiQue and 0.925 s on HotpotQA and is the fastest method on both datasets. It is $4.18\times$ – $34.38\times$ faster than the LLM-based methods, e.g., $17.93\times$ – $34.38\times$ over HyDE. Compared with the graph simulation baselines, it is $1.48\times$ – $1.97\times$ faster than BSIM and $12.11\times$ – $30.65\times$ faster than ParaSIM.

Exp-4: Scalability. We evaluate the scalability and computational overhead of the proposed HyperMatch algorithm with respect to varying document sizes and parameters.

Varying document length $|D|$. We first analyze the impact of the retrieved context size by varying the document length from 1,000 to 5,000 tokens. As illustrated in Figures (5g) and (5h), the execution time of HyperMatch increases as the document length grows. This behavior is expected, as longer documents introduce more entities and relationships, thereby expanding the underlying hypergraph structure. However, the runtime exhibits a controlled, near-linear growth, indicating that the algorithm efficiently handles realistic context windows without prohibitive overhead.

Varying σ . We further examine the influence of the similarity threshold σ on computational cost. Figures (5i) and (5j) show a clear downward trend in execution time as σ increases. This aligns with the algorithmic design: a higher σ imposes a stricter condition for forming valid hyperedge clusters ($HC^{\sigma,b}$). Consequently, fewer hyperedge clusters are retained during the initialization phase, which directly reduces the search space and accelerates the subsequent

Table 7: Ablation study of HyperMatch.

Model Configuration	Substitutes	Answer F1 (%)	Runtime (s)
HyperMatch (Full Model)	–	69.21	1.086
Ablations			
Hypergraph	Graph	41.96	1.091
w/o Coreference Resolution	–	54.32	1.031
w/o Hyperedge Cluster	–	55.04	0.825
Replace for HCCalc $\mathcal{M}_1$	Qwen3-EMB-0.6B	58.25	1.010
	MiniLM-RR	65.48	0.719
	Qwen3.5-9B (LLM)	71.29	105.20
w/o $\mathcal{D}$ -match	–	33.67	0.799
Replace for $\mathcal{D}$ -match $\mathcal{M}_2$	DeBERTa-NLI	69.07	1.176
	MiniLM-RR	68.84	0.974
	Qwen3.5-9B (LLM)	64.94	101.02

$\mathcal{D}$ -match enumeration and the main hyper simulation process.

Varying b . Conversely, Figures (5k) and (5l) report the execution time when scaling the branch bound b . The computational overhead increases as b grows. A larger b forces the algorithm to explore a broader structural neighborhood to compute $\text{HC}^{\sigma,b}$. This expanded search space increases the probability of discovering complex mappings, but inevitably incurs additional matching cost.

Varying δ . Figures (5m) and (5n) depict the runtime variations with respect to the semantic alignment threshold δ . Interestingly, the execution time remains almost constant regardless of the δ value. This stability arises since δ functions strictly as a filtering condition for evaluating the semantic similarity between individual vertices. It does not alter the structural topology or the combinatorial number of $\mathcal{D}$ -matches that the algorithm must enumerate during execution, thus having a negligible impact on the overall runtime complexity.

Synthetic data. Finally, to rigorously verify the scalability of HyperMatch on massive, complex structures far exceeding real-life document lengths, we conduct stress tests using synthetic hypergraphs. We construct a synthetic query hypergraph with $|V_Q| = 300K$ and $|E_Q| = 100K$, and a data hypergraph with $|V| = 1,000K$ and $|E| = 400K$. We vary the query and data hypergraph sizes separately from 20% to 100%, while holding the other fixed. As demonstrated in Figures (5o) and (5p), the execution time increases predictably as the scale of the hypergraphs expands. The results show predictable runtime growth over the tested range, demonstrating that HyperMatch can process the evaluated large hypergraphs.

Exp-5: Ablation Study. Table 7 reports the impact of key designs in HyperMatch on effectiveness (Answer F1) and efficiency (Runtime). Both measures are averaged over HotpotQA and MuSiQue. (a) *Hypergraph*: We compare HyperMatch with two variants where one replaces hypergraphs with standard graphs and the other omits coreference resolution; (b) *Hyperedge Cluster*: Full HyperMatch computes hyperedge clusters, whereas the w/o HC variant directly compares individual hyperedges; and (c) *$\mathcal{D}$ -match*: Full HyperMatch vs. a w/o $\mathcal{D}$ -match variant that assumes all $\mathcal{D}$ -match checks are satisfied by default. For both HC and $\mathcal{D}$ -match, we further construct variants that replace $\mathcal{M}_1$ and $\mathcal{M}_2$ with models listed in Table 7.

Hypergraph. Replacing the hypergraph with a graph reduces accuracy from 69.21% to 41.96%, which shows that decomposing n -ary relations into edges loses the information required for joint consistency checking. Although removing coreference resolution reduces

runtime by only 0.055 s, from 1.086 s to 1.031 s, it lowers accuracy by 14.89%, from 69.21% to 54.32%.

Hyperedge Cluster. Without HC, directly comparing individual hyperedges reduces accuracy by 14.17%, from 69.21% to 55.04% in 0.825 s. Among the $\mathcal{M}_1$ substitutes, Qwen3.5-9B achieves the highest accuracy (71.29%), while MiniLM-RR is the fastest (0.719 s). Although the LLM improves upon Full HyperMatch by 2.08%, it increases runtime from 1.086 s to 105.20 s (over 96 $\times$). Thus, the default method provides a balance between cluster quality and efficiency.

$\mathcal{D}$ -match. Removing $\mathcal{D}$ -match reduces accuracy by 35.54% to 33.67% in 0.799 s, confirming the importance of $\mathcal{D}$ -match. Among the $\mathcal{M}_2$ substitutes, DeBERTa-NLI achieves the highest accuracy (69.07%), while MiniLM-RR is the fastest (0.974 s). Their accuracy differs from Full HyperMatch by only 0.14%–4.27%, substantially less than removing $\mathcal{D}$ -match. The default method achieves the highest accuracy (69.21%) in 1.086 s, demonstrating that it balances effectiveness and efficiency without costly LLM inference.

Exp-6: Case Study. We illustrate how HyperMatch marks retrieved documents via the following query instance from HotpotQA.

Q: Which American football running back for the New Orleans Saints of the NFL did Samuel C. Morrison interview for *The Source*?

*D*₁: Morrison ... began as a contributing writer for The Source Magazine, interviewing ... Adrian Peterson ...

*D*₂: Adrian Lewis Peterson ... is an American football running back for the New Orleans Saints of the NFL ...

*D*₃: Ricky Williams ... running back who played ... in the NFL ... drafted by the New Orleans Saints ... traded to the Miami Dolphins.

Let H_Q and H_i be the hypergraphs of Q and document D_i , respectively, and Π denotes the hyper simulation result. In H_Q , $\llbracket C_1 \rrbracket = \text{interview}(\text{PERSON\#1}, \text{PERSON\#2}, \text{ORG})$, $\llbracket C_2 \rrbracket = \text{belongsTo}(\text{PERSON}, \text{ORG\#1}, \text{ORG\#2})$. The values of the vertices are $\llbracket u_0 \rrbracket = \text{"running back"}$, $\llbracket u_1 \rrbracket = \text{"Samuel C. Morrison"}$, $\llbracket u_2 \rrbracket = \text{"Source Magazine"}$, $\llbracket u_3 \rrbracket = \text{"New Orleans Saints"}$, and $\llbracket u_4 \rrbracket = \text{"NFL"}$.

(1) *D*₁: *Consistent*. Let $\llbracket v_0 \rrbracket = \text{"Samuel C. Morrison"}$, $\llbracket v_1 \rrbracket = \text{"Adrian Peterson"}$, and $\llbracket v_2 \rrbracket = \text{"The Source Magazine"}$. The function HC returns $(C_1, C'_1) \in \text{HC}^{\sigma,b}$, where $\llbracket C'_1 \rrbracket = \text{interview}(\text{PERSON\#1}, \text{PERSON\#2}, \text{ORG})$ is constructed from H_1 , and $\{(u_0, v_1), (u_1, v_0), (u_2, v_2)\} \subseteq \mathcal{D}^\delta(C_1, C'_1) \subseteq \Pi$. Thus, the corresponding vertices and the "interview" relation remain matched by Π , and D_1 is marked as *consistent*.

(2) *D*₂: *Consistent*. Let $\llbracket v_3 \rrbracket = \text{"Adrian Lewis Peterson"}$, $\llbracket v_4 \rrbracket = \text{"New Orleans Saints"}$, and $\llbracket v_5 \rrbracket = \text{"NFL"}$. Similarly, HC returns $(C_2, C'_2) \in \text{HC}^{\sigma,b}$ for $\llbracket C'_2 \rrbracket = \text{belongsTo}(\text{PERSON}, \text{ORG\#1}, \text{ORG\#2})$ in H_2 , and $\{(u_0, v_3), (u_3, v_4), (u_4, v_5)\} \subseteq \mathcal{D}^\delta(C_2, C'_2) \subseteq \Pi$. Hence, D_2 matches the entities and relation concerning the player, team, and league, and is also marked as *consistent*.

(3) *D*₃: *Conflict*. Let $\llbracket v_6 \rrbracket = \text{"Ricky Williams"}$, $\llbracket v_7 \rrbracket = \text{"New Orleans Saints"}$, $\llbracket v_8 \rrbracket = \text{"NFL"}$, and $\llbracket v_9 \rrbracket = \text{"Miami Dolphins"}$. The function HC returns two cluster pairs (C_2, C'_2) and (C_2, C''_2) , whose $\mathcal{D}$ -matches contain $\{(u_0, v_6), (u_3, v_7), (u_4, v_8)\} \subseteq \mathcal{D}^\delta(C_2, C'_2)$ and $\{(u_0, v_6), (u_3, v_9), (u_4, v_8)\} \subseteq \mathcal{D}^\delta(C_2, C''_2)$. However, $\llbracket u_3 \rrbracket = \text{"New Orleans Saints"}$ and $\llbracket v_9 \rrbracket = \text{"Miami Dolphins"}$ make inconsistent claims about the team. Therefore, $h_v(u_3, v_9) = \text{false}$

and $(u_3, v_9) \notin \Pi$. The $\mathcal{D}^\delta(C_2, C_2'')$ consequently violates Condition (2)(b), causing (u_0, v_6) and (u_4, v_8) to be removed from Π in the first iteration. Their removal further invalidates $\mathcal{D}^\delta(C_2, C_2')$, and (u_3, v_7) is removed in the second iteration. Hence, both pairs fail Condition (2), and D_3 is marked as *conflict*, with the Saints–Dolphins vertex pair reported as the reason.

Vanilla RAG uses the original top- k retrieved documents, including D_3 , without this marking. In contrast, HyperMatch marks D_3 as conflict before the retrieved documents are passed to the LLM, while D_1 and D_2 contain the information yielding the answer *Adrian Lewis Peterson* (see Appendix G in [1] for additional cases).

Summary. The experiments yield four conclusions: (1) *Effectiveness*: HyperMatch achieves high accuracy on simple RAG queries, ranks first on four of five RAG datasets, and improves the strongest multi-hop baselines by up to 10.78 percentage points; its maximum F1 gain on CD is 2.80 points. (2) *Robustness*: Retrieval inconsistency persists across embeddings, while HyperMatch achieves 94.9%–99.4% detection recall. Performance peaks at $\sigma = 0.7$ and $\delta = 0.75$ and stabilizes near $b = 9$. (3) *Efficiency and Scalability*: Including preprocessing, HyperMatch requires at most 1.247 s per query and is up to 34.38× and 30.65× faster than the LLM-based and graph simulation baselines, respectively. Its runtime grows near-linearly with document length and predictably on massive synthetic hypergraphs. (4) *Design and Explainability*: Ablations validate the importance of the hypergraph representation, coreference resolution, HC, and $\mathcal{D}$ -match, while the default method balances effectiveness and efficiency. The case study further shows that document markings identify the inconsistent entities and relations.

7 Related Works

Data Consistency. Research on data consistency can be categorized into 3 lines. (1) Rule-based methods enforce data integrity through logical formalisms, including consistent query answering [6], integrity constraint enforcement [3, 17], conflict resolution [11, 49], and dependency-based methods over relational [32–34] and graph data [22, 30, 39, 43]. (2) ML-driven methods [56, 82, 108] automate inconsistency detection with reduced manual effort. (3) Unified frameworks combine logic rules with machine learning, covering both relational [9, 41] and graph data [31, 35, 38].

Different from existing data constraint settings where data has been organized as tuples, vertices, and edges, vector retrieval returns unstructured documents with implicit entities and relations, which invalidates previous methods. To address this, we construct a hypergraph from each retrieved document and apply HyperMatch to detect inconsistencies.

Graph Simulation. Graph simulation [57] establishes a binary vertex relation in PTIME. Extensions enhance expressiveness and applicability: bounded [37] and regular [36] simulation extend to path matching, strong simulation [81] enforces bidirectional constraints, incremental algorithms [42] maintain relations under updates, and parametric simulation [40] integrates ML for entity resolution.

Our method extends graph simulation to hypergraphs, which checks the entities involved in multi-way relations while retaining complexity [37, 40, 81]. Unlike graph simulation that matches vertices through edges, hyper simulation jointly considers vertices in-

involved in hyperedges, which incorporates ML models for matching.

Hypergraph Construction. Hypergraphs generalize graphs by allowing hyperedges to connect arbitrary numbers of vertices, which enables multi-way relation modeling [78, 91]. Research on hypergraph construction falls into two lines: implicit methods that infer multi-way relations from vertex features [45, 62], and explicit methods that construct hyperedges from attributes or structures [25].

Different from previous methods that rely on predefined features or structures, we construct hypergraphs from documents via NLP techniques, where hyperedges capture relations among text spans without requiring a predefined relation schema.

Contradiction Detection. Semantic contradiction detection in NLP has evolved from symbolic logic to neural representations to neuro-symbolic verification. The Recognizing Textual Entailment (RTE) challenges [24] established the foundation, grounding early work in logic-based methods [21, 27, 53]. Large-scale corpora [13] enabled neural approaches, with recent models addressing self-contradictions in LLMs [74, 86] and persona coherence in dialogue [67, 88, 128]. Neuro-symbolic frameworks combine neural models with symbolic reasoning for logical consistency [44, 125]. For RAG systems, recent work detects context conflicts via document-level inference [48, 85, 100, 124].

Our hypergraph-based simulation approach differs by providing entity-level detection with structural explainability. Unlike document-level methods, we model fine-grained semantic dependencies among entities through hypergraph matching, enabling traceable contradiction localization.

Vector Database & Application. Vector databases power applications from recommendation to RAG and LLM agents. Current systems comprise: (1) native vector databases for storage and retrieval [18, 93, 94, 111, 114]; (2) vector-enabled extensions augmenting relational or NoSQL databases [5, 97]. Research focuses on three areas: (i) indexing efficiency [65, 83]; (ii) system scalability [99, 105]; and (iii) hybrid query expressiveness [29, 52, 106].

While these systems excel at scalable vector search, they lack native quality management, treating vectors as opaque and assuming correctness. To address the gap, this work constructs hypergraphs from the query and retrieved documents, applying HyperMatch for post-retrieval consistency checking.

8 Conclusion

This work presents a framework for detecting inconsistencies in vector similarity retrieval. Based on a hypergraph representation of n -ary relations in queries and documents, our core contributions are hyper simulation and HyperMatch. Hyper simulation recursively checks the consistency of entities and relations in the retrieval, while HyperMatch computes the hyper simulation no harder than graph simulation. Experiments on RAG and conflict detection show that retrieval inconsistency is prevalent across embeddings, and HyperMatch efficiently detects most inconsistencies while pinpointing the conflicting entities and relations.

Future work will develop incremental algorithms to maintain hyper simulation under dynamic updates to vector databases and extend the framework to detect inconsistencies in multimodal retrieval commonly used for agent memory management.

References

- [1] 2026. Code, full version paper including proofs and examples. link.
- [2] News 31366. 2025. From Shiny Object to Sober Reality: The Vector Database Story, Two Years Later. *Medium* (nov 2025). https://medium.com/@news_31366/from-shiny-object-to-sober-reality-the-vector-database-story-two-years-later-970c6421ca1e States: "Billions of dollars flowed into startups like Pinecone, Weaviate, and Chroma... over the past two years".
- [3] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley.
- [4] Aisha Alansari and Hamzah Luqman. 2026. Large language models hallucination: A comprehensive survey. *Computer Science Review* 61 (2026), 100970.
- [5] Jonathan Apel et al. 2023. pgvector: Open-source vector similarity search for Postgres. <https://github.com/pgvector/pgvector> Accessed: 2026-02-05.
- [6] Marcelo Arenas, Leopoldo Bertossi, and Jan Chomicki. 1999. Consistent query answers in inconsistent databases. In *Proceedings of the eighteenth ACM SIGMOD-SIGACT-SIGART symposium on Principles of database systems*. 68–79.
- [7] Akari Asai, Sewon Min, Zexuan Zhong, and Danqi Chen. 2023. Retrieval-based Language Models and Applications. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 6: Tutorial Abstracts)*, Yun-Nung (Vivian) Chen, Margot Margot, and Siva Reddy (Eds.). Association for Computational Linguistics, Toronto, Canada, 41–46. doi:10.18653/v1/2023.acl-tutorials.6
- [8] Fu Bang. 2023. GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS 2023)*, Liling Tan, Dmitrijs Milajevs, Geeticka Chauhan, Jeremy Gwinnup, and Elijah Rippeth (Eds.). Association for Computational Linguistics, Singapore, 212–218. doi:10.18653/v1/2023.nlp-oss.1.24
- [9] Xianchun Bao, Zian Bao, Bie Binbin, QingSong Duan, Wenfei Fan, Hui Lei, Daji Li, Wei Lin, Peng Liu, Zhicong Lv, et al. 2024. Rock: Cleaning Data by Embedding ML in Logic Rules. In *Companion of the 2024 International Conference on Management of Data*. 106–119.
- [10] James Bergstra and Yoshua Bengio. 2012. Random search for hyper-parameter optimization. *Journal of machine learning research* 13, 2 (2012).
- [11] Leopoldo Bertossi. 2011. Database repairing and consistent query answering. *Synthesis Lectures on Data Management* 3, 5 (2011), 1–121.
- [12] Steven Bird, Ewan Klein, and Edward Loper. 2009. *Natural language processing with Python: analyzing text with the natural language toolkit*. O'Reilly Media, Inc.
- [13] Samuel R Bowman, Gabor Angeli, Christopher Potts, and Christopher D Manning. 2015. A large annotated corpus for learning natural language inference. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*. 632–642.
- [14] Alain Bretto. 2013. Hypergraph theory. *An introduction. Mathematical Engineering*. Cham: Springer 1 (2013), 209–216.
- [15] Eric Brill. 1992. A simple rule-based part of speech tagger. In *Proceedings of the Third Conference on Applied Natural Language Processing*. 152–155.
- [16] Arun Chandrasekaran and Radu Miclaus. 2023. *Innovation Insight: Vector Databases*. Innovation Insight 4705699. Gartner. <https://www.gartner.com/en/documents/4705699>
- [17] Jan Chomicki. 2007. Consistent query answers in inconsistent databases. In *Proceedings of the Twenty-Sixth ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*. ACM, 163–171.
- [18] Chroma. 2023. Chroma: The AI-native open-source vector database. <https://github.com/chroma-core/chroma> Accessed: 2026-02-05.
- [19] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have solved question answering? Try ARC, the AI2 reasoning challenge. *arXiv preprint arXiv:1803.05457* (2018).
- [20] Daniel Cohen, Bhaskar Mitra, Oleg Lesota, Navid Rekabsaz, and Carsten Eickhoff. 2021. Not all relevance scores are equal: Efficient uncertainty and calibration modeling for deep retrieval models. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 654–664.
- [21] Cleo Condoravdi, Dick Crouch, Valeria De Paiva, Reinhard Stolle, and Daniel Bobrow. 2003. Entailment, intentionality and text understanding. In *Proceedings of the HLT-NAACL 2003 workshop on Text meaning*. 38–45.
- [22] Erick Cortés-Soto and Marc Thurley. 2021. Graph pattern detection: Hardness for all induced patterns and faster non-induced cycles. *Algorithmica* 83, 6 (2021), 1600–1650.
- [23] Florin Cuconasu, Giovanni Trappolini, Federico Siciliano, Simone Filice, Cesare Campagnano, Yoelle Maarek, Nicola Tonellotto, and Fabrizio Silvestri. 2024. The Power of Noise: Redefining Retrieval for RAG Systems. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval* (Washington DC, USA) (SIGIR '24). Association for Computing Machinery, New York, NY, USA, 719–729. doi:10.1145/3626772.
- [24] Ido Dagan, Oren Glickman, and Bernardo Magnini. 2005. The pascal recognising textual entailment challenge. In *Machine learning challenges workshop*. Springer, 177–190.
- [25] Qionghai Dai and Yue Gao. 2023. Hypergraph Modeling. In *Hypergraph Computation*. Springer Nature Singapore, 49–71. doi:10.1007/978-981-99-0185-2_4
- [26] Nicola De Cao, Wilker Aziz, and Ivan Titov. 2021. Editing Factual Knowledge in Language Models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. 6491–6506.
- [27] Marie-Catherine De Marneffe, Anna N Rafferty, and Christopher D Manning. 2008. Finding contradictions in text. In *Proceedings of acl-08: HLT*. 1039–1047.
- [28] Bhuwan Dhingra, Jeremy R Cole, Julian Martin Eisenschlos, Daniel Gillick, Cora Hayward, and Sharan Narang. 2022. Time-aware language models as temporal knowledge bases. *Transactions of the Association for Computational Linguistics* 10 (2022), 483–499.
- [29] Elastic. 2023. Elasticsearch with kNN Plugin. <https://www.elastic.co/guide/en/elasticsearch/reference/current/knn-search.html> Accessed: 2026-02-05.
- [30] Wenfei Fan. 2019. Dependencies for graphs. In *Proceedings of the 38th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems*. ACM, 403–416.
- [31] W. Fan, Wenzhi Fu, Ruochun Jin, Muiyang Liu, Ping Lu, and Chao Tian. 2023. Making It Tractable to Catch Duplicates and Conflicts in Graphs. *Proceedings of the ACM on Management of Data* 1 (2023), 1 – 28. doi:10.1145/3588940
- [32] Wenfei Fan, Floris GEERTS, and Xibei Jia. 2007. Improving data quality: Consistency and accuracy. ACM.
- [33] Wenfei Fan, Floris Geerts, Xibei Jia, and Anastasios Kementsietsidis. 2008. Conditional functional dependencies for capturing data inconsistencies. *ACM Transactions on Database Systems (TODS)* 33, 2 (2008), 1–48.
- [34] Wenfei Fan, Floris Geerts, Nan Tang, and Wenyuan Yu. 2014. Conflict resolution with data currency and consistency. *Journal of Data and Information Quality (JDIQ)* 5, 1-2 (2014), 1–37.
- [35] Wenfei Fan, Ruochun Jin, Muiyang Liu, Ping Lu, Chao Tian, and Jingren Zhou. 2020. Capturing associations in graphs. *Proceedings of the VLDB Endowment* 13, 12 (2020), 1863–1876.
- [36] Wenfei Fan, Jianzhong Li, Shuai Ma, Nan Tang, and Yinghui Wu. 2011. Adding regular expressions to graph reachability and pattern queries. In *2011 IEEE 27th International Conference on Data Engineering*. IEEE, 39–50.
- [37] Wenfei Fan, Jianzhong Li, Shuai Ma, Nan Tang, Yinghui Wu, and Yunpeng Wu. 2010. Graph pattern matching: From intractable to polynomial time. *Proceedings of the VLDB Endowment (PVLDB)* 3, 1 (2010), 264–275.
- [38] Wenfei Fan, Muiyang Liu, Shuhao Liu, and Chao Tian. 2024. Capturing more associations by referencing external graphs. *Proceedings of the VLDB Endowment* 17, 6 (2024), 1173–1186.
- [39] Wenfei Fan, Xueli Liu, Ping Lu, and Chao Tian. 2018. Catching numeric inconsistencies in graphs. In *Proceedings of the 2018 International Conference on Management of Data*. 381–393.
- [40] Wenfei Fan, Ping Lu, Kehan Pang, Ruochun Jin, and Wenyuan Yu. 2024. Linking entities across relations and graphs. *ACM Transactions on Database Systems* 49, 1 (2024), 1–50.
- [41] Wenfei Fan, Ping Lu, and Chao Tian. 2020. Unifying logic rules and machine learning for entity enhancing. *Science China Information Sciences* 63, 7 (2020), 172001.
- [42] Wenfei Fan, Xin Wang, and Yinghui Wu. 2013. Incremental graph pattern matching. *ACM Transactions on Database Systems (TODS)* 38, 3 (2013), 1–47.
- [43] Wenfei Fan, Yinghui Wu, and Jingbo Xu. 2016. Functional dependencies for graphs. In *Proceedings of the 2016 international conference on management of data*. 1843–1857.
- [44] Yu Feng, Nathaniel Weir, Kaj Bostrom, Sam Bayless, Darion Cassel, Sapana Chaudhary, Benjamin Kiesel-Reiter, and Huzefa Rangwala. 2025. VeriCoT: Neuro-symbolic Chain-of-Thought Validation via Logical Consistency Checks. *arXiv preprint arXiv:2511.04662* (2025).
- [45] Yifan Feng, Zizhao Zhang, Xibin Zhao, Rongrong Ji, and Yue Gao. 2022. Hypergraph Learning: Methods and Practices. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 44, 5 (2022), 2548–2566. doi:10.1109/TPAMI.2020.3035046
- [46] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2023. Precise Zero-Shot Dense Retrieval without Relevance Labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, Toronto, Canada, 1762–1777. doi:10.18653/v1/2023.acl-long.99
- [47] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* 2, 1 (2023).
- [48] Vignesh Gokul, Srikanth Tenneti, and Alwarappan Nakkiran. 2025. Contradiction Detection in RAG Systems: Evaluating LLMs as Context Validators for Improved Information Consistency. *arXiv preprint arXiv:2504.00180* (2025).

- [49] Sergio Greco and Ester Zumpano. 2003. Preferred repairs for inconsistent databases. In *Proceedings of the Seventh International Database Engineering and Applications Symposium*. IEEE, 202–211.
- [50] Neel Guha, Julian Nyarko, Daniel E Ho, Christopher Ré, Adam Chen, Arthur Chilton, et al. 2023. LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models. In *Advances in Neural Information Processing Systems*, Vol. 36. 42767–42813.
- [51] Jiafeng Guo, Yixing Fan, Liang Pang, Liu Yang, Qingyao Ai, Hamed Zamani, Chen Wu, W Bruce Croft, and Xueqi Cheng. 2020. A deep look into neural ranking models for information retrieval. *Information Processing & Management* 57, 6 (2020), 102067.
- [52] Yikun Han, Rui Liu, Lei Ma, Shuo Yu, Zhen Wang, Zhiyuan Ning, Jun Zhang, Pei Xu, Peng Li, Wenxuan Ju, et al. 2023. A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge. *arXiv preprint arXiv:2310.11703* (2023).
- [53] Sanda Harabagiu, Andrew Hickl, and Finley Lacatusu. 2006. Negation, contrast and contradiction in text processing. In *AAAI*, Vol. 6. 755–762.
- [54] Kostas Hatalis, Despina Christou, Joshua Myers, Steven Jones, Keith Lambert, Adam Amos-Binks, Zohreh Dannenhauer, and Dustin Dannenhauer. 2023. Memory matters: The need to improve long-term memory in llm-agents. In *Proceedings of the AAAI Symposium Series*, Vol. 2. 277–280.
- [55] Pengcheng He, Jianfeng Gao, and Weizhu Chen. 2023. DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. In *The Eleventh International Conference on Learning Representations*.
- [56] Alireza Heidari, Joshua McGrath, Ihab F Ilyas, and Theodoros Rekatsinas. 2019. Holodetect: Few-shot learning for error detection. In *Proceedings of the 2019 International Conference on Management of Data*. 829–846.
- [57] Monika Rauch Henzinger, Thomas A Henzinger, and Peter W Kopke. 1995. Computing simulations on finite and infinite graphs. In *Proceedings of IEEE 36th Annual Foundations of Computer Science*. IEEE, 453–462.
- [58] Xanh Ho, Anh-Khoa Du, and Akiko Aizawa. 2020. Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In *Proceedings of the 28th International Conference on Computational Linguistics*. 6609–6625.
- [59] Sirui Hong, Yizhang Lin, Bang Liu, Bangbang Liu, Binhao Wu, Ceyao Zhang, Danyang Li, Jiaqi Chen, Jiayi Zhang, Jinlin Wang, Li Zhang, Lingyao Zhang, Min Yang, Mingchen Zhuge, Taicheng Guo, Tuo Zhou, Wei Tao, Robert Tang, Xiangtao Lu, Xiaowu Zheng, Xinbing Liang, Yaying Fei, Yuheng Cheng, Yongxin Ni, Zhibin Gou, Zongze Xu, Yuyu Luo, and Chenglin Wu. 2025. Data Interpreter: An LLM Agent for Data Science. In *Findings of the Association for Computational Linguistics: ACL 2025*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 19796–19821. doi:10.18653/v1/2025.findings-acl.1016
- [60] Matthew Honnibal, Ines Montani, Sofie Van Landeghem, and Adriane Boyd. 2020. spaCy: Industrial-strength Natural Language Processing in Python. (2020). doi:10.5281/zenodo.1212303
- [61] Yufang Hou, Alessandra Pascale, Javier Carnerero-Cano, Tigran Tchrakian, Radu Marinescu, Elizabeth Daly, Inkit Padhi, and Prasanna Sattigeri. 2024. WikiContradict: A Benchmark for Evaluating LLMs on Real-World Knowledge Conflicts from Wikipedia. In *Advances in Neural Information Processing Systems*.
- [62] Yuchi Huang, Qingshan Liu, and Dimitris Metaxas. 2009. Video Object Segmentation by Hypergraph Cut. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 1738–1745. doi:10.1109/CVPR.2009.5206795
- [63] Yue Huang, Lichao Sun, Haoran Wang, Siyuan Wu, Qihui Zhang, Yuan Li, Chujie Gao, Yixin Huang, Wenhan Lyu, Yixuan Zhang, et al. 2024. Trustllm: Trustworthiness in large language models. *arXiv preprint arXiv:2401.05561* (2024).
- [64] Gautier Izcard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2021. Unsupervised Dense Information Retrieval with Contrastive Learning. doi:10.48550/ARXIV.2112.09118
- [65] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2010. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2010), 117–128. doi:10.1109/TPAMI.2010.57
- [66] Cheng Jiayang, Chunkit Chan, Qianqian Zhuang, Lin Qiu, Tianhang Zhang, Tengxiao Liu, Yangqiu Song, Yue Zhang, Pengfei Liu, and Zheng Zhang. 2024. ECON: On the Detection and Resolution of Evidence Conflicts. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 7816–7844.
- [67] Di Jin, Sijia Liu, Yang Liu, and Dilek Hakkani-Tur. 2022. Improving Bot Response Contradiction Detection via Utterance Rewriting. In *Proceedings of the 23rd Annual Meeting of the Special Interest Group on Discourse and Dialogue*. 605–614.
- [68] Zhi Jing, Yongye Su, and Yikun Han. 2025. When Large Language Models Meet Vector Databases: A Survey. In *2025 Conference on Artificial Intelligence x Multimedia (AIXMM)*. 7–13. doi:10.1109/AIXMM62960.2025.00008
- [69] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. doi:10.1109/TPAMI.2010.57
- [70] Michael Karlinger, Millist Vincent, and Michael Schrefl. 2009. Inclusion Dependencies in XML: Extending Relational Semantics. In *Proceedings of the 20th International Conference on Database and Expert Systems Applications (Linz, Austria) (DEXA '09)*. Springer-Verlag, Berlin, Heidelberg, 23–37. doi:10.1007/978-3-642-03573-9_3
- [71] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*. 6769–6781.
- [72] Yuta Koreeda and Christopher D Manning. 2021. ContractNLI: A dataset for document-level natural language inference for contracts. In *Findings of the Association for Computational Linguistics: EMNLP 2021*. 1907–1919.
- [73] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems (Vancouver, BC, Canada) (NIPS '20)*. Curran Associates Inc., Red Hook, NY, USA, Article 793, 16 pages.
- [74] Jierui Li, Vipul Raheja, and Dhruv Kumar. 2024. ContraDoc: Understanding Self-Contradictions in Documents with Large Language Models. arXiv:2311.09182 [cs.CL] <https://arxiv.org/abs/2311.09182>
- [75] Yinheng Li, Shaofei Wang, Han Ding, and Hang Chen. 2023. Large Language Models in Finance: A Survey. In *Proceedings of the Fourth ACM International Conference on AI in Finance (Brooklyn, NY, USA) (ICAIF '23)*. Association for Computing Machinery, New York, NY, USA, 374–382. doi:10.1145/3604237.3626869
- [76] Hanmeng Liu, Leyang Cui, Jian Liu, and Yue Zhang. 2021. Natural Language Inference in Context - Investigating Contextual Reasoning over Long Texts. *Proceedings of the AAAI Conference on Artificial Intelligence* 35, 15 (May 2021), 13388–13396. doi:10.1609/aaai.v35i15.17580
- [77] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. 2019. RoBERTa: A Robustly Optimized BERT Pretraining Approach. *arXiv preprint arXiv:1907.11692* (2019).
- [78] Xiaohua Lu, Liubov Tupikina, and Mehwish Alam. 2025. Two-dimensional Taxonomy for N-ary Knowledge Representation Learning Methods. arXiv:2506.05626 [cs.LG] <https://arxiv.org/abs/2506.05626>
- [79] Haoran Luo, Guanting Chen, Yandan Zheng, Xiaobao Wu, Yikai Guo, Qika Lin, Yu Feng, Zemin Kuang, Meina Song, Yifan Zhu, et al. 2026. Hypergraphrag: Retrieval-augmented generation via hypergraph-structured knowledge representation. *Advances in Neural Information Processing Systems* 38 (2026), 152206–152234.
- [80] Kexin Ma, Ruochun Jin, Wang Haotian, Wang Xi, Huan Chen, Yuhua Tang, and Qian Wang. 2024. Context-Driven Index Trimming: A Data Quality Perspective to Enhancing Precision of RALMs. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (Eds.). Association for Computational Linguistics, Miami, Florida, USA, 4886–4901. doi:10.18653/v1/2024.findings-emnlp.281
- [81] Shuai Ma, Yang Cao, Wenfei Fan, Jinpeng Huai, and Tianyu Wo. 2014. Strong simulation: Capturing topology in graph pattern matching. *ACM Transactions on Database Systems (TODS)* 39, 1 (2014), 1–46.
- [82] Mohammad Mahdavi, Zia Wasch Abedjan, Raul Castro Fernandez, Samuel Madden, Mourad Ouzzani, Michael Stonebraker, and Nan Tang. 2019. Raha: A configuration-free error detection system. In *Proceedings of the 2019 International Conference on Management of Data*. 865–882.
- [83] Yu. A. Malkov and D. A. Yashunin. 2019. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2019), 824–836. doi:10.1109/TPAMI.2018.2889473
- [84] Yu. A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. doi:10.1109/TPAMI.2018.2889473
- [85] Puneet Mathur, Gautam Kunapuli, Riyaz Bhat, Manish Shrivastava, Dinesh Manocha, and Maneesh Singh. 2022. DocInfer: Document-level Natural Language Inference using Optimal Evidence Selection. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.). Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 809–824. doi:10.18653/v1/2022.emnlp-main.51
- [86] Niels Münder, Jingxuan He, Slobodan Jenko, and Martin Vechev. 2024. Self-contradictory Hallucinations of Large Language Models: Evaluation, Detection and Mitigation. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*. OpenReview.
- [87] Jianmo Ni, Chen Qu, Jing Lu, Zhuyun Dai, Gustavo Hernandez Abrego, Ji Ma, Vincent Zhao, Yi Luan, Keith Hall, Ming-Wei Chang, and Yinfei Yang. 2022. Large Dual Encoders Are Generalizable Retrievers. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. Association

- for Computational Linguistics, Abu Dhabi, United Arab Emirates, 9844–9855. doi:10.18653/v1/2022.emnlp-main.669
- [88] Yixin Nie, Mary Williamson, Mohit Bansal, Douwe Kiela, and Jason Weston. 2021. I like fish, especially dolphins: Addressing contradictions in dialogue modeling. In *Proceedings of the 59th annual meeting of the association for computational linguistics and the 11th international joint conference on natural language processing (volume 1: long papers)*. 1699–1713.
- [89] Joakim Nivre, Marie-Catherine de Marneffe, Filip Ginter, Jan Hajič, Christopher D Manning, Sampo Pyysalo, Sebastian Schuster, Francis Tyers, and Daniel Zeman. 2020. Universal Dependencies v2: An Evergrowing Multilingual Treebank Collection. In *Proceedings of the 12th Language Resources and Evaluation Conference*. European Language Resources Association, 4034–4043.
- [90] Shon Otmazgin, Arie Cattan, and Yoav Goldberg. 2022. LingMess: Linguistically Informed Multi Expert Scorers for Coreference Resolution. arXiv. doi:10.48550/ARXIV.2205.12644
- [91] Xavier Ouyard. 2020. Hypergraphs: an introduction and review. arXiv:2002.05014 [cs.DM]. <https://arxiv.org/abs/2002.05014>
- [92] Gustavo Penha and Arie Hauff. 2021. On the calibration and uncertainty of neural learning to rank models. *arXiv preprint arXiv:2101.04356* (2021).
- [93] Pinecone Systems, Inc. 2025. Pinecone: The Vector Database for AI Applications. <https://www.pinecone.io/> Accessed: 2026-02-05.
- [94] Qdrant. 2023. Qdrant: An Open-Source Vector Database and Search Engine. <https://github.com/qdrant/qdrant> Accessed: 2026-02-05.
- [95] Peng Qi, Yuhao Zhang, Yuhui Zhang, Jason Bolton, and Christopher D. Manning. 2020. Stanza: A Python Natural Language Processing Toolkit for Many Human Languages. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*. <https://nlp.stanford.edu/pubs/qi2020stanza.pdf>
- [96] Qwen Team. 2026. Qwen3.5: Towards Native Multimodal Agents. <https://qwen.ai/blog?id=qwen3.5>
- [97] Redis Ltd. 2024. RedisVL: The Redis Vector Library. <https://github.com/redis/redis-vl-python> Accessed: 2026-02-05.
- [98] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, Hong Kong, China, 3982–3992. doi:10.18653/v1/D19-1410
- [99] Matthew Richardson et al. 2023. Vald: A Highly Scalable Distributed Fast Approximate Nearest Neighbor Dense Vector Search Engine. In *Proceedings of the 2023 ACM SIGMOD International Conference on Management of Data*. Association for Computing Machinery, 4177–4180. doi:10.1145/3584372.3593530
- [100] Tal Schuster, Sihao Chen, Senaka Buttipitiya, Alex Fabrikant, and Donald Metzler. 2022. Stretching Sentence-pair NLI Models to Reason over Long Documents and Clusters. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang (Eds.). Association for Computational Linguistics, Abu Dhabi, United Arab Emirates, 394–412. doi:10.18653/v1/2022.findings-emnlp.28
- [101] Sina Semnani, Jirayu Burapachee, Arpandeep Khatua, Thanawan Atchariyachanvanit, Zheng Wang, and Monica Lam. 2025. Detecting corpus-level knowledge inconsistencies in Wikipedia with large language models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 34827–34854.
- [102] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. 2020. MPNet: Masked and Permuted Pre-training for Language Understanding. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., 16857–16867. <https://proceedings.neurips.cc/paper/2020/hash/c3a690be93aa602ee2dc0ccab5b7b67e-Abstract.html>
- [103] Katia P Sycara. 1993. Machine learning for intelligent support of conflict resolution. *Decision Support Systems* 10, 2 (1993), 121–136.
- [104] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics* 10 (2022), 539–554.
- [105] Vearch. 2023. Vearch: A Cloud-Native Distributed Vector Database. <https://github.com/vearch/vearch> Accessed: 2026-02-05.
- [106] Vespa.ai. 2023. Vespa: The AI Search Platform. <https://github.com/vespa-engine/vespa> Accessed: 2026-02-05.
- [107] Millist W Vincent, Jixue Liu, and Chengfei Liu. 2004. Strong functional dependencies and their application to normal forms in XML. *ACM Transactions on Database Systems (TODS)* 29, 3 (2004), 445–462.
- [108] Larysa Visengeriyeva and Ziawasch Abedjan. 2018. Metadata-driven error detection. In *Proceedings of the 30th International Conference on Scientific and Statistical Database Management*. 1–12.
- [109] Loan T. H. Vo, Jinli Cao, and Wenny Rahayu. 2011. Discovering conditional functional dependencies in XML data. In *Proceedings of the Twenty-Second Australasian Database Conference - Volume 115 (Perth, Australia) (ADC '11)*. Australian Computer Society, Inc., AUS, 143–152.
- [110] Fei Wang, Xingchen Wan, Ruoxi Sun, Jiefeng Chen, and Sercan O Arik. 2025. Astute rag: Overcoming imperfect retrieval augmentation and knowledge conflicts for large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*. 30553–30571.
- [111] Jianguo Wang, Xiaofan Yi, Ruifeng Guo, Han Jin, Peng Xu, Shuang Li, Xiang Wang, Xiangyu Guo, Changsheng Li, Xinyu Xu, and Kun Yu. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 2614–2627. doi:10.1145/3448016.3457550
- [112] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text Embeddings by Weakly-Supervised Contrastive Pre-training. *arXiv preprint arXiv:2212.03533* (2022). doi:10.48550/arXiv.2212.03533
- [113] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. 2020. MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., 5776–5788. <https://proceedings.neurips.cc/paper/2020/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>
- [114] Weaviate. 2025. Weaviate: An Open-Source Vector Database. <https://github.com/weaviate/weaviate> Accessed: 2026-02-05.
- [115] Orion Weller, Michael Boratko, Iftekhar Naim, and Jinhyuk Lee. 2025. On the Theoretical Limitations of Embedding-Based Retrieval. arXiv:2508.21038 [cs.IR] <https://arxiv.org/abs/2508.21038>
- [116] Jianfeng Wen, Jianxin Li, Yongyi Mao, Shini Chen, and Richong Zhang. 2016. On the representation and embedding of knowledge bases beyond binary relations. *arXiv preprint arXiv:1604.08642* (2016).
- [117] Adina Williams, Nikita Nangia, and Samuel Bowman. 2018. A broad-coverage challenge corpus for sentence understanding through inference. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)*. 1112–1122.
- [118] Haikuo Xu, Zongyu Lin, Kai-Wei Chang, Yizhou Sun, and Piotr Indyk. 2025. Contradiction Retrieval via Contrastive Learning with Sparsity. In *Forty-second International Conference on Machine Learning*.
- [119] Hao Yang, Zhiyu Yang, Xupeng Zhang, Wei Wei, Yunjie Zhang, and Lin Yang. 2026. CompactRAG: Reducing LLM Calls and Token Overhead in Multi-Hop Question Answering. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*. ACM.
- [120] Tiannuo Yang, Wen Hu, Wangqi Peng, Yusen Li, Jianguo Li, Gang Wang, and Xiaoguang Liu. 2024. VDTuner: Automated Performance Tuning for Vector Data Management Systems. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*.
- [121] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. 2369–2380.
- [122] Ziqi Yang, Xuhai Xu, Bingsheng Yao, Ethan Rogers, Shao Zhang, Stephen Intille, Nawar Shara, Guodong Gordon Gao, and Dakuo Wang. 2024. Talk2Care: An LLM-based Voice Assistant for Communication between Healthcare Providers and Older Adults. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.* 8, 2, Article 73 (May 2024), 35 pages. doi:10.1145/3659625
- [123] Limei Yao, Kexin Ma, Ruochun Jin, Haoqi Zheng, and Dong Wang. 2025. Enhancing Vector Data Quality through Negative Learning for Retrieval-augmented Large Models. In *2025 International Joint Conference on Neural Networks (IJCNN)*. IEEE, 1–8.
- [124] Wenpeng Yin, Dragomir Radev, and Caiming Xiong. 2021. DOCNLI: A Large-scale Dataset for Document-level Natural Language Inference. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*. Association for Computational Linguistics (ACL), 4913–4922.
- [125] Moy Yuan and Andreas Vlachos. 2024. Zero-shot fact-checking with semantic triples and knowledge graphs. In *Proceedings of the 1st Workshop on Knowledge Graphs and Large Language Models (KaLLM 2024)*. 105–115.
- [126] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *arXiv preprint arXiv:2506.05176* (2025).
- [127] Zheyuan Zhang, Daniel Zhang-Li, Jifan Yu, Linlu Gong, Jinchang Zhou, Zhanxin Hao, Jianxiao Jiang, Jie Cao, Huiqin Liu, Zhiyuan Liu, Lei Hou, and Juanzi Li. 2025. Simulating Classroom Education with LLM-Empowered Agents. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, Luis Chiruzzo, Alan Ritter, and Lu Wang (Eds.). Association for Computational Linguistics, Albuquerque, New Mexico, 10364–10379. doi:10.18653/v1/2025.naacl-long.520
- [128] Chuji Zheng, Jinfeng Zhou, Yinhe Zheng, Libiao Peng, Zhen Guo, Wenquan Wu, Zheng-Yu Niu, Hua Wu, and Minlie Huang. 2022. CDConv: A benchmark for contradiction detection in Chinese conversations. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*. 18–29.
- [129] Ziyuan Zhuang, Zhiyang Zhang, Sitao Cheng, Fangkai Yang, Jia Liu, Shujian

Huang, Qingwei Lin, Saravan Rajmohan, Dongmei Zhang, and Qi Zhang. 2024. EfficientRAG: Efficient Retriever for Multi-Hop Question Answering. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 3392–3411.

Appendix

A Detailed Specifications of Linguistic Annotations

In this appendix, we detail the specific tag sets and labels utilized for the Universal Dependency (UD) parsing and Named Entity Recognition (NER) models discussed in the main text. These sets define the formal spaces for the mappings Pos, dep, and Ent utilized in our hypergraph construction algorithm.

A.1 Universal Dependency and Part-of-Speech Specifications

As defined in Section 2, Universal Dependency (UD) serves as our framework for grammatical annotation. The UD structure of a document is represented as the triple $(T, E_{\text{dep}}, \text{Pos})$. Here, T denotes the ordered set of tokens that act as the vertex set within our model.

The function $\text{Pos} : T \rightarrow \text{POS}$ maps each token to a coarse-grained part-of-speech tag. The complete finite set of POS tags utilized is detailed in Table 8.

Furthermore, the set of dependency arcs is defined as $E_{\text{dep}} \subseteq T \times \text{DEP} \times T$, where each element (t, dep, t') constitutes a directed n -ary relation from a head token t to a dependent token t' . Table 9 enumerates the dependency relations (DEP) parsed by spaCy [60], categorized by their syntactic functions.

A.2 Named Entity Recognition Specifications

Named Entity Recognition (NER) is a natural language processing task designed to identify and classify specific named entities into predefined categories. Over a token sequence T^* , we formalize this as a mapping function $\text{Ent} : T^* \rightarrow \text{ENT} \cup \{\perp\}$. Here, ENT represents the finite set of valid entity types, and $\perp$ denotes tokens that do not belong to any named entity. The complete set of ENT labels utilized for semantic tracking in our approach is detailed in Table 10.

B Missing Rules Definition

B.1 Deduction Rules for Vertex Extraction

This section details the deduction rules $\mathcal{R}_1$ utilized in the VerticesDeduction function introduced in Section 3.2. The core objective of these rules is to group tokens into vertices by leveraging Part-of-Speech (POS) tags, Universal Dependency (DEP) relations, and Named Entity Recognition (ENT).

The deduction system operates on a state denoted as $C = (I, R)$, where I represents the set of currently extracted token spans, and $R \subseteq T$ is the pool of unresolved tokens. The deduction initiates from an empty state $C_0 = (\emptyset, T)$ and iteratively applies the rules until a fixed point is reached (*i.e.*, no more rules can be applied).

We utilize the deduction notation:

$$\frac{\Gamma}{(I, R) \Rightarrow (I', R')}$$

to denote a state transition. This indicates that if the premises Γ (a set of conditions) hold in the current state (I, R) , the system deduces a new state (I', R') .

The complete set of deduction rules $\mathcal{R}_1$ is formally defined as follows:

$$\frac{\text{ENTITY} \quad \text{Ent}(t) = \perp \quad v = \{t' \mid \text{anc}(t, t') \wedge t' \in R \wedge \text{Ent}(t') \neq \perp\} \neq \emptyset}{(I, R) \Rightarrow (I \cup \{v\}, R \setminus v)}$$

$$\frac{\text{BASE-ITEM} \quad \text{Ent}(t) = \perp \quad t \in R \quad \text{Pos}(t) \in \{\text{NOUN}, \text{VERB}, \text{ADJ}, \text{PROPN}, \text{AUX}, \text{NUM}, \text{PRON}\}}{(I, R) \Rightarrow (I \cup \{t\}, R \setminus \{t\})}$$

$$\frac{\text{EVENT} \quad (t, \text{dep}, t') \in E_{\text{dep}} \quad t \in v \in I \quad t' \in R \quad \text{Pos}(t) \in \{\text{NOUN}, \text{PROPN}\} \quad \text{dep} \in \{\text{advmod}, \text{neg}, \text{nummod}, \text{quantmod}, \text{npadvmod}, \text{case}, \text{compound}\}}{(I, R) \Rightarrow (I \setminus \{v\} \cup \{v \cup \{t'\}\}, R \setminus \{t'\})}$$

$$\frac{\text{ACTION} \quad (t, \text{dep}, t') \in E_{\text{dep}} \quad t \in v \in I \quad t' \in R \quad \text{Pos}(t) \in \{\text{VERB}, \text{AUX}\} \quad \text{dep} \in \{\text{aux}, \text{auxpass}, \text{neg}, \text{advmod}, \text{prt}, \text{acom}, \text{xcomp}, \text{ccomp}\}}{(I, R) \Rightarrow (I \setminus \{v\} \cup \{v \cup \{t'\}\}, R \setminus \{t'\})}$$

$$\frac{\text{ATTRIBUTE} \quad (t, \text{dep}, t') \in E_{\text{dep}} \quad t \in v \in I \quad t' \in R \quad \text{Pos}(t) = \text{ADJ} \vee (_, \text{amod}, t) \in E_{\text{dep}} \quad \text{dep} \in \{\text{advmod}, \text{neg}, \text{prep}, \text{conj}, \text{cc}, \text{det}\}}{(I, R) \Rightarrow (I \setminus \{v\} \cup \{v \cup \{t'\}\}, R \setminus \{t'\})}$$

The rules are designed to be applied in a specific conceptual hierarchy: extracting explicit entities first, initializing base semantic items, and subsequently expanding these items by absorbing their syntactic modifiers.

1. The Entity Rule. This rule prioritizes the extraction of recognized named entities (*e.g.*, PERSON, ORG, GPE). The premise evaluates whether a contiguous span of tokens belongs to a valid entity type ($\text{Ent}(t') \neq \perp$). Here, the function $\text{anc}(t, t')$ ensures that the tokens are structurally connected or adjacent within the span of the identified entity. If such a set v exists within the unresolved tokens R , the rule extracts these tokens collectively as a single cohesive vertex and removes them from R . This ensures that multi-word entities (*e.g.*, "New York") are not erroneously fragmented during subsequent syntactic parsing.

2. The Base-Item Rule. For tokens that do not belong to a named entity ($\text{Ent}(t) = \perp$), this rule acts as an initializer for semantic nodes. It scans the unresolved set R for tokens carrying primary semantic weight, specifically those tagged as nouns, verbs, adjectives, proper nouns, auxiliaries, numerals, or pronouns. Each qualifying token is instantiated as a standalone base vertex in I . These base items serve as the structural anchors for the subsequent expansion rules.

3. The Event Rule. This rule handles the expansion of nominal items (nouns and proper nouns) into broader event or object descriptors. Given an existing vertex v containing a nominal token t , the rule inspects the dependency arcs D . If an unresolved token t' modifies t through specific dependency relations, such as numeric

Table 8: Part-of-Speech (POS) Tag Set

Tag	Full Name	Description	Examples
ADP	Adposition	Words expressing spatial/temporal relations	in, on, at, of
ADV	Adverb	Modifiers of verbs, adjectives, or adverbs	quickly, very, well
ADJ	Adjective	Modifiers describing noun properties	big, happy, blue
AUX	Auxiliary	Verbs accompanying lexical verbs	is, have, will, can
CCONJ	Coordinating Conjunction	Connectors for syntactically equal elements	and, but, or
DET	Determiner	Words determining the reference of a noun	the, a, this, some
INTJ	Interjection	Words expressing emotion or exclamation	oh, hello, wow
NOUN	Noun	Words representing entities, places, or concepts	cat, book, London
NUM	Numeral	Words expressing quantities or sequence	one, 100, first
PART	Particle	Function words without standalone lexical meaning	's, not
PRON	Pronoun	Substitutes for nouns or noun phrases	I, you, he, it
PROPN	Proper Noun	Specific names of distinct entities	John, Paris, Google
PUNCT	Punctuation	Typographical symbols organizing text	., ,, !, ?
SCONJ	Subordinating Conjunction	Connectors introducing subordinate clauses	that, if, because
SYM	Symbol	Word-like symbols or special characters	%, %, &
VERB	Verb	Words expressing actions, events, or states	run, eat, think
X	Other	Words bypassing standard grammatical categorization	etc., ...
SPACE	Space	Whitespace characters	(space)

modifiers (nummod), quantifiers (quantmod), or case markers (case), t' is absorbed into the vertex v . For instance, the tokens "three" and "apples" are merged into a single vertex representing the quantified object.

4. The Action Rule. This rule reconstructs complete predicate structures. Verbs and auxiliaries often rely on dependent tokens to convey tense, voice, modality, or negation. If a token t within an existing vertex is a verb or auxiliary, the rule absorbs dependent tokens connected via relations such as passive auxiliaries (auxpass), negations (neg), verb particles (prt), or clausal complements (xcomp, ccomp). For example, the tokens "did", "not", and "run" or the phrasal verb "gave" and "up" are consolidated into unified action vertices, preserving the exact operational semantics of the document.

5. The Attribute Rule. This rule captures complex descriptive structures. It targets tokens that are either explicitly tagged as adjectives (ADJ) or act as adjectival modifiers (amod). The rule expands these attributes by absorbing their associated adverbs (advmod), negations (neg), prepositions (prep), or coordinating conjunctions (cc, conj). This mechanism effectively groups compound modifiers (e.g., "very happy" or "neither black nor white") into a single, highly expressive attribute vertex, preventing the loss of modifying context during the hyper simulation phase.

B.2 Deduction Rules for Hyperedge Clustering

This section details the formal rule set $\mathcal{R}_2$ utilized by the HCCalc algorithm (Algorithm 2) to combine individual hyperedges into semantic hyperedge clusters C_Q . In natural language, a single hyperedge often captures only a localized relation. The objective of $\mathcal{R}_2$ is to group syntactically dependent hyperedges to form cohesive, higher-order semantic contexts, such as clauses with their clausal complements or nouns with their relative clauses.

To formalize these rules, let E_Q be the set of hyperedges in the query hypergraph H_Q . For any hyperedge $e \in E_Q$, we denote its root vertex as $rt(e)$ and its set of child vertices as $ch(e) = e \setminus \{rt(e)\}$. Furthermore, we utilize the functions $Pos(\cdot)$, $dep(\cdot)$, and $head(\cdot)$

to retrieve the structural linguistic properties of a vertex.

The clustering system operates on a state configuration denoted as (E_{un}, C_Q) , where $E_{un} \subseteq E_Q$ is the set of currently unclustered hyperedges, and C_Q is the generated set of hyperedge clusters. The process initializes from state $(E_Q, \emptyset)$ and recursively applies the following prioritized rules until $E_{un} = \emptyset$. To prevent conflicting overlaps, the rules are evaluated in the strict sequential order presented below (Rule 1 $\rightarrow$ Rule 4).

PREDICATE-COMPLEMENT

$$\begin{array}{l}
 e \in E_{un} \quad Pos(rt(e)) \notin \{VERB, AUX\} \\
 E_{sub} = \{e' \in E_{un} \mid rt(e') \in ch(e) \wedge Pos(rt(e')) \in \{VERB, AUX\}\} \\
 E_{sub} \neq \emptyset \\
 \hline
 (E_{un}, C_Q) \Rightarrow (E_{un} \setminus (\{e\} \cup E_{sub}), C_Q \cup \{\{e\} \cup E_{sub}\})
 \end{array}$$

RELATIVE-CLAUSE

$$\begin{array}{l}
 e_1, e_2 \in E_{un} \\
 e_1 \neq e_2 \quad dep(rt(e_1)) = relcl \quad head(rt(e_1)) = rt(e_2) \\
 \hline
 (E_{un}, C_Q) \Rightarrow (E_{un} \setminus \{e_1, e_2\}, C_Q \cup \{\{e_1, e_2\}\})
 \end{array}$$

VERB-COMPLEMENT

$$\begin{array}{l}
 e_1, e_2 \in E_{un} \quad e_1 \neq e_2 \\
 Pos(rt(e_1)) \in \{VERB, AUX\} \quad Pos(rt(e_2)) \in \{VERB, AUX\} \\
 dep(rt(e_1)) \in \{advcl, ccomp\} \quad head(rt(e_1)) = rt(e_2) \\
 \hline
 (E_{un}, C_Q) \Rightarrow (E_{un} \setminus \{e_1, e_2\}, C_Q \cup \{\{e_1, e_2\}\})
 \end{array}$$

SINGLETON-FALLBACK

$$\begin{array}{l}
 e \in E_{un} \\
 \hline
 (E_{un}, C_Q) \Rightarrow (E_{un} \setminus \{e\}, C_Q \cup \{\{e\}\})
 \end{array}$$

1. The Predicate-Complement Rule. This rule handles the integration of nominal or attributive hyperedges with their verbal modifiers. The condition specifies that if the root of a hyperedge e is not a verb or auxiliary, but it possesses child vertices that serve

Table 9: Universal Dependency (DEP) Label Set

Category	Label	Full Name	Description
Core	ROOT	Root	The core predicate of the sentence.
	nsubj	Nominal Subject	The syntactic subject of an active clause.
	nsubjpass	Nominal Subject (Passive)	The syntactic subject of a passive clause.
	csubj	Clausal Subject	A subordinate clause acting as the subject.
	csubjpass	Clausal Subject (Passive)	A subordinate clause acting as a passive subject.
	dobj	Direct Object	The core argument undergoing the verb’s action.
	iobj	Indirect Object	The core argument receiving the direct object.
	pobj	Prepositional Object	The object governed by a preposition.
	dative	Dative	The recipient of an action (similar to indirect object).
Modifier	amod	Adjectival Modifier	An adjective modifying a noun phrase.
	advmod	Adverbial Modifier	An adverb modifying a predicate or adjective.
	nummod	Numeric Modifier	A numeral modifying a noun.
	quantmod	Quantifier Modifier	A quantifier modifying a noun.
	npadvmod	Noun Phrase Adverbial Modifier	A noun phrase functioning as an adverbial modifier.
	neg	Negation Modifier	A modifier conveying syntactic negation.
Clause	acl	Clausal Modifier	A finite or non-finite clause modifying a noun.
	advcl	Adverbial Clause	A clause modifying a predicate as a circumstance.
	ccomp	Clausal Complement	A dependent clause functioning as a core argument.
	xcomp	Open Clausal Complement	A clausal complement without an overt subject.
	relcl	Relative Clause	A relative clause modifying a noun.
	mark	Marker	A word introducing a subordinate clause.
Relation	prep	Preposition	A preposition preceding an object.
	agent	Agent	The logical subject of a passive clause.
	cc	Coordinating Conjunction	A conjunction linking coordinated structures.
	conj	Conjunct	The right-hand element of a coordinated structure.
	case	Case Marker	A word indicating the grammatical case.
	prt	Particle	A verb particle forming a phrasal verb.
Other	appos	Apposition	A nominal immediately following another for definition.
	attr	Attribute	A subject complement following a copular verb.
	acomp	Adjectival Complement	An adjectival phrase functioning as a complement.
	oprd	Object Predicate	An object complement.
	aux	Auxiliary	An auxiliary verb accompanying a lexical verb.
	auxpass	Auxiliary (Passive)	A passive auxiliary verb.
	expl	Expletive	A nominal fulfilling a syntactic role without meaning.
	parataxis	Parataxis	Loosely joined clauses without explicit coordination.
	meta	Meta Modifier	Meta-information modifiers (e.g., email subjects).
	det	Determiner	A determiner attached to a noun phrase.
	poss	Possessive Modifier	A modifier indicating possession.
	predet	Predeterminer	A modifier preceding a determiner.
	preconj	Preconjunction	The first element of a bipartite conjunction.
	intj	Interjection	An isolated expression of emotion.
	punct	Punctuation	Punctuation marks linking or separating text.
	compound	Compound	Elements forming a compound word.
	pcomp	Prepositional Complement	A complement attached to a preposition.
	nmod	Nominal Modifier	A noun functioning as a non-core modifier.
	dep	Unclassified Dependency	A syntactically unresolved dependency relation.

Table 10: Named Entity Recognition (ENT) Label Set

Tag	Full Name	Description	Examples
PERSON	Person	Real or fictional human names	Barack Obama, Sherlock Holmes
NORP	Nationalities/Religions/Political	Named nationalities, religious groups, political parties	American, Christian, Republican
FAC	Facility	Man-made structures, buildings, airports, highways	Golden Gate Bridge, Heathrow
ORG	Organization	Companies, agencies, institutions	Google, United Nations, NASA
GPE	Geo-Political Entity	Countries, cities, states	China, New York, California
LOC	Location	Non-GPE locations, mountain ranges, bodies of water	Mount Everest, Pacific Ocean
PRODUCT	Product	Objects, vehicles, foods, software	iPhone, Toyota Camry
EVENT	Event	Named historical, sporting, or natural events	Olympic Games, World War II
WORK_OF_ART	Work of Art	Titles of books, songs, movies, paintings	Mona Lisa, Harry Potter
LAW	Law	Named documents made into laws	First Amendment, Paris Agreement
LANGUAGE	Language	Any named human language	English, Mandarin, Spanish
DATE	Date	Absolute or relative dates and periods	January 1st, yesterday, next week
TIME	Time	Times smaller than a day	3:00 PM, two hours, noon
PERCENT	Percent	Percentage values (including "%")	50%, twenty percent
MONEY	Money	Monetary values, including unit	\$100, 50 euros
QUANTITY	Quantity	Measurements of weight, distance, etc.	10 kilometers, 5 pounds
ORDINAL	Ordinal	Words expressing an order in a series	first, second, third
CARDINAL	Cardinal	Numerals that do not fall under another type	one, 100, a dozen

as the roots of other verbal hyperedges (E_{sub}), they should be combined. This effectively aggregates structures like participial phrases or reduced relative clauses into a singular cluster, maintaining the descriptive semantic bond between the non-verb head and its verbal complements.

2. The Relative-Clause Rule. This rule captures explicit adjectival clauses that modify a noun phrase. If the root of a hyperedge e_1 acts as a relative clause (relcl) and structurally depends on the root of another hyperedge e_2 (the head noun), the algorithm merges e_1 and e_2 . For example, in the phrase "the book that I read", the hyperedge representing the "book" and the hyperedge representing the action "read" are syntactically distinct but semantically coupled; this rule fuses them to ensure holistic matching during hyper simulation.

3. The Verb-Complement Rule. This rule reconstructs compound events and complex predications. It identifies instances where a primary action is modified by a subordinate action. Specifically, if the roots of both hyperedges e_1 and e_2 are verbs or auxiliaries, and e_1 acts as an adverbial clause (advcl) or clausal complement (ccomp) to e_2 , they are clustered. This ensures that matrix clauses and their subordinate clauses (e.g., "She said [that] he left" or "She cried because she was sad") are evaluated as a unified semantic context rather than fragmented, isolated actions.

4. The Singleton-Fallback Rule. As the final stage of the sequential clustering process, this rule guarantees the exhaustiveness of C_Q . Any hyperedge e that does not satisfy the structural dependencies outlined in the preceding rules is packaged as a singleton cluster $\{e\}$. This ensures that simple, independent relationships within the document are preserved and correctly routed into the hyper simulation pipeline.

C Missing Proofs

C.1 Proof of Theorem 1

Proof: We establish the existence and uniqueness of the maximum hyper simulation through the following steps.

(*Existence, $\Rightarrow$*) Conceptually, a hyper simulation Π can be identified by defining the universal search space of candidate vertex pairs as the Cartesian product $V_1 \times V_2$, enumerating all possible binary relations $B \subseteq V_1 \times V_2$, and verifying whether each relation strictly satisfies the hyper simulation conditions with respect to h_v , HC, and $\mathcal{D}$. Once a valid match Π is found (e.g., the empty set $\emptyset$ trivially satisfies the conditions), it can be iteratively extended to a maximum one by testing the remaining candidate pairs in $V_1 \times V_2$. We add a candidate pair to Π if and only if the extension still forms a valid hyper simulation. Since the state space is inherently finite, this bounded extension guarantees that the maximum hyper simulation always exists. However, the naive approach of enumerating all possible binary relations requires exponential time, $\mathcal{O}(2^{|V_1|+|V_2|})$, which is computationally prohibitive for large hypergraphs. To resolve this, rather than generating and checking power sets, we develop an efficient polynomial-time fixed-point algorithm, HyperMatch (Algorithm 3), which systematically computes this maximum hyper simulation by pruning invalid candidates.

(*Uniqueness, $\Leftarrow$*) We prove the uniqueness of the maximum hyper simulation by contradiction. Assume that there exist two distinct maximum hyper simulations, denoted as Π_1 and Π_2 . Let $\Pi_3 = \Pi_1 \cup \Pi_2$. Since Π_1 and Π_2 are distinct maximums, their union Π_3 must have a strictly larger size than both, meaning $\Pi_1 \subset \Pi_3$ and $\Pi_2 \subset \Pi_3$.

We will now demonstrate that Π_3 itself is a valid hyper simulation. If this holds, it contradicts our initial assumption that Π_1 and Π_2 are maximums, thereby proving that the maximum hyper

simulation must be unique. To establish that Π_3 is a valid hyper simulation, we must verify that for any pair $(u, v) \in \Pi_3$, the foundational conditions of hyper simulation hold.

Given $(u, v) \in \Pi_3$, by the definition of a union, we have either $(u, v) \in \Pi_1$ or $(u, v) \in \Pi_2$. Without loss of generality, let us assume $(u, v) \in \Pi_1$. Since Π_1 is a valid hyper simulation, the following conditions hold for (u, v) :

- First, $h_v(u, v) = \text{true}$. This satisfies the first condition for Π_3 .
- Second, for each hyperedge cluster pair $(C_u, C_v) \in \text{HC}^{\sigma, b}(u, v)$, there exists a $\mathcal{D}$ -match $\mathcal{D}^\delta(C_u, C_v)$ such that $(u, v) \in \mathcal{D}^\delta(C_u, C_v)$; and for every $(u', v') \in \mathcal{D}^\delta(C_u, C_v)$, we have $(u', v') \in \Pi_1$.

Since $\Pi_3 = \Pi_1 \cup \Pi_2$, we naturally have $\Pi_1 \subseteq \Pi_3$. Consequently, any pair (u', v') that belongs to Π_1 also inherently belongs to Π_3 . Therefore, for every $(u', v') \in \mathcal{D}^\delta(C_u, C_v)$, it holds that $(u', v') \in \Pi_3$. This satisfies the second requirement for Π_3 . The exact same logical deduction applies if we alternatively assume $(u, v) \in \Pi_2$.

Putting this all together, we have verified that Π_3 satisfies all conditions of a hyper simulation. Since Π_3 strictly contains both Π_1 and Π_2 , this contradicts the assumption that Π_1 and Π_2 are maximum hyper simulations. Thus, the maximum hyper simulation with respect to $h_v, \text{HC}, \mathcal{D}, \sigma, b, \delta$ must be unique. $\square$

C.2 Proof of Theorem 2

Proof: We first analyze the computational time complexity of the algorithm. The outer logic requires enumerating all distinct vertex pairs (u, u') within the query hyperedge clusters. To establish an upper bound for this enumeration, we leverage the 2-section (or clique expansion) theory of hypergraphs. For any hypergraph $H = (V, E)$, its 2-section graph $[H]_2$ shares the same vertex set V , and an edge exists between two vertices if and only if they are co-contained in at least one hyperedge $e \in E$. The total number of edges in $[H]_2$ is strictly bounded by $\sum_{e \in E} \binom{|e|}{2} \leq \frac{1}{2} |E| \Delta_E^2$, where Δ_E is the maximum degree (size) of any hyperedge. Applying this property to the query hypergraph H_Q , the upper bound for the evaluated distinct vertex pairs (u, u') is $O(|E_Q| \Delta_E^2)$.

During the candidate matching phase, the algorithm filters data vertex pairs (v, v') based on the bounded likely function, i.e., $\text{likely}^b(u, v) = \text{true}$ and $\text{likely}^b(u', v') = \text{true}$. Given the search branch bound b , the number of valid data vertex pairs (v, v') for each query pair (u, u') is restricted to $O(b^2)$. Consequently, the total number of candidate evaluations, which corresponds to the total number of shortest path queries $|P|$, is bounded by $O(|E_Q| \Delta_E^2 b^2)$.

For each evaluation, constructing the candidate hyperedge cluster C_D requires finding the shortest path between v and v' in the data hypergraph H . By applying a breadth-first search (BFS) pre-processing strategy on the hypergraph, resolving these $|P|$ queries takes $O(|P|(|V| + |E| \Delta_E))$ time. Substituting the expression for $|P|$, the overall time complexity is $O(|E_Q| \Delta_E^2 b^2 (|V| + |E| \Delta_E))$. Considering that in the hypergraph representation of natural language texts, the branch bound b and the maximum hyperedge size Δ_E are typically sufficiently small constants, this bound asymptotically simplifies to $O(|E_Q|(|V| + |E|))$.

To establish the theoretical bound on the output set size $|\text{HC}^{\sigma, b}|$, we observe that for each valid candidate evaluation, the algo-

rithm inserts at most one matched pair (C_Q, C_D) into $\text{HC}^{\sigma, b}$. In the worst-case scenario, all evaluated data clusters C_D satisfy both the structural size constraint m and the semantic similarity threshold σ . Therefore, the cardinality of $\text{HC}^{\sigma, b}$ is inherently constrained by the maximum number of candidate evaluations, yielding $|\text{HC}^{\sigma, b}| \leq |P| = O(|E_Q| \Delta_E^2 b^2)$. By treating Δ_E and b as constants, the asymptotic space complexity of the hyperedge cluster set ultimately simplifies to $O(|E_Q|)$. $\square$

C.3 Proof of Theorem 3

Proof: (1) For the correctness of HyperMatch, we show that the algorithm is both sound (the output relation Π is a valid hyper simulation) and maximal (the output relation contains all valid hyper simulation pairs, i.e., $\Pi^* \subseteq \Pi$).

(Soundness, $\Rightarrow$) By definition, a relation is a valid hyper simulation if for every $(u, v) \in \Pi$, $h_v(u, v) = \text{true}$, and for all $(C_u, C_v) \in \text{HC}^{\sigma, b}(u, v)$, it holds that $\mathcal{D}^\delta(C_u, C_v) \subseteq \Pi$. Firstly, the algorithm initializes Π strictly with pairs satisfying $h_v(u, v) = \text{true}$ and structurally sound D-matches. Then, a pair (u, v) is retained in the final Π only if it is never pushed to the deletion queue Q . A pair is enqueued either during initialization (if $\text{HC}^{\sigma, b}(u, v) \not\subseteq \mathcal{V}_C$) or during propagation (if a previously valid cluster (C_u, C_v) is removed from $\mathcal{V}_C$ due to the deletion of a dependent pair). Consequently, for any pair (u, v) surviving in the final Π , all its associated hyperedge clusters maintain their presence in $\mathcal{V}_C$. This guarantees that $\mathcal{D}^\delta(C_u, C_v) \subseteq \Pi$ for all $(C_u, C_v) \in \text{HC}^{\sigma, b}(u, v)$. Thus, the final Π satisfies all conditions of hyper simulation.

(Maximality, $\Leftarrow$) We proceed by contradiction. Assume there exists at least one pair belonging to the true maximum hyper simulation Π^* that is incorrectly removed by the algorithm. Let $(u, v) \in \Pi^*$ be the first such pair removed during the execution sequence.

- *Case 1:* (u, v) is excluded during the initialization of Π . This implies either $h_v(u, v) \neq \text{true}$ or $(u, v) \notin \mathcal{D}^\delta(C_u, C_v)$ for some $(C_u, C_v) \in \text{HC}^{\sigma, b}(u, v)$. Both conditions strictly violate the prerequisites for hyper simulation, meaning (u, v) cannot exist in Π^* , which yields a contradiction.
- *Case 2:* (u, v) is removed during the queue initialization or Phase 2's propagation. The pair is removed since a required cluster $(C_u, C_v) \in \text{HC}^{\sigma, b}(u, v)$ was excluded from $\mathcal{V}_C$. This exclusion occurs strictly since some pair $(u', v') \in \mathcal{D}^\delta(C_u, C_v)$ was absent from or removed from Π . Since (u, v) is assumed to be the *first* pair from Π^* to be removed, the absent pair (u', v') must not belong to Π^* . However, the definition of hyper simulation dictates that if $(u, v) \in \Pi^*$, the entire set $\mathcal{D}^\delta(C_u, C_v)$ must be a subset of Π^* , mandating that $(u', v') \in \Pi^*$. This yields a logical contradiction.

Therefore, no pair from Π^* is ever removed, establishing $\Pi^* \subseteq \Pi$. Conclusively, the algorithm outputs exactly Π^* .

(2) For the complexity of HyperMatch, we evaluate the physical execution bounds of the pre-computation, the initialization sets, and the propagation loop using amortized analysis.

First, the algorithm pre-computes the hyperedge cluster pairs by invoking HCCalc. According to Theorem 2, this pre-computation takes $O(|E_Q|(|V| + |E|))$ time, and the size of the

output set $|HC^{\sigma,b}|$ is asymptotically bounded by $O(|E_Q|)$. Consequently, the aggregate sizes of the associated mapping structures, $\sum_{(u,v) \in V_Q \times V} |HC^{\sigma,b}(u,v)|$ and $\sum_{(C_u, C_v)} |\mathcal{D}^\delta(C_u, C_v)|$, are bounded by this pre-computation step, satisfying $\sum_{(u,v) \in V_Q \times V} |HC^{\sigma,b}(u,v)| + \sum_{(C_u, C_v)} |\mathcal{D}^\delta(C_u, C_v)| \leq O(|E_Q|(|V| + |E|))$.

During the declarative initialization, computing the initial Π requires verifying h_v and localized $\mathcal{D}$ -match constraints for all vertex pairs, taking $O(|V_Q||V| + \sum_{(u,v) \in V_Q \times V} |HC^{\sigma,b}(u,v)|)$ time. Next, populating the index $D_{cluster}$ evaluates the mappings defined by $HC^{\sigma,b}$, bounded by $O(\sum_{(u,v) \in V_Q \times V} |HC^{\sigma,b}(u,v)|)$. To evaluate the valid clusters $\mathcal{V}_C$ and populate D_{pair} , the algorithm traverses $\mathcal{D}^\delta(C_u, C_v)$ for all candidate clusters, taking $O(\sum_{(C_u, C_v)} |\mathcal{D}^\delta(C_u, C_v)|)$ time. Following this, extracting the initial invalid queue Q requires checking cluster validity for each pair’s dependencies, running in $O(\sum_{(u,v) \in V_Q \times V} |HC^{\sigma,b}(u,v)|)$. Thus, incorporating the bounds from Theorem 2, the entirety of the initialization operates within $O(|V_Q|(|V| + |E_Q|(|V| + |E|)))$.

Subsequently, in the while loop, each invalid pair (u', v') is enqueued into Q and popped exactly once. Upon popping, the algorithm iterates over its dependent clusters in $D_{pair}[(u', v')]$. Across the entire execution lifecycle, the total number of accesses to D_{pair} elements is strictly bounded by its aggregate size, $\sum_{(C_u, C_v)} |\mathcal{D}^\delta(C_u, C_v)|$. When a valid cluster is subsequently removed from $\mathcal{V}_C$, the algorithm scans $D_{cluster}[(C_u, C_v)]$. Since a cluster is invalidated at most once, this inner loop executes at most once per cluster. The aggregate operations of this inner loop over the entire algorithm are bounded by the total size of $D_{cluster}$, which is exactly $\sum_{(u,v) \in V_Q \times V} |HC^{\sigma,b}(u,v)|$. Therefore, the cascaded propagation completes in $O(\sum_{(u,v) \in V_Q \times V} |HC^{\sigma,b}(u,v)| + \sum_{(C_u, C_v)} |\mathcal{D}^\delta(C_u, C_v)|)$ time.

Summing the overhead of these steps, the aggregate structural terms $\sum_{(u,v) \in V_Q \times V} |HC^{\sigma,b}(u,v)|$ and $\sum_{(C_u, C_v)} |\mathcal{D}^\delta(C_u, C_v)|$ are strictly absorbed by the bounds established in Theorem 2. As a result, the total time complexity is strictly bounded by $O(|V_Q|(|V| + |E_Q|(|V| + |E|)))$. $\square$

D Full Examples

D.1 Examples for Hypergraph Representation

Example 9: Figure 3 illustrates the universal dependency parse of the corresponding document D_2 , computed via the spaCy [60] library. This dependency graph exhibits a tree structure rooted at the token “transferred”. According to the Pos mapping, its part-of-speech tag is VERB, indicating it acts as the primary predicate governing the entire sentence. The dependency arc (“transferred”, nsubj, “board”) originates from the root, identifying “board” as the nominal subject. Additionally, the arcs (“transferred”, prep, “to”) and (“to”, pobj, “corporation”) establish that “corporation” serves as the object of the preposition attached to the root verb. Furthermore, the text structure reveals a cascading modification: “corporation” is modified by “shell” via a compound relation, which in turn is modified by “offshore” via an amod (adjectival modifier) relation.

Concurrently, the named entity recognition mapping Ent extracts two semantic entities from D_2 : “board of directors” and “cor-

poration”. Both entities are classified under the ORG category, designating as organizations, institutions, or corporate bodies. $\square$

Example 10: Figure 4 illustrates the document hypergraphs constructed from the vector data entries in Figure 1.

(1) Consider the query H_{Q_1} regarding a historical event. The extraction process identifies key semantic units as vertices. For instance, the entity vertices include $\llbracket u_2 \rrbracket$ = “Emperor” (OCCUPATION), $\llbracket u_3 \rrbracket$ = “Holy Roman Empire” (COUNTRY), $\llbracket u_4 \rrbracket$ = “Pope” (PERSON), and $\llbracket u_5 \rrbracket$ = “Christmas Day” (TEMPORAL). Additionally, it captures interrogatives and predicates, such as $\llbracket u_0 \rrbracket$ = “who” and $\llbracket u_1 \rrbracket$ = “officially crowned”. The multi-way structural relationships are then modeled as directed hyperedges. For example, the 5-ary hyperedge $e_1 = (u_0, u_1, u_2, u_4, u_5)$ is anchored by the text sequence $\llbracket e_1 \rrbracket$ = “who was officially crowned Emperor by the Pope on Christmas Day”, while $e_2 = (u_2, u_3)$ captures the affiliation $\llbracket e_2 \rrbracket$ = “Emperor of the Holy Roman Empire”.

(2) Similarly, H_{Q_3} represents a complex factual statement involving multiple interconnected items. The extracted vertices encompass entities like $\llbracket u_0 \rrbracket$ = “Scholar Nilsson” (PERSON) and $\llbracket u_3 \rrbracket$ = “Stockholmsmässan, venue” (FAC), alongside actions ($\llbracket u_1 \rrbracket$ = “delivered”) and events ($\llbracket u_8 \rrbracket$ = “AI ethics seminar”). A complex 5-ary hyperedge $e_3 = (u_3, u_4, u_7, u_8, u_9)$ maps to the relation $\llbracket e_3 \rrbracket$ = “Stockholmsmassan on August hosted AI ethics seminar which featured”, demonstrating how complex event descriptions are topologically connected.

(3) The vertices illustrated above exhibit two critical characteristics in our formulation. First, a single vertex often encapsulates multiple adjacent tokens to represent a cohesive concept (e.g., $\llbracket u_8 \rrbracket$ = “AI ethics seminar”). Second, owing to the coreference resolution mapping CR introduced in Section 2, a vertex can span non-contiguous token groups across the document. For instance, in H_{Q_3} , the tokens “venue” and “Stockholmsmässan” refer to the same underlying entity; thus, they are unified into the single merged vertex u_3 .

(4) Furthermore, the hyperedges successfully overcome the binary limitations of standard graphs. Relationships in natural language, such as an action involving a subject, an object, a time, and a location, are inherently n -ary. For example, the hyperedge e'_1 in H_1 connects six distinct vertices to capture the complete factual context: “Pope Leo III officially crowned Charlemagne as Emperor in Rome on December 1st”. Additionally, hyperedges naturally share common vertices (e.g., u_2 is shared by e_1 and e_2 in H_{Q_1}), weaving an interconnected topological structure that accurately reflects the complex structural semantics of the text. $\square$

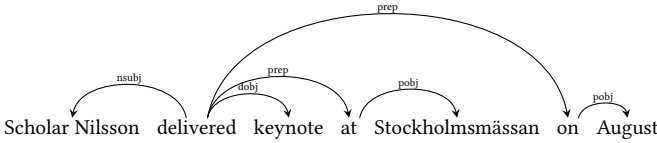
Example 11: We use the query Q_3 as a running example to illustrate the hypergraph construction process detailed in Algorithm 1. First, the algorithm extracts the universal dependency and Named Entity Recognition (NER) annotations of the input document.

Next, the VerticesDeduction function computes the initial vertex set V based on the dependency parsing results. During this process, NER first identifies specific entities, such that Ent(“Scholar Nilsson”) = PERSON, Ent(“August”) = DATE, and Ent(“Stockholmsmässan”) = FAC; these tokens are directly extracted as independent vertices. Subsequently, based on the de-

duction rules in $\mathcal{R}_1$, compound nouns are merged. For instance, “AI” $\xleftarrow{\text{compound}}$ “ethics” $\xleftarrow{\text{compound}}$ “seminar” is combined into a single vertex. For the phrase “roundtable” $\xleftarrow{\text{amod}}$ “discussions”, which exhibits an adjective-modifier relation, a specific rule in $\mathcal{R}_1$ calculates its bigram likelihood score. Since this score exceeds the predefined threshold, these tokens are also merged into a single vertex. The remaining single-token vertices are deduced based on their part-of-speech tags, encompassing verbs (VERB: hosted, delivered, featured, participated), nouns (NOUN: keynote, venue), and pronouns (PRON: he, his).

In the coreference resolution stage, the algorithm merges vertices that refer to the same real-world entity. Since {“venue”, “Stockholmsmässan”} belong to the same equivalence class u_3 , and {“Scholar Nilsson”, “he”, “his”} belong to the equivalence class u_0 , they are respectively mapped and combined into identical vertices. This process yields the refined vertex set $\tilde{V}$.

Finally, the algorithm performs a breadth-first search (BFS) over the dependencies D among the vertices in $\tilde{V}$ to compute the links map ψ . Specifically, this mapping includes $\psi(u_1) = \{u_0, u_2, u_3, u_4\}$, derived from the dependencies:



that derive paths “delivered” $\xrightarrow{\text{nsubj}}$ “Scholar Nilsson”, “delivered” $\xrightarrow{\text{dobj}}$ “keynote”, “delivered” $\xrightarrow{\text{prep}}$ “at” $\xrightarrow{\text{pobj}}$ “Stockholmsmässan”, and “delivered” $\xrightarrow{\text{prep}}$ “on” $\xrightarrow{\text{pobj}}$ “August”. The remaining mappings include $\psi(u_5) = \{u_0, u_6\}$, $\psi(u_7) = \{u_3, u_4, u_8, u_9\}$, and $\psi(u_9) = \{u_2, u_6\}$. Based on these mappings, the algorithm invokes the HyperedgesConstruction function to connect each non-leaf vertex u with all vertices in $\psi(u)$ (Algorithm 1, line 19), ultimately forming the complete set of hyperedges E . $\square$

D.2 Examples for Hyper Simulation Definition

Example 12: We utilize the hypergraphs illustrated in Figure 4 as a running example to introduce h_v , $\text{HC}^{\sigma, b}$, and $\mathcal{D}^\delta$.

(1) We examine the hypergraphs H_{Q_1} and H_1 . The vertex pairs that satisfy $h_v(u, v) = \text{true}$ encompass (u_0, v_0) , (u_0, v_2) , (u_1, v_1) , (u_2, v_3) , (u_3, v_4) , and (u_4, v_0) . The vertex $\llbracket u_0 \rrbracket = \text{“who”}$ acts as an interrogative pronoun that matches all vertices categorized as PERSON entities. Although $\text{Ent}(\llbracket u_5 \rrbracket) = \text{Ent}(\llbracket v_6 \rrbracket) = \text{TEMPORAL}$, a semantic conflict arises since $\llbracket v_6 \rrbracket = \text{“December 1st, 800 AD”}$ contradicts $\llbracket u_5 \rrbracket = \text{“Christmas Day”}$.

Let $C_1 = [e_1]$, $C'_1 = [e'_1]$, $C_2 = [e_2]$, and $C'_2 = [e'_2]$. The matched hyperedge clusters between Q_1 and D_1 consist of (C_1, C'_1) and (C_2, C'_2) . For $\llbracket C_1 \rrbracket = \text{crown}(\text{PERSON\#1}, \text{PERSON\#2}, \text{OCCUPATION}, \text{TEMPORAL})$ and $\llbracket C'_1 \rrbracket = \text{crown}(\text{PERSON\#1}, \text{PERSON\#2}, \text{OCCUPATION}, \text{TEMPORAL}, \text{LOC})$, the semantics of both hyperedge clusters converge to “PERSON#1 officially crown PERSON#2 a OCCUPATION during TEMPORAL”. Similarly, $\llbracket C_2 \rrbracket = \llbracket C'_2 \rrbracket = \text{belongsTo}(\text{OCCUPATION}, \text{COUNTRY})$ indicates that their shared semantics align with “the OCCUPATION belongs to

the COUNTRY”.

Consequently, we obtain $\mathcal{D}^\delta(C_1, C'_1) = \{(u_0, v_2), (u_4, v_0), (u_2, v_3), (u_5, v_6)\}$. This result stems from the fact that within the actions denoted by the two predicates as “officially crown”, the matched vertices possess identical types and fulfill identical roles. Specifically, both vertices in the pair (u_0, v_2) act as the agents of the action, whereas the vertices in (u_4, v_0) serve as the patients. Additionally, the pair (u_2, v_3) represents the state of the action, and (u_5, v_6) designates the temporal context. By the same logic, we derive $\mathcal{D}^\delta(C_2, C'_2) = \{(u_2, v_3), (u_3, v_4)\}$, since the pairs associated with this relation share identical types and semantic responsibilities.

(2) We consider the hypergraphs H_{Q_2} and H_2 . The vertex pairs that satisfy $h_v(u, v) = \text{true}$ encompass (u_0, v_0) and (u_1, v_1) . Although $\text{Ent}(\llbracket u_3 \rrbracket) = \text{Ent}(\llbracket v_2 \rrbracket) = \text{ORG}$, an entity mismatch arises since $\llbracket v_2 \rrbracket = \text{“corporation”}$ specifies a contradictory target object compared to $\llbracket u_3 \rrbracket = \text{“open-source foundation”}$.

Let $C_1 = [e_1]$ and $C'_1 = [e'_1]$. The matched hyperedge clusters between H_{Q_2} and H_2 consist of (C_1, C'_1) . For $\llbracket C_1 \rrbracket = \text{transfer}(\text{ORG\#1}, \text{CONCEPT}, \text{ORG\#2})$ and $\llbracket C'_1 \rrbracket = \text{transfer}(\text{ORG\#1}, \text{ORG\#2})$, the semantics of C_1 converge to a binary object relation where the CONCEPT is transferred to ORG#2. In contrast, C'_1 converges to a simplified structure that denotes ORG#1 transferred to ORG#2.

Consequently, we obtain $\mathcal{D}^\delta(C_1, C'_1) = \{(u_0, v_0), (u_3, v_2)\}$. This result occurs since within the actions denoted by the transfer predicate, the matched vertices possess identical types and fulfill identical structural roles. Specifically, both vertices in the pair (u_0, v_0) act as the agents that initiate the action. Additionally, the pair (u_3, v_2) serves as the recipient or destination of the transfer.

(3) We evaluate the hypergraphs H_{Q_3} and H_3 . The vertex pairs that satisfy $h_v(u, v) = \text{true}$ encompass (u_0, v_0) , (u_1, v_1) , (u_2, v_2) , (u_3, v_3) , (u_4, v_4) , (u_5, v_5) , (u_6, v_6) , (u_7, v_7) , and (u_9, v_9) . Although $\text{Ent}(\llbracket u_8 \rrbracket) = \text{Ent}(\llbracket v_8 \rrbracket) = \text{EVENT}$, a semantic conflict arises since $\llbracket v_8 \rrbracket = \text{“quantum computing seminar”}$ introduces a distinct topic that contradicts $\llbracket u_8 \rrbracket = \text{“AI ethics seminar”}$.

Let $C_1 = [e_1]$, $C'_1 = [e'_1]$, $C_2 = [e_2]$, $C'_2 = [e'_2]$, $C_3 = [e_3 + e_4]$, and $C'_3 = [e'_3 + e'_4]$. The matched hyperedge clusters between H_{Q_3} and H_3 consist of (C_1, C'_1) , (C_2, C'_2) , and (C_3, C'_3) . For $\llbracket C_1 \rrbracket = \text{deliver}(\text{PERSON}, \text{EVENT}, \text{FAC}, \text{TEMPORAL})$ and $\llbracket C'_1 \rrbracket = \text{give}(\text{PERSON}, \text{EVENT}, \text{FAC}, \text{TEMPORAL})$, the semantics of both hyperedge clusters converge to “PERSON conducts an EVENT at a FAC during TEMPORAL”. Similarly, $\llbracket C_2 \rrbracket = \text{participate}(\text{PERSON}, \text{EVENT})$ and $\llbracket C'_2 \rrbracket = \text{engage}(\text{PERSON}, \text{EVENT})$ indicate that their shared semantics align with “the PERSON joins the EVENT”. For the composite clusters, $\llbracket C_3 \rrbracket = \text{host}(\text{FAC}, \text{EVENT\#1}, \text{TEMPORAL}) \wedge \text{feature}(\text{EVENT\#1}, \text{EVENT\#2}, \text{EVENT\#3})$ and $\llbracket C'_3 \rrbracket = \text{host}(\text{FAC}, \text{EVENT\#1}, \text{TEMPORAL}) \wedge \text{include}(\text{EVENT\#1}, \text{EVENT\#2}, \text{EVENT\#3})$ demonstrate a complex structural alignment that represents “the FAC hosts EVENT#1 during TEMPORAL, which encompasses EVENT#2 and EVENT#3”.

Consequently, we obtain $\mathcal{D}^\delta(C_1, C'_1) = \{(u_0, v_0), (u_2, v_2), (u_3, v_3), (u_4, v_4)\}$. This result occurs since within the actions denoted by the primary predicates, the matched vertices possess identical types and fulfill identical structural roles. Specifically, both vertices in the pair (u_0, v_0) act as the agents that initiate the action, whereas the vertices in (u_2, v_2) serve as the objects of the action.

Additionally, the pair (u_3, v_3) represents the spatial location, and (u_4, v_4) designates the temporal context. By the same logic, we derive $\mathcal{D}^\delta(C_2, C'_2) = \{(u_0, v_0), (u_6, v_6)\}$, which correctly maps the agent and the participated event. Finally, for the composite relation, we obtain $\mathcal{D}^\delta(C_3, C'_3) = \{(u_3, v_3), (u_8, v_8), (u_4, v_4), (u_2, v_2), (u_6, v_6)\}$. Here, (u_3, v_3) and (u_4, v_4) map the spatial and temporal conditions, whereas the remaining pairs map the hierarchical event structures. The structural $\mathcal{D}$ -match rigorously forces the alignment of the query's primary event $\llbracket u_8 \rrbracket$ to the document's primary event $\llbracket v_8 \rrbracket$. This structural binding successfully exposes the semantic contradiction between the "AI ethics seminar" and the "quantum computing seminar" during the subsequent hyper simulation phase.

(4) We analyze the multi-hop query H_{Q_4} and the retrieved hypergraphs H_4 , H_5 , and H_6 . The vertex pairs that satisfy $h_v(u, v) = \text{true}$ encompass (u_0, v_5) , (u_0, v_9) , (u_1, v_7) , (u_1, v_{12}) , and (u_2, v_0) . Although $\text{Ent}(\llbracket u_2 \rrbracket) = \text{Ent}(\llbracket v_8 \rrbracket) = \text{ORG}$, a semantic conflict arises since $\llbracket v_8 \rrbracket = \text{"Verdant Canopy ecological conservation park"}$ specifies a distinct organization that contradicts the target query entity $\llbracket u_2 \rrbracket = \text{"Verdant Canopy expedition"}$.

Let $C_1 = [e_1]$ denote the query cluster. For the data clusters, we evaluate a multi-document synthesis $C'_1 = [e'_1 + e'_6 + e'_7]$ derived from D_4 and D_6 , alongside a single-document cluster $C'_2 = [e'_4 + e'_5]$ derived from H_5 . For $\llbracket C_1 \rrbracket = \text{belongsTo}(\text{PERSON}, \text{OCCUPATION}, \text{ORG})$, the semantics converge to a structural dependency where a PERSON holds an OCCUPATION associated with the ORG. The composite cluster $\llbracket C'_1 \rrbracket = \text{funded\&organized}(\text{ORG\#1}, \text{ORG\#2}) \wedge \text{serves}(\text{PERSON}, \text{OCCUPATION\#1}, \text{OCCUPATION\#2}) \wedge \text{property}(\text{OCCUPATION\#2}, \text{ORG\#2})$ logically bridges the query by demonstrating that "ORG#1 is funded and organized by ORG#2, where the PERSON serves as OCCUPATION#2". Conversely, the alternative cluster $\llbracket C'_2 \rrbracket = \text{belongsTo}(\text{PERSON}, \text{OCCUPATION}) \wedge \text{property}(\text{OCCUPATION}, \text{ORG})$ establishes a direct structural alignment equivalent to "PERSON is the OCCUPATION of the ORG".

Consequently, the $\mathcal{D}$ -match evaluates to $\mathcal{D}^\delta(C_1, C'_1) = \{(u_0, v_9), (u_1, v_{12}), (u_2, v_0)\}$. This result originates from the structural reasoning path that bridges D_4 and D_6 via the shared entity $\llbracket v_4 \rrbracket = \text{"Sylva Research Foundation"}$. Specifically, u_0 maps to the serving agent v_9 , u_1 aligns with the target occupation v_{12} , and u_2 binds to the indirectly managed organization v_0 . Alternatively, for the single-document candidate, we obtain $\mathcal{D}^\delta(C_1, C'_2) = \{(u_0, v_5), (u_1, v_7), (u_2, v_8)\}$. The vertices in this mapping structurally fulfill identical roles, which represent the human agent, the occupation, and the organizational target. However, the structural $\mathcal{D}$ -match rigorously mandates the alignment of u_2 to v_8 . Since $h_v(u_2, v_8)$ fails the non-conflict evaluation due to the aforementioned organizational mismatch, the hyper simulation algorithm systematically prunes this contradictory reasoning path. This operation correctly prioritizes the logically consistent synthesis derived from D_4 and D_6 . $\square$

Example 13: We now provide an analysis of the hyper simulation results between the hypergraphs depicted in Figure 4.

(1) The hyper simulation relation Π between the hypergraphs H_{Q_1} and H_1 evaluates to $\emptyset$. Initially, according to Condition (1), the candidate vertex pairs that satisfy the non-conflict evaluation $h_v(u, v) = \text{true}$ form the set

$\{(u_0, v_0), (u_0, v_2), (u_1, v_1), (u_2, v_3), (u_3, v_4), (u_4, v_0)\}$. However, the temporal pair (u_5, v_6) yields $h_v(u_5, v_6) = \text{false}$ due to a semantic contradiction, which indicates $(u_5, v_6) \notin \Pi$. This initial exclusion triggers a cascade of violations under Condition (2). Specifically, for the primary action clusters C_1 and C'_1 , the established $\mathcal{D}$ -match $\mathcal{D}^\delta(C_1, C'_1) = \{(u_0, v_2), (u_4, v_0), (u_2, v_3), (u_5, v_6)\}$ requires all its constituent pairs to be present in Π to satisfy Condition (2)(b). Since $(u_5, v_6) \notin \Pi$, this $\mathcal{D}$ -match becomes invalid, and it consequently excludes dependent pairs such as (u_0, v_2) , (u_4, v_0) , and (u_2, v_3) from the simulation relation. This invalidation further propagates to the topological dependencies of the hypergraphs. For the belongsTo relation clusters C_2 and C'_2 , their corresponding $\mathcal{D}$ -match $\{(u_2, v_3), (u_3, v_4)\}$ violates Condition (2)(b) due to the absence of (u_2, v_3) , and this leads to the removal of (u_3, v_4) . Finally, the remaining candidate (u_0, v_0) fails to satisfy Condition (2)(a) since it is absent from the structural mapping $\mathcal{D}^\delta(C_1, C'_1)$. Consequently, since every initial candidate pair violates either the structural mapping constraints or the recursive consistency conditions, the maximum hyper simulation relation evaluates to $\Pi = \emptyset$.

(2) We detail the verification process for H_{Q_2} versus H_2 , where their hyper simulation relation Π also evaluates to $\emptyset$. During the initial phase guided by Condition (1), the vertex pairs (u_0, v_0) and (u_1, v_1) are retained in the candidate relation. This retention occurs since both pairs satisfy the non-conflict prerequisite $h_v(u, v) = \text{true}$. Specifically, u_0 and v_0 represent the entity *board of directors* [ORG], and u_1 and v_1 both denote the action *transferred*. However, these candidates are pruned during the structural verification phase defined by Condition (2). To satisfy this recursive structural dependency, any pair $(u, v) \in \Pi$ must be supported by a valid structural $\mathcal{D}$ -match, denoted as $\mathcal{D}^\delta(C_u, C_v)$, between their corresponding hyperedge clusters, which are $e_1 \in Q_2$ and $e'_1 \in D_2$ in this scenario. Formally, establishing this match requires that every adjacent vertex within e_1 , including u_3 (*open-source foundation* [ORG]), finds a valid matching counterpart in e'_1 . Given that the potential target $v_2 \in e'_1$ constitutes a semantic entity mismatch (*corporation* [ORG]), the mapping for u_3 fails. As a consequence, no valid $\mathcal{D}$ -match can be constructed to encompass (u_0, v_0) and (u_1, v_1) . Since this violates the structural constraint of Condition (2), both candidate pairs are eliminated, and the final simulation relation becomes $\Pi = \emptyset$.

(3) We detail the hyper simulation process for H_{Q_3} versus H_3 , which results in $\Pi = \emptyset$. Initially, Condition (1) retains nine candidate vertex pairs, specifically (u_0, v_0) through (u_7, v_7) and (u_9, v_9) , since they satisfy the semantic non-conflict property $h_v(u, v) = \text{true}$. The vertex pair (u_8, v_8) fails this initial semantic check and is consequently excluded from the candidate relation Π . This omission triggers a cascading invalidation during the structural verification phase. According to Condition (2), establishing a valid $\mathcal{D}$ -match for the hyperedge cluster pair (C_3, C'_3) requires that all its constituent mappings, including (u_8, v_8) , exist within Π . Since $(u_8, v_8) \notin \Pi$, the structural mapping for (C_3, C'_3) becomes invalid, and it mandates the removal of its dependent pairs (u_2, v_2) , (u_3, v_3) , (u_4, v_4) , and (u_6, v_6) , along with their topologically bound action vertices (u_7, v_7) and (u_9, v_9) . The removal of these vertices destroys the topological integrity of adjacent clusters. Specifically, the $\mathcal{D}$ -match for (C_1, C'_1) , which relies on the deleted pairs (u_2, v_2) , (u_3, v_3) , and (u_4, v_4) , be-

comes invalid and forces the elimination of (u_0, v_0) and (u_1, v_1) . This propagation reaches (C_2, C'_2) , where the prerequisite mapped pairs (u_0, v_0) and (u_6, v_6) are no longer available. This results in the removal of the remaining action pair (u_5, v_5) . Consequently, the recursive structural constraints empty the candidate set and yield a final hyper simulation relation of $\Pi = \emptyset$.

(4) Finally, we verify the hyper simulation relation for H_{Q_4} versus H_4 , which converges to a non-empty relation $\Pi = \{(u_0, v_9), (u_1, v_{12}), (u_2, v_0)\}$. During the initialization phase, the candidate relation Π is populated with five provisional vertex pairs: (u_0, v_5) , (u_0, v_9) , (u_1, v_7) , (u_1, v_{12}) , and (u_2, v_0) . These pairs are retained since they satisfy the semantic non-conflict prerequisite mandated by Condition (1), formally expressed as $h_v(u, v) = \text{true}$. This ensures that the data graph vertices encompass the entity or action semantics defined by their query counterparts. However, the enforcement of Condition (2) as the recursive structural matching constraint necessitates topological pruning. For a pair to persist in Π , its corresponding hyperedge clusters must exhibit a valid $\mathcal{D}$ -match where all structurally dependent vertex pairs exist within Π . When evaluating (u_0, v_5) , the structural verification fails since the local topological neighborhood of v_5 in the data graph cannot construct a complete $\mathcal{D}$ -match that maps to the adjacent query vertices using only the available candidates in Π . A similar topological deficiency occurs in the neighborhood of v_7 , and it renders the requisite $\mathcal{D}$ -match for (u_1, v_7) invalid. Lacking the necessary structural support to satisfy Condition (2), both (u_0, v_5) and (u_1, v_7) are eliminated from the candidate set.

Following the elimination of these structurally unsupported candidates, the relation converges to a refined set $\Pi = \{(u_0, v_9), (u_1, v_{12}), (u_2, v_0)\}$. To establish this as a valid hyper simulation, we must verify that this final set adheres to the recursive definitions without triggering further cascading deletions. Condition (1) remains satisfied by the initial semantic filtering. Condition (2) is fulfilled since the remaining pairs form a self-consistent topological closure. For instance, the structural $\mathcal{D}$ -match required to sustain (u_0, v_9) depends on the valid mappings of its adjacent vertices. These dependencies are fulfilled by (u_1, v_{12}) and (u_2, v_0) , which are both present in Π . Symmetrically, the $\mathcal{D}$ -matches for (u_1, v_{12}) and (u_2, v_0) are supported by the other pairs within this tri-vertex closure. Since every vertex pair in this converged set can locate its structurally mandated dependencies internally, the set guarantees topological preservation across all involved hyperedge clusters. This self-contained structural validation proves that the final relation $\Pi = \{(u_0, v_9), (u_1, v_{12}), (u_2, v_0)\}$ constitutes a mathematically sound hyper simulation. $\square$

D.3 Examples for Implementation of Hyper Simulation

Example 14: We now outline the HCCalc process using examples.

(1) For H_{Q_3} and H_3 , the algorithm HCCalc computes the hyperedge cluster mapping $\text{HC}^{\sigma, b}$ as $(C_1 = [e_1], C'_1 = [e'_1])$, $(C_2 = [e_2], C'_2 = [e'_2])$, and $(C_3 = [e_3 + e_4], C'_3 = [e'_3 + e'_4])$. Specifically, the algorithm derives C_3 from e_3 and e_4 based on syntactic rules at Line 2. This derivation originates from the attributive clause structure introduced by “which” in the source

document, and the universal dependency parser identifies this structure as $\text{dep} = \text{relcl}$. Consequently, the process yields the query hyperedge cluster set $C_Q = \{C_1, C_2, C_3\}$. Regarding the first cluster pair, we observe $u_0, u_3 \in C_1$ alongside the matching predicates $\text{likely}^b(u_0, v_0) = \text{true}$ and $\text{likely}^b(u_3, v_3) = \text{true}$. Within the data hypergraph H_3 , the vertices v_0 and v_3 belong to e'_1 , which establishes the hyperedge cluster $[e'_1] = C'_1$. Furthermore, the semantic similarity satisfies the threshold, as $\cos(\llbracket C_1 \rrbracket, \llbracket C'_1 \rrbracket) = \cos(\text{deliver}(\text{PERSON}, \text{EVENT}, \text{FAC}, \text{TEMPORAL}), \text{give}(\text{PERSON}, \text{EVENT}, \text{FAC}, \text{TEMPORAL})) = 0.966 > \sigma$. Similarly for the second cluster pair, we identify $u_0, u_6 \in C_2$ with $\text{likely}^b(u_0, v_0) = \text{true}$ and $\text{likely}^b(u_6, v_6) = \text{true}$. In H_3 , the corresponding vertices v_0 and v_6 reside in e'_2 , and this forms hyperedge cluster $[e'_2] = C'_2$. This pair also exceeds the similarity threshold, with $\cos(\llbracket C_2 \rrbracket, \llbracket C'_2 \rrbracket) = \cos(\text{participate}(\text{PERSON}, \text{EVENT}), \text{engage}(\text{PERSON}, \text{EVENT})) = 0.926 > \sigma$. Finally, for the third cluster pair, we have $u_8, u_6 \in C_3$ along with $\text{likely}^b(u_8, v_8) = \text{true}$ and $\text{likely}^b(u_6, v_6) = \text{true}$. In H_3 , the shortest path connecting v_3 to v_6 traverses $v_3, e'_3, v_9, e'_4, v_6$, and this topology defines the hyperedge cluster $[e'_3 + e'_4] = C'_3$. The semantic evaluation confirms this structural alignment, as $\cos(\llbracket C_3 \rrbracket, \llbracket C'_3 \rrbracket) = \cos(\text{host}(\text{FAC}, \text{EVENT\#1}, \text{TEMPORAL}) \wedge \text{feature}(\text{EVENT\#1}, \text{EVENT\#2}, \text{EVENT\#3}), \text{host}(\text{FAC}, \text{EVENT\#1}, \text{TEMPORAL}) \wedge \text{include}(\text{EVENT\#1}, \text{EVENT\#2}, \text{EVENT\#3})) = 0.987 > \sigma$.

(2) For H_{Q_4} and the data graphs H_4 , H_5 , and H_6 , the algorithm HCCalc computes two candidate hyperedge cluster mappings for the single query cluster $C_1 = [e_1]$. These mappings emerge as $(C_1 = [e_1], C'_1 = [e'_1 + e'_6 + e'_7])$ and $(C_1 = [e_1], C''_1 = [e'_4 + e'_5])$. Specifically, the algorithm derives these expanded data clusters by traversing topological structures both within and across the source documents. Consequently, the process yields the query hyperedge cluster set $C_Q = \{C_1\}$. Regarding the first cluster pair, we observe $u_0, u_2 \in C_1$ alongside the matching predicates $\text{likely}^b(u_0, v_9) = \text{true}$ and $\text{likely}^b(u_2, v_0) = \text{true}$. Within the data hypergraphs, connecting the vertices v_9 and v_0 requires traversing e'_1, e'_6 , and e'_7 , which establishes the hyperedge cluster $[e'_1 + e'_6 + e'_7] = C'_1$. Furthermore, the semantic similarity satisfies the threshold, as $\cos(\llbracket C_1 \rrbracket, \llbracket C'_1 \rrbracket) = \cos(\text{belongsTo}(\text{PERSON}, \text{OCCUPATION}, \text{ORG}), \text{funded \& organized}(\text{ORG\#1}, \text{ORG\#2}) \wedge \text{serves}(\text{PERSON}, \text{OCCUPATION\#1}, \text{OCCUPATION\#2}) \wedge \text{property}(\text{OCCUPATION\#2}, \text{ORG\#2})) = 0.839 > \sigma$. Similarly for the second candidate cluster pair, we identify $u_0, u_1 \in C_1$ with $\text{likely}^b(u_0, v_5) = \text{true}$ and $\text{likely}^b(u_1, v_7) = \text{true}$. In H_5 , the corresponding vertices v_5 and v_7 (extending to v_8) reside across e'_4 and e'_5 , and this forms $[e'_4 + e'_5] = C''_1$. This pair also exceeds the similarity threshold, with $\cos(\llbracket C_1 \rrbracket, \llbracket C''_1 \rrbracket) = \cos(\text{belongsTo}(\text{PERSON}, \text{OCCUPATION}, \text{ORG}), \text{belongsTo}(\text{PERSON}, \text{OCCUPATION}) \wedge \text{property}(\text{OCCUPATION}, \text{ORG})) = 0.932 > \sigma$. $\square$

Example 15: Now, we present an example illustrating the execution of Algorithm 3.

(1) We first detail the execution of HyperMatch for H_{Q_3} versus H_3 , which ultimately yields $\Pi = \emptyset$. During the declarative initialization phase (lines 1-11), the algorithm evaluates the semantic non-conflict condition $h_v(u, v) = \text{true}$. The vertex pair (u_8, v_8) fails this evaluation and is excluded, initializing the tentative relation Π with the remaining nine candidate pairs. Next, the auxiliary indices are constructed. Since $(u_8, v_8) \notin \Pi$, the structural $\mathcal{D}$ -match

for the hyperedge cluster pair (C_3, C'_3) is incomplete, meaning $\mathcal{D}^\delta(C_3, C'_3) \not\subseteq \Pi$. Consequently, (C_3, C'_3) is excluded from the valid cluster set $\mathcal{V}_C$, while structurally intact clusters like (C_1, C'_1) and (C_2, C'_2) are added. The dependency mappings are then initialized: $D_{cluster}$ maps (C_3, C'_3) to its dependent vertex pairs such as (u_2, v_2) and (u_3, v_3) , and D_{pair} maps these vertices back to their requisite clusters. Finally, the algorithm initializes the queue Q by identifying all pairs in Π that depend on the invalid cluster (C_3, C'_3) . These unsupported pairs— (u_2, v_2) , (u_3, v_3) , (u_4, v_4) , (u_6, v_6) , (u_7, v_7) , and (u_9, v_9) —are pushed into Q and concurrently removed from the working relation Π .

In the second phase, the algorithm enters a while loop to process cascading deletions via the queue Q (lines 12-19). As an invalid pair, such as (u_2, v_2) , is popped from Q , the algorithm queries D_{pair} and discovers that the currently valid cluster $(C_1, C'_1) \in \mathcal{V}_C$ relies on this removed pair to satisfy its $\mathcal{D}$ -match. Consequently, the required structural dependency is broken, (C_1, C'_1) is invalidated, and it is removed from $\mathcal{V}_C$. The algorithm then consults $D_{cluster}[(C_1, C'_1)]$ to find surviving dependent candidate pairs, specifically (u_0, v_0) and (u_1, v_1) , which are subsequently removed from Π and pushed onto Q to propagate the deletion. Similarly, dequeuing (u_6, v_6) triggers the invalidation of (C_2, C'_2) via the D_{pair} index, which in turn forces its dependent pair (u_5, v_5) out of Π and into Q . This recursive enqueue and dequeue process systematically dismantles the topological structure until Q is depleted, resulting in a finalized hyper simulation relation of $\Pi = \emptyset$.

(2) Conversely, the execution for H_{Q_4} versus H_4 demonstrates a convergence to a non-empty hyper simulation relation. In the initialization phase, the semantic check retains five provisional vertex pairs: (u_0, v_5) , (u_0, v_9) , (u_1, v_7) , (u_1, v_{12}) , and (u_2, v_0) . The algorithm then populates the auxiliary structures. Let $C_{invalid}$ denote the hyperedge clusters associated with (u_0, v_5) and (u_1, v_7) ; since their local data graph neighborhoods fail to form a complete $\mathcal{D}$ -match from the candidates in Π , these clusters are omitted from $\mathcal{V}_C$. In contrast, the clusters associated with (u_0, v_9) , (u_1, v_{12}) , and (u_2, v_0) internally satisfy their $\mathcal{D}$ -matches and are successfully added to $\mathcal{V}_C$. The dictionaries $D_{cluster}$ and D_{pair} index these dependencies accordingly. Since (u_0, v_5) and (u_1, v_7) depend on the invalid clusters in $C_{invalid}$, they are identified as structurally unsupported. These two pairs are pushed onto the queue Q and excised from Π , refining the tentative relation to $\Pi = \{(u_0, v_9), (u_1, v_{12}), (u_2, v_0)\}$.

During the iterative execution phase, the algorithm processes the structurally unsupported pairs in Q to enforce recursive pruning. The pair (u_0, v_5) is popped from the queue, and the algorithm queries D_{pair} to identify any valid clusters in $\mathcal{V}_C$ that depend on it. However, the remaining vertex pairs in Π form a self-consistent topological closure; their required clusters in $\mathcal{V}_C$ rely exclusively on other members of the closure $\{(u_0, v_9), (u_1, v_{12}), (u_2, v_0)\}$ and do not intersect with the removed pair (u_0, v_5) . Since this intersection is empty, no valid clusters are compromised. The same structural isolation applies when (u_1, v_7) is dequeued. As no clusters in $\mathcal{V}_C$ are invalidated, no further candidate pairs are removed from Π , and no new elements are pushed onto Q . The queue quickly empties without triggering a cascading deletion, proving that the remaining elements are topologically self-contained. The loop terminates naturally, and the algorithm returns the maximum hyper simulation

relation $\Pi = \{(u_0, v_9), (u_1, v_{12}), (u_2, v_0)\}$. $\square$

E Details of Document Marking

We next detail how the results of hyper simulation are converted into marks for retrieved documents. Each mark is computed with respect to the current query and consists of a category and a reason for the assignment. It therefore describes how a document participates in the hyper simulation.

E.1 Definitions of the Four Categories

For each retrieved document D_i represented by H_i , let $\Pi^{(0)} := \{(u, v) \in H_Q.V \times H.V \mid h_v(u, v) = \text{true}\}$ denote the tentative relation after candidate generation and the evaluation of h_v , but before the fixed-point pruning of HyperMatch. We define

$$\Pi_i = \{(u, v) \in \Pi \mid u \in H_Q.V \wedge v \in H_i.V\}$$

Thus, Π_i contains exactly the pairs in the final Π whose second vertex belongs to $H_i.V$.

We further define

$$\Delta_i = \{(u, v) \in \Pi^{(0)} \setminus \Pi \mid v \in H_i.V\}$$

For every $(u, v) \in \Delta_i$, the pair belongs to $\Pi^{(0)}$, its second vertex belongs to $H_i.V$, and it does not belong to the final Π . Since $\Pi^{(0)}$ is obtained after evaluating h_v , such a pair satisfies $h_v(u, v) = \text{true}$ but is subsequently removed during the fixed-point computation because Condition (2) fails.

We define

$$\mathcal{R}_i = \{(C_Q, C_D) \in \text{HC}^{\sigma, b} \mid C_D.E \cap H_i.E \neq \emptyset \wedge \mathcal{D}^\delta(C_Q, C_D) \subseteq \Pi\}.$$

where $C_D.E$ denotes the hyperedges contained in C_D . For every $(C_Q, C_D) \in \mathcal{R}_i$, at least one hyperedge in C_D belongs to $H_i.E$, and every vertex pair in the $\mathcal{D}^\delta(C_Q, C_D)$ belongs to Π .

Let $\mathcal{K} \subseteq \bigcup_{j=1}^k \mathcal{R}_j$ be a set of hyperedge cluster pairs, a consistent vector query instance I requires

$$\bigcup_{(C_Q, C_D) \in \mathcal{K}} C_Q.E = H_Q.E$$

and

$$\left\{ u \in H_Q.V \mid \begin{array}{l} \exists (C_Q, C_D) \in \mathcal{K}, \exists v \in C_D.V : \\ (u, v) \in \mathcal{D}^\delta(C_Q, C_D) \end{array} \right\} = H_Q.V.$$

The first equality states that, for every $e \in H_Q.E$, some $(C_Q, C_D) \in \mathcal{K}$ satisfies $e \in C_Q.E$. The second equality states that, for every $u \in H_Q.V$, some $\mathcal{D}^\delta(C_Q, C_D)$ with $(C_Q, C_D) \in \mathcal{K}$ contains a pair (u, v) . A consistent vector query instance additionally require that no proper subset $\mathcal{K}' \subsetneq \mathcal{K}$ satisfies both equalities. That is, after removing any one or more elements from $\mathcal{K}$, at least one query hyperedge no longer occurs in the remaining query hyperedge clusters, or at least one query vertex no longer occurs as the first vertex of any remaining $\mathcal{D}$ -match. We define

$$\text{FullCore}(D_i) \Leftrightarrow \exists \mathcal{K} \subseteq \bigcup_{j=1}^k \mathcal{R}_j : \mathcal{K} \cap \mathcal{R}_i \neq \emptyset,$$

$\mathcal{K}$ satisfies both equalities above,
no $\mathcal{K}' \subsetneq \mathcal{K}$ satisfies both.

Here, $\mathcal{K} \cap \mathcal{R}_i \neq \emptyset$ states that $\mathcal{K}$ contains at least one hyperedge

cluster pair (C_Q, C_D) satisfying $C_D.E \cap H_i.E \neq \emptyset$.

We define $\text{ConflictCert}(D_i)$ to be true if there exists a hyperedge cluster pair (C_Q, C_D) such that $(C_Q, C_D) \in \text{HC}^{\sigma, b}$ and $C_D.E \cap H_i.E \neq \emptyset$, and one of the following two conditions holds. For Condition (2)(a), $\mathcal{D}^\delta(C_Q, C_D) = \emptyset$, and the stored records contain $\langle (u, v), (C_Q, C_D), 2(a) \rangle$ for some $(u, v) \in \Pi^{(0)}$. For Condition (2)(b), $\mathcal{D}^\delta(C_Q, C_D) \neq \emptyset$, and the stored records contain $\langle (u, v), (C_Q, C_D), (u', v'), 2(b) \rangle$ for some $(u', v') \in \mathcal{D}^\delta(C_Q, C_D) \setminus \Pi$.

Intuitively, $\text{FullCore}(D_i)$ is true when a hyperedge cluster pair whose C_D contains a hyperedge from H_i belongs to a set $\mathcal{K}$ that includes every query hyperedge through its C_Q components and every query vertex through its $\mathcal{D}$ -matches. Since no proper subset of $\mathcal{K}$ has both properties, this pair of groups cannot be removed from $\mathcal{K}$; the remaining pairs of hyperedge groups may contain hyperedges from other documents retrieved. Moreover, $\text{ConflictCert}(D_i)$ is true when HC returns a hyperedge cluster pair whose C_D contains a hyperedge from H_i , but its $\mathcal{D}$ -match is empty or contains a vertex pair absent from Π , and HyperMatch records the corresponding deletion under Condition (2). At the document level, this means that HC selects a relation for comparison, but the entities that occur in the two relations cannot be paired in Π .

Based on these results, the category of D_i is defined as

$$\text{Category}(D_i) = \begin{cases} \text{consistent}, & \text{FullCore}(D_i); \\ \text{conflict}, & \neg \text{FullCore}(D_i) \\ & \wedge \text{ConflictCert}(D_i); \\ \text{useless}, & \neg \text{FullCore}(D_i) \\ & \wedge \neg \text{ConflictCert}(D_i) \\ & \wedge \Pi_i \neq \emptyset; \\ \text{irrelevant}, & \neg \text{FullCore}(D_i) \\ & \wedge \neg \text{ConflictCert}(D_i) \\ & \wedge \Pi_i = \emptyset. \end{cases} \quad (3)$$

Consistent. A document D_i is marked as *consistent* when $\text{FullCore}(D_i)$ is true. That is, there exists a set $\mathcal{K} \subseteq \bigcup_{j=1}^k \mathcal{R}_j$ that satisfies the two equalities above, has no proper subset satisfying both equalities, and satisfies $\mathcal{K} \cap \mathcal{R}_i \neq \emptyset$. The value of Δ_i does not change this category. Therefore, D_i remains *consistent* even if some pairs (u, v) with $v \in H_i.V$ belong to $\Pi^{(0)} \setminus \Pi$. The cluster pairs in $\mathcal{K}$ need not all belong to $\mathcal{R}_i$; some may belong to $\mathcal{R}_j$ for $j \neq i$.

Conflict. A document D_i is marked as *conflict* when $\text{FullCore}(D_i)$ is false and $\text{ConflictCert}(D_i)$ is true. By the definition above, there exists $(C_Q, C_D) \in \text{HC}^{\sigma, b}$ with $C_D.E \cap H_i.E \neq \emptyset$. For Condition (2)(a), $\mathcal{D}^\delta(C_Q, C_D) = \emptyset$ and the fixed-point records contain $\langle (u, v), (C_Q, C_D), 2(a) \rangle$ for some $(u, v) \in \Pi^{(0)}$. For Condition (2)(b), the records contain $\langle (u, v), (C_Q, C_D), (u', v'), 2(b) \rangle$ for some $(u', v') \in \mathcal{D}^\delta(C_Q, C_D) \setminus \Pi$. The condition $\Delta_i \neq \emptyset$ alone is insufficient, since it does not imply that either of these two records exists.

Useless. A document D_i is marked as *useless* when $\text{FullCore}(D_i)$ and $\text{ConflictCert}(D_i)$ are both false, while $\Pi_i \neq \emptyset$. The last condition means that there exists at least one pair $(u, v) \in \Pi$ with $v \in H_i.V$. It is possible that $\mathcal{R}_i \neq \emptyset$ also holds. However, this only establishes the existence of a cluster pair (C_Q, C_D) satisfying $C_D.E \cap H_i.E \neq \emptyset$ and $\emptyset \neq \mathcal{D}^\delta(C_Q, C_D) \subseteq \Pi$. It does not imply the existence of a set $\mathcal{K}$ that satisfies the two equalities above and $\mathcal{K} \cap \mathcal{R}_i \neq \emptyset$.

Irrelevant. Finally, a document D_i is marked as *irrelevant* when $\text{FullCore}(D_i)$ and $\text{ConflictCert}(D_i)$ are both false and $\Pi_i = \emptyset$. By the definition of Π_i , this equality means that there exists no pair $(u, v) \in \Pi$ whose second vertex v belongs to $H_i.V$. Thus, the distinction between *useless* and *irrelevant* is determined by whether Π_i is nonempty or empty.

These four cases are mutually exclusive and complete. If $\text{FullCore}(D_i)$ is true, D_i is marked as *consistent*, regardless of the values of $\text{ConflictCert}(D_i)$ and Δ_i . Pairs in Δ_i may still be reported, but they do not change the category. If $\text{FullCore}(D_i)$ is false, $\text{ConflictCert}(D_i)$ determines whether D_i is marked as *conflict*. When both predicates are false, $\Pi_i \neq \emptyset$ and $\Pi_i = \emptyset$ determine the *useless* and *irrelevant* categories, respectively.

The *useless*, *conflict*, and *irrelevant* categories are collectively treated as *inconsistent*. Here, *inconsistent* means exactly that $\text{FullCore}(D_i)$ is false. It does not mean that every useless or irrelevant document contradicts the query; only the *conflict* category requires one of the deletion records specified by $\text{ConflictCert}(D_i)$.

E.2 Computing the Marks and Reasons

For each document, we produce

$$\text{Mark}(D_i) = \langle \text{Category}(D_i), \text{Reason}(D_i) \rangle.$$

The category is determined by the rules above, while the reason is generated from the same vertex pairs, hyperedge cluster pairs, $\mathcal{D}$ -matches, predicates $\text{ConflictCert}(D_i)$ and $\text{FullCore}(D_i)$.

Consistent. The reason reports one set $\mathcal{K}$ for which $\text{FullCore}(D_i)$ is true and at least one cluster pair $(C_Q, C_D) \in \mathcal{K} \cap \mathcal{R}_i$. For this pair, it reports the hyperedges in $C_D.E \cap H_i.E$ and the vertex pairs in $\mathcal{D}^\delta(C_Q, C_D)$. If $\mathcal{K} \setminus \mathcal{R}_i \neq \emptyset$, the reason also reports the cluster pairs in this difference, together with their query hyperedges and $\mathcal{D}$ -matches. These reported sets show how $\mathcal{K}$ satisfies the two equalities in the definition of $\text{FullCore}(D_i)$.

Useless. Since $\Pi_i \neq \emptyset$, the reason reports at least one pair $(u, v) \in \Pi_i$, where $v \in H_i.V$. If $\mathcal{R}_i \neq \emptyset$, it also reports at least one $(C_Q, C_D) \in \mathcal{R}_i$ and its $\mathcal{D}^\delta(C_Q, C_D)$. Finally, it states that $\text{FullCore}(D_i)$ is false: there exists no set $\mathcal{K}$ that satisfies the two equalities and the proper-subset condition above while also satisfying $\mathcal{K} \cap \mathcal{R}_i \neq \emptyset$.

Conflict. The reason reports one stored deletion record that makes $\text{ConflictCert}(D_i)$ true. For Condition (2)(a), it reports $\langle (u, v), (C_Q, C_D), 2(a) \rangle$ and states that $\mathcal{D}^\delta(C_Q, C_D) = \emptyset$. It also reports the values of u and v . For Condition (2)(b), it reports $\langle (u, v), (C_Q, C_D), (u', v'), 2(b) \rangle$, where $(u', v') \in \mathcal{D}^\delta(C_Q, C_D) \setminus \Pi$. It reports the values of u' and v' when $h_v(u', v') = \text{false}$. If later deletions are also reported, each later record explicitly identifies its deleted pair and its missing pair; the missing-pair field equals a pair deleted in an earlier reported record.

Irrelevant. The reason states that $\Pi_i = \emptyset$, which means that there is no pair $(u, v) \in \Pi$ with $v \in H_i.V$. It also states that $\text{FullCore}(D_i)$ is false because no set $\mathcal{K}$ satisfying the conditions above has $\mathcal{K} \cap \mathcal{R}_i \neq \emptyset$. Finally, $\text{ConflictCert}(D_i)$ is false because no cluster pair and stored deletion record satisfy either of its two defining conditions.

These reasons are generated using fixed templates rather than an LLM. They only verbalize the vertex values and relations from the query and retrieved document, together with the hyper sim-

ulation computed by HyperMatch, without introducing external information or unsupported claims.

The following block specifies the fixed templates.

Reason Templates	
(1) Purpose	Generate a brief natural-language explanation for the suggested category of a retrieved document. The explanation should identify the relevant statement, missing information, or direct difference between the query and the document without exposing any internal computation.
(2) Template fields	– {CURRENT_FACT}: a relation stated in the current document. – {OTHER_FACT} or {OTHER_FACTS}: relations stated in other retrieved documents that are needed together with the current document. – {OVERLAPPING_CONTENT}: people, objects, events, or other values that occur in both the query and the document. – {RELATED_FACT}: a fact in the document that concerns some query values but does not provide all the information asked for. – {MISSING_FACT}: a relation or condition expressed in the query but not stated in the document. – {EXPECTED_FACT} and {EXPECTED_VALUE}: the relation or value specified by the query. – {OBSERVED_FACT} and {OBSERVED_VALUE}: the relation or value stated by the document. – {QUERY_CONDITION}: the relevant condition expressed in the query. – {QUERY_TARGET}: the person, object, location, organization, event, or other information requested by the query. – {DOCUMENT_TOPIC} and {QUERY_FOCUS}: the main content of the document and the subject of the query, respectively.
(3) Templates for consistent documents	– Single-document answer. <i>“This passage states that {CURRENT_FACT}. This directly provides the information requested about {QUERY_TARGET}.”</i> – Two-document answer. <i>“This passage states that {CURRENT_FACT}. Another retrieved passage states that {OTHER_FACT}. Together, these statements provide the information requested about {QUERY_TARGET}.”</i> – Answer using more than two documents. <i>“This passage states that {CURRENT_FACT}. The other retrieved passages state {OTHER_FACTS}. Together, these statements provide the information requested about {QUERY_TARGET}.”</i>
(4) Templates for useless documents	– Mention-only overlap. <i>“This passage mentions {OVERLAPPING_CONTENT}, but it does not state {MISSING_FACT}. It therefore does not help determine {QUERY_TARGET}.”</i> – Incomplete relation. <i>“This passage states that {RELATED_FACT}, but it does not state {MISSING_FACT}. It therefore does not establish {QUERY_TARGET}.”</i> – Different but related fact. <i>“This passage describes {OBSERVED_FACT}. The query instead asks about {EXPECTED_FACT}, which this passage does not state.”</i>
(5) Templates for conflict documents	– Different entity or value. <i>“The query asks about {EXPECTED_FACT}, whereas this passage states {OBSERVED_FACT}. The information in this passage therefore does not satisfy {QUERY_CONDITION}.”</i> – Different relation. <i>“This passage mentions {OVERLAPPING_CONTENT}, but it states {OBSERVED_FACT} rather than {EXPECTED_FACT}. It therefore does not satisfy {QUERY_CONDITION}.”</i>

(6) Template for irrelevant documents

“This passage discusses {DOCUMENT_TOPIC}, but it does not mention {QUERY_FOCUS} or state any information needed to answer the query.”

(7) Output format

Reason: {GENERATED_REASON}

E.3 Using the Marks in Downstream Tasks

We preserve the original text and retrieval order of every document and prepend the category and reason:

Context ID: { i }
Category: {Category(D_i)}
Reason: {Reason(D_i)}
Text: { D_i }

The category provides auxiliary hints for downstream tasks, while the reason explains the result at the entity and relation levels. For RAG, we use the following prompt template.

Answer Generation Prompt	
(1) Role and task	You are an evidence-grounded question-answering assistant. Given a query and a set of retrieved contexts, answer the query using evidence from the original context texts. Each context is accompanied by a suggested category and reason. Treat them only as auxiliary hints for locating potentially useful or problematic information. Assess the original texts independently and disregard a suggestion whenever it is not supported by the textual evidence.
(2) Interpretation of category suggestions	– A <i>consistent</i> category suggests that the context may contain information relevant to and compatible with the query. – A <i>useless</i> category suggests that the context may mention query-related content without providing the information needed to answer the query. – A <i>conflict</i> category warns that some information in the context may fail to satisfy the relations or constraints expressed in the query, or may conflict with other relevant information. – An <i>irrelevant</i> category suggests that no useful information for the query has been identified in the context. These categories are query-specific suggestions and may be imperfect. They do not determine whether a context can be used. The original context text remains the primary source of factual evidence.
(3) Input data	– Query: The question to be answered. – Retrieved contexts: Each retrieved context contains the following fields: * Context: Its identifier in the retrieval results. * Category: A suggested assessment of the context. * Reason: A brief explanation supporting the suggestion. * Text: The original retrieved text.
(4) Guidance for using the suggestions	– Use the category and reason to guide your inspection of the retrieved contexts, but verify every claim used in the answer against the corresponding Text field. – Give initial attention to contexts categorized as <i>consistent</i>, but do not accept their content solely because of the suggested category. – Do not automatically discard contexts categorized as <i>useless</i>, <i>conflict</i>, or <i>irrelevant</i>. You may use any context whose original text provides clear and relevant evidence.

- If a category or reason is inconsistent with the original text, disregard the suggestion and follow the textual evidence.
- Use the **Reason** field only to help locate relevant or potentially problematic content. Do not treat the reason itself as factual evidence.
- Multiple contexts may be combined when their original texts jointly support the information and connections required by the query. Do not introduce unsupported connections or external knowledge.
- If the retrieved texts do not provide sufficient evidence, return exactly: Insufficient evidence.

(5) **Input content template**

- **Query:** {QUERY}
- **Retrieved contexts:**
[Context {ID}]
Category: {CATEGORY}
Reason: {REASON}
Text: {ORIGINAL CONTEXT TEXT}

(6) **Output specification**

- Return a concise answer grounded in the original retrieved texts. You may follow or override any suggested category based on your assessment of the textual evidence. Do not discuss the category suggestions unless explicitly requested.
- **Desired output format:**
Answer: {ANSWER}

E.4 Complexity Analysis

Document marking reuses the intermediate results of Algorithm 3. For each document D_i , a breadth-first traversal over the query hypergraph and the surviving hyperedge cluster pairs computes a set $\mathcal{K}_i$ satisfying $\text{FullCore}(D_i)$, if such a set exists. A subsequent pass uses the coverage counts of the query hyperedges and query vertices to remove any hyperedge cluster pair whose removal preserves both coverage conditions, thereby making $\mathcal{K}_i$ inclusion-minimal. When Condition (2)(a) removes a vertex pair (u, v) , the computation retains the corresponding hyperedge cluster pair (C_Q, C_D) . When Condition (2)(b) removes (u, v) , it additionally retains the missing pair $(u', v') \in \mathcal{D}^\delta(C_Q, C_D) \setminus \Pi$. These quantities are sufficient to evaluate $\text{FullCore}(D_i)$ and $\text{ConflictCert}(D_i)$ and to instantiate the fixed reason templates. Document marking therefore does not repeat the fixed-point computation for each document.

Let $\Pi^{(0)}$ denote the initial working relation, $\mathcal{A}_{\text{cluster}}$ the set of candidate hyperedge cluster pairs, and

$$N_{\mathcal{D}} = \sum_{(C_Q, C_D) \in \mathcal{A}_{\text{cluster}}} |\mathcal{D}^\delta(C_Q, C_D)|.$$

To account for the traversal used to compute $\mathcal{K}_i$, let

$$N_{\text{cov}} = \sum_{(C_Q, C_D) \in \mathcal{A}_{\text{cluster}}} (|C_Q.E| + |\{u \in V_Q \mid (u, _) \in \mathcal{D}^\delta(C_Q, C_D)\}|).$$

Since a hyperedge cluster contains at most m hyperedges and each query hyperedge contains at most Δ_E vertices, each candidate hyperedge cluster pair contributes at most $m(1 + \Delta_E)$ entries to N_{cov} . Moreover, Theorem 2 gives $|\mathcal{A}_{\text{cluster}}| = O(|E_Q|\Delta_E^2 b^2)$. It follows that

$$N_{\text{cov}} \leq m(1 + \Delta_E)|\mathcal{A}_{\text{cluster}}| = O(|E_Q|b^2 m \Delta_E^2 (1 + \Delta_E)).$$

Because b , m , and Δ_E are treated as constants, $N_{\text{cov}} = O(|E_Q|)$. Thus, one breadth-first traversal followed by the coverage-count

Table 11: Runtime of Document Marking.

Dataset	HotpotQA			MuSiQue		
	Total	Marking	Share	Total	Marking	Share
Runtime (ms)	925	8.339	0.902%	1,247	6.762	0.542%

pass takes $O(|V_Q| + |E_Q|)$ time, and computing $\mathcal{K}_i$ for all k documents takes $O(k(|V_Q| + |E_Q|))$ time. After the sets $\mathcal{K}_i$ have been computed, all categories and reasons can be produced by scanning Π , $\Pi^{(0)}$, the candidate hyperedge cluster pairs, their $\mathcal{D}$ -matches, and the removed and missing vertex pairs once. This step takes

$$O(k + |\Pi^{(0)}| + |\mathcal{A}_{\text{cluster}}| + N_{\mathcal{D}})$$

time. Each $\mathcal{D}$ -match contains at most $m\Delta_E$ vertex pairs. Therefore,

$$N_{\mathcal{D}} \leq m\Delta_E |\mathcal{A}_{\text{cluster}}| = O(|E_Q|b^2 m \Delta_E^3) = O(|E_Q|),$$

where the last equality again treats b , m , and Δ_E as constants.

According to Theorems 2 and 3, $|\Pi^{(0)}| \leq |V_Q||V|$, $|\mathcal{A}_{\text{cluster}}| = O(|E_Q|)$, and $N_{\mathcal{D}} = O(|E_Q|)$. Furthermore, $k \leq |V|$ because every retrieved document contributes at least one vertex to the aggregated data hypergraph. Hence,

$$k(|V_Q| + |E_Q|) = O(|V_Q||V| + |E_Q||V|).$$

Consequently, the document-marking step preserves the overall time complexity

$$O(|V_Q||V| + |E_Q|(|V| + |E|)).$$

It requires no additional invocation of the NLI or embedding models. Retaining one category and one reason-template selection for each document requires $O(k)$ additional space. The removed pairs, their associated hyperedge cluster pairs, and the missing pairs used by Condition (2) require at most $O(|\Pi^{(0)}|)$ additional space. Materializing the natural-language reasons and constructing the downstream prompt take time and space linear in the total length of the generated reasons and retrieved document texts.

The runtime reported in Exp-3 already includes document marking. As shown in Table 11, document marking takes only 8.339 ms per HotpotQA query and 6.762 ms per MuSiQue query. These values account for 0.902% and 0.542% of the corresponding total runtimes, respectively. Thus, the measured cost of producing the document categories and their template-generated reasons is below 1% on both datasets.

F Parameter Configuration

The parameters can be divided into two categories: retrieval-stage parameters and HyperMatch parameters. The former is determined by the external retrieval module and does not require tuning within our framework, while the latter controls the matching behavior of HyperMatch and follows a stable default configuration with only limited local adjustment when necessary.

For the retrieval stage, the threshold θ is not a free parameter of HyperMatch. It only determines which retrieved documents are forwarded for matching. Therefore, we directly adopt the cutoff defined by the deployed retriever. In a top- k retrieval setting, this is equivalent to retaining all returned documents, i.e., setting θ below the minimum similarity score among retrieved items. Since similarity scores are not comparable across different embedding models, we do not specify a universal numerical value for θ .

For the HyperMatch, we further organize the parameters into three groups according to their roles: filtering strength, matching granularity, and computational budget. First, the filtering parameters (σ, δ) control the strictness of structural and semantic alignment. We recommend $(\sigma, \delta) = (0.70, 0.75)$ as the default setting, which provides a balanced trade-off between precision and recall across tasks. Second, the parameter b determines the number of candidate data vertices considered for each query vertex, thereby controlling the matching granularity and efficiency. We set $b = 8$ as the default configuration, and observe that values in the range 7, 8, 9 yield stable performance, where smaller values improve efficiency while larger values slightly increase coverage. Finally, the remaining constants $\alpha = 2.36$ and $m = 5$ are fixed across all experiments and are held fixed and are not tuned per dataset.

When validation data are available, a lightweight one-at-a-time adjustment can be performed around the default configuration, *i.e.*, $\sigma \in \{0.6, 0.7, 0.8\}$, $\delta \in \{0.65, 0.75, 0.85\}$, and $b \in \{7, 8, 9\}$. This local search is optional and is mainly intended for deployment scenarios with strict efficiency or accuracy preferences.

G Additional Case Studies

We provide four cases to show how the results of hyper simulation determine the category and reason of each retrieved document. For readability, only the excerpts used in the computation are shown. In Case m , H_i denotes the hypergraph of $D_i^{(m)}$, and Π_i and $\mathcal{R}_i$ (different from $\mathcal{R}_1$ and $\mathcal{R}_2$ in HypergraphBuild and HCCalc) are defined from that H_i as in Appendix E. Thus, these symbols are evaluated separately in each case.

The four categories are determined by

D_i is *consistent* $\iff \text{FullCore}(D_i)$,

D_i is *conflict* $\iff \neg \text{FullCore}(D_i) \wedge \text{ConflictCert}(D_i)$,

D_i is *useless* $\iff \neg \text{FullCore}(D_i) \wedge \neg \text{ConflictCert}(D_i) \wedge \Pi_i \neq \emptyset$,

D_i is *irrelevant* $\iff \neg \text{FullCore}(D_i) \wedge \neg \text{ConflictCert}(D_i) \wedge \Pi_i = \emptyset$.

When $\text{FullCore}(D_i)$ is true, we display a set $\mathcal{K} \subseteq \bigcup_j \mathcal{R}_j$ for which $\bigcup_{(C_Q, C_D) \in \mathcal{K}} C_Q.E = H_Q.E$ and $\{u \in H_Q.V \mid \exists (C_Q, C_D) \in \mathcal{K}, \exists v \in C_D.V : (u, v) \in \mathcal{D}^\delta(C_Q, C_D)\} = H_Q.V$. We then state why every proper subset of $\mathcal{K}$ fails at least one equality. When $\text{ConflictCert}(D_i)$ is true, we display a stored Condition (2)(a) or Condition (2)(b) record that satisfies its definition. Thus, $\Delta_i \neq \emptyset$ does not by itself imply $\text{ConflictCert}(D_i)$, and $\mathcal{R}_i \neq \emptyset$ does not by itself imply $\text{FullCore}(D_i)$.

$Q^{(1)}$: "Which American football running back for the New Orleans Saints of the NFL did Samuel C. Morrison interview for *The Source*?"

$D_1^{(1)}$: Samuel C. Morrison's writing career began as a contributing writer for The Source Magazine, interviewing celebrities by the likes of ..., Adrian Peterson, and more.

$D_2^{(1)}$: Adrian Lewis Peterson is an American football running back for the New Orleans Saints of the National Football League (NFL).

$D_3^{(1)}$: Errick Lynne "Ricky" Williams Jr. ... is a retired American football running back who played twelve seasons in the National Football League (NFL) ... Williams was drafted by the New Orleans Saints ... and spent three seasons with the team before he was traded to the Miami Dolphins in 2002. ... Williams re-joined the Dolphins in 2007 ...

Case 1: We first expand the HotpotQA example in Exp-6. The query

asks which running back for the New Orleans Saints of the NFL was interviewed by Samuel C. Morrison for *The Source*. The answer is *Adrian Lewis Peterson*.

The query asks for one player who satisfies two conditions. First, "Samuel C. Morrison interviewed the player for *The Source*"; we denote the corresponding query hyperedge cluster by C_I . Second, the "same player was an American football running back for the New Orleans Saints in the NFL"; we denote the corresponding query hyperedge cluster by C_R . Let u_p, u_m, u_s, u_t , and u_ℓ denote the query vertices for the "requested player", "Samuel C. Morrison", "The Source Magazine", "the New Orleans Saints", and "the NFL", respectively. In C_I , u_m, u_p , and u_s represent the "interviewer", "interviewee", and "publication". In C_R , u_p, u_t , and u_ℓ represent the "player", "team", and "league". Thus, u_p occurs in both hyperedge clusters: "the player interviewed by Morrison must be the same player who was a running back for the Saints in the NFL".

For $D_1^{(1)}$, HC returns (C_I, C_I') , where $C_I'.E \cap H_1.E \neq \emptyset$ and

$$\{(u_p, v_{P,1}), (u_m, v_M), (u_s, v_S)\} \subseteq \mathcal{D}^\delta(C_I, C_I') \subseteq \Pi.$$

The values of $v_{P,1}$, v_M , and v_S are "Adrian Peterson", "Samuel C. Morrison", and "The Source Magazine". Hence, $(C_I, C_I') \in \mathcal{R}_1$. For $D_2^{(1)}$, HC returns (C_R, C_R') , where $C_R'.E \cap H_2.E \neq \emptyset$ and

$$\{(u_p, v_{P,2}), (u_t, v_N), (u_\ell, v_L)\} \subseteq \mathcal{D}^\delta(C_R, C_R') \subseteq \Pi.$$

The values of $v_{P,2}$, v_N , and v_L are "Adrian Lewis Peterson", "the New Orleans Saints", and "the NFL". Hence, $(C_R, C_R') \in \mathcal{R}_2$.

Set $\mathcal{K}^{(1)} = \{(C_I, C_I'), (C_R, C_R')\}$. The two displayed equalities hold for $\mathcal{K}^{(1)}$. If (C_I, C_I') is removed, the first equality lacks the interview hyperedge and the second lacks u_m and u_s . If (C_R, C_R') is removed, the first equality lacks the running-back-team-league hyperedge and the second lacks u_t and u_ℓ . Thus, no proper subset of $\mathcal{K}^{(1)}$ satisfies both equalities.

(1) $D_1^{(1)}$: *Consistent*. Since $(C_I, C_I') \in \mathcal{K}^{(1)} \cap \mathcal{R}_1$, $\text{FullCore}(D_1^{(1)})$ is true.

[Reason]. This passage states that Samuel C. Morrison for The Source Magazine interviewing of Adrian Peterson. Another retrieved passage states that Adrian Lewis Peterson is an running back for the New Orleans Saints of the National Football League (NFL). Together, these statements provide the information requested about Adrian Lewis Peterson as the player asked about in the query. $\square$

(2) $D_2^{(1)}$: *Consistent*. Since $(C_R, C_R') \in \mathcal{K}^{(1)} \cap \mathcal{R}_2$, $\text{FullCore}(D_2^{(1)})$ is true.

[Reason]. This passage states that Adrian Lewis Peterson is an running back for the New Orleans Saints of the National Football League (NFL). Another retrieved passage states that Samuel C. Morrison for The Source Magazine interviewing of Adrian Peterson. Together, these statements provide the information requested about Adrian Lewis Peterson as the player asked about in the query. $\square$

The computation also removes other vertex pairs. For example, it removes pairs between the "Saints" query vertex and an "NFL" document vertex in $D_2^{(1)}$, and between the "Morrison" query vertex and an "Adrian Peterson" document vertex in $D_1^{(1)}$. Such pairs are elements of Δ_2 or Δ_1 , respectively. They do not change either category because $\text{FullCore}(D_1^{(1)})$ and $\text{FullCore}(D_2^{(1)})$ are true.

(3) $D_3^{(1)}$: *Conflict*. Let v_W, v'_N, v'_L , and v_D be the vertices of H_3 whose

values are “Ricky Williams”, the “New Orleans Saints”, the “NFL”, and the “Miami Dolphins”. The function HC returns (C_R, C'_N) and (C_R, C'_D) , with $C'_N.E \cap H_3.E \neq \emptyset$ and $C'_D.E \cap H_3.E \neq \emptyset$. Their $\mathcal{D}$ -matches contain

$$\begin{aligned} \{(u_p, v_W), (u_t, v'_N), (u_t, v'_L)\} &\subseteq \mathcal{D}^\delta(C_R, C'_N), \\ \{(u_p, v_W), (u_t, v_D), (u_t, v'_L)\} &\subseteq \mathcal{D}^\delta(C_R, C'_D). \end{aligned}$$

The computation gives $h_u(u_t, v_D) = \text{false}$, $(u_t, v_D) \in \mathcal{D}^\delta(C_R, C'_D) \setminus \Pi$. The stored Condition (2)(b) records include

$$\begin{aligned} \langle (u_p, v_W), (C_R, C'_D), (u_t, v_D), 2(b) \rangle, \\ \langle (u_t, v'_L), (C_R, C'_D), (u_t, v_D), 2(b) \rangle. \end{aligned}$$

They state that (u_p, v_W) and (u_t, v'_L) are removed because (u_t, v_D) is not in Π . In the next iteration, a stored record for (C_R, C'_N) is

$$\langle (u_t, v'_N), (C_R, C'_N), (u_p, v_W), 2(b) \rangle.$$

Thus, $\mathcal{D}^\delta(C_R, C'_N) \not\subseteq \Pi$, $\mathcal{D}^\delta(C_R, C'_D) \not\subseteq \Pi$, and $\mathcal{R}_3 = \emptyset$. Therefore, $\text{FullCore}(D_3^{(1)})$ is false. The nonempty $\mathcal{D}$ -match for (C_R, C'_D) , the missing pair (u_t, v_D) , and the displayed Condition (2)(b) records make $\text{ConflictCert}(D_3^{(1)})$ true. Hence, $D_3^{(1)}$ is marked as *conflict*.

[Reason]. The query asks about a the New Orleans Saints, whereas this passage states the Miami Dolphins. The information in this passage therefore does not satisfy running back. $\square$

This case distinguishes deletion of a vertex pair from the two predicates in the category formula. For the first two documents, FullCore is true despite other deleted pairs. For the third document, FullCore is false and ConflictCert is true.

Case 2: This case asks which company satisfies four conditions: it spent \$26.3 billion in 2021 to become the default search engine; it could address the Android distribution and payment problem; it could win a legal case depending on the definition of the app-store market; and it faces a publisher antitrust suit. The answer is *Google*.

$Q^{(2)}$: “Which company spent \$26.3 billion in 2021 to become the default search engine, could address the Android distribution and payment problem, could win a legal case depending on how the app-store market is defined, and faces a publisher antitrust suit?”

$D_1^{(2)}$: Google spent \$26.3 billion to make itself the default search engine across platforms in 2021.

$D_2^{(2)}$: For all the other benefits they get from Android, Google could solve this problem today if they wanted to.

$D_3^{(2)}$: If the relevant market is phones and app stores rather than specifically Android ones, Google is on the road to victory.

$D_4^{(2)}$: News publisher files class action antitrust suit against Google, citing AI’s harms to their bottom line. The case ... argues that Google “siphons off” news publishers’ content, their readers and ad revenue through anti-competitive means.

$D_5^{(2)}$: ... Store (iOS) or Google Play Store. Some apps are free, and others have a price, with the profit being split between the application’s creator and the distribution platform.

$D_6^{(2)}$: ... Microsoft paid Verizon Wireless \$550 million USD to use Bing as the default search provider on Verizon’s BlackBerry ...

Let $C_1, \dots, C_4$ be the four query clusters. For $1 \leq j \leq 4$, let (C_j, C'_j) be the pair returned by HC for $D_j^{(2)}$. The computed results satisfy $C'_j.E \cap H_j.E \neq \emptyset$, $\emptyset \neq \mathcal{D}^\delta(C_j, C'_j) \subseteq \Pi$. Therefore, $(C_j, C'_j) \in \mathcal{R}_j$ ($1 \leq j \leq 4$).

The data vertices in these four $\mathcal{D}$ -matches have, respectively, the following values:

- (C_1, C'_1) : Google; \$26.3 billion; 2021; default search,
- (C_2, C'_2) : Google; Android distribution and payment problem,
- (C_3, C'_3) : Google; phones-and-app-stores market condition,
- (C_4, C'_4) : Google; antitrust case; publisher-revenue statement.

Set $\mathcal{K}^{(2)} = \{(C_1, C'_1), (C_2, C'_2), (C_3, C'_3), (C_4, C'_4)\}$. The two displayed equalities hold for $\mathcal{K}^{(2)}$. Each C_j contains a query hyper-edge that is absent from the other three query clusters. Therefore, removing any element of $\mathcal{K}^{(2)}$ makes the first equality false. No proper subset satisfies both equalities.

(1)–(4) $D_1^{(2)} - D_4^{(2)}$: *Consistent*. For every $j \in \{1, 2, 3, 4\}$, $(C_j, C'_j) \in \mathcal{K}^{(2)} \cap \mathcal{R}_j$. Hence, $\text{FullCore}(D_j^{(2)})$ is true for $1 \leq j \leq 4$, and all four documents are marked as *consistent*.

[Reason for $D_1^{(2)}$]. This passage states that Google spent \$26.3 billion in 2021 to make itself the default search engine. The other retrieved passages state the other benefits they get from Android, Google could solve; the relevant market is phones and app stores rather than specifically Android ones; News publisher files class action antitrust suit against Google. Together, these statements provide the information requested about the company. $\square$

Some pairs in $\Pi^{(0)} \setminus \Pi$ have second vertices in $H_1.V$ or $H_4.V$. These pairs belong to Δ_1 or Δ_4 . They do not change the categories because $\text{FullCore}(D_1^{(2)})$ and $\text{FullCore}(D_4^{(2)})$ are true.

(5) $D_5^{(2)}$: *Useless*. The computed result satisfies $\Pi_5 \neq \emptyset$ and $\mathcal{R}_5 = \emptyset$. The second vertices in the displayed pairs from Π_5 have values such as *app stores* and *distribution platforms*. Since $\mathcal{R}_5 = \emptyset$, no $\mathcal{K}$ satisfying both equalities and the proper subset condition can intersect $\mathcal{R}_5$. Thus, $\text{FullCore}(D_5^{(2)})$ is false. The stored records contain neither a Condition (2)(a) record nor a Condition (2)(b) record that satisfies the definition of $\text{ConflictCert}(D_5^{(2)})$. Therefore, $\neg \text{FullCore}(D_5^{(2)})$, $\neg \text{ConflictCert}(D_5^{(2)})$, $\Pi_5 \neq \emptyset$, and $D_5^{(2)}$ is marked as *useless*.

[Reason]. This passage mentions Google Play Store, app stores, application, and distribution platforms, but it does not state about a company. It therefore does not help identify that company. $\square$

(6) $D_6^{(2)}$: *Useless*. For $D_6^{(2)}$, $\Pi_6 \neq \emptyset$. A pair $(C_1, C'_6) \in \mathcal{R}_6$ has a $\mathcal{D}$ -match containing vertices for “Microsoft”, “Bing”, and “Verizon”, but no pair whose first component is the query vertex for “\$26.3 billion” or the query vertex for “2021”. Among the pairs returned in this case, (C_1, C'_1) is the only element of $\bigcup_i \mathcal{R}_i$ whose $\mathcal{D}$ -match contains first components for both “\$26.3 billion” and “2021”. Therefore, every set satisfying the second equality must contain (C_1, C'_1) . Once (C_1, C'_1) is present, removing (C_1, C'_6) leaves both equalities true. Hence, no set satisfying the proper-subset condition can intersect $\mathcal{R}_6$, and $\text{FullCore}(D_6^{(2)})$ is false. The stored records contain neither a Condition (2)(a) record nor a Condition (2)(b) record that satisfies the definition of $\text{ConflictCert}(D_6^{(2)})$. Thus, $\text{ConflictCert}(D_6^{(2)})$ is false. Together with $\Pi_6 \neq \emptyset$, these facts make $D_6^{(2)}$ *useless*.

[Reason]. This passage describes Microsoft paying Verizon \$550 million to use Bing as a default search provider. The query instead asks about a company that spent \$26.3 billion in 2021, which this passage not state. $\square$

For $D_5^{(2)}$ and $D_6^{(2)}$, Π_i is nonempty while $\text{FullCore}(D_i)$ and $\text{ConflictCert}(D_i)$ are both false. The category formula therefore marks both documents as *useless*.

Case 3: This MuSiQue case asks what the form of Latin associated with the surname Sylvester and used in the era of Charlemagne was later called. The answer is *Medieval Latin*.

$Q^{(3)}$: “What was the form of Latin associated with the surname Sylvester and used in the era of Charlemagne later called?”

$D_1^{(3)}$: Sylvester is a name derived from the Latin adjective *silvestris*; spellings with Sylv- in place of Silv- date from after the Classical period.

$D_2^{(3)}$: By the reign of Charlemagne, the language had so diverged from the classical that it was later called Medieval Latin.

$D_3^{(3)}$: The imperial period of Rome lasted approximately 1,500 years and included emperors from Augustus through later dynasties.

The query asks for the later name of a form of “Latin” described by two parts of the question: “it is associated with the surname Sylvester” and “was used in the era of Charlemagne.” We denote the corresponding query hyperedge cluster by C . Let u_n , u_e , and u_ℓ denote the query vertices for the “surname Sylvester”, the “era of Charlemagne”, and the “requested later language name”, respectively. In C , u_n corresponds to “the surname Sylvester,” u_e corresponds to “the era of Charlemagne,” and u_ℓ is the answer requested by “later called.” Thus, C expresses that the answer must be the later name of the same form of “Latin” that satisfies both descriptions. In the displayed documents, $D_1^{(3)}$ provides the association with “Sylvester”, while $D_2^{(3)}$ provides the “Charlemagne-era” description and the later name.

For $D_1^{(3)}$, HC returns (C, C'_1) , where $C'_1.E \cap H_1.E \neq \emptyset$, $\mathcal{D}^\delta(C, C'_1) = \{(u_n, v_n^{(1)})\} \subseteq \Pi$. The value of $v_n^{(1)}$ is the name “Sylvester” as connected to its “Latin” origin. Hence, $(C, C'_1) \in \mathcal{R}_1$.

For $D_2^{(3)}$, HC returns (C, C'_2) , where $C'_2.E \cap H_2.E \neq \emptyset$, $\mathcal{D}^\delta(C, C'_2) = \{(u_e, v_e^{(2)}), (u_\ell, v_\ell^{(2)})\} \subseteq \Pi$. The values of $v_e^{(2)}$ and $v_\ell^{(2)}$ are the “reign of Charlemagne” and “Medieval Latin”. Hence, $(C, C'_2) \in \mathcal{R}_2$.

Set $\mathcal{K}^{(3)} = \{(C, C'_1), (C, C'_2)\}$. Because $C.E = H_Q.E$, the first equality holds. The first components of the pairs in the two displayed $\mathcal{D}$ -matches are $\{u_n, u_e, u_\ell\} = H_Q.V$, so the second equality also holds. Removing (C, C'_1) removes u_n from the left-hand side of the second equality, whereas removing (C, C'_2) removes u_e and u_ℓ . Hence, no proper subset of $\mathcal{K}^{(3)}$ satisfies both equalities. Since $\mathcal{K}^{(3)} \cap \mathcal{R}_1 \neq \emptyset$, $\mathcal{K}^{(3)} \cap \mathcal{R}_2 \neq \emptyset$, $\text{FullCore}(D_1^{(3)})$ and $\text{FullCore}(D_2^{(3)})$ are true.

(1) $D_1^{(3)}$: *Consistent*. Pairs whose first component is u_ℓ and whose second components have the values “name”, “the Latin adjective *silvestris*”, or “*Sylvester*” may belong to Δ_1 . Their presence in Δ_1 does not change the category because $\text{FullCore}(D_1^{(3)})$ is true.

[Reason]. This passage states that the name Sylvester is derived from the Latin adjective *silvestris*. Another retrieved passage states that the Latin used during the reign of Charlemagne was later called Medieval Latin. Together, these statements provide the information requested about Medieval Latin as the answer. $\square$

(2) $D_2^{(3)}$: *Consistent*. Pairs whose first component is u_ℓ and whose

second components have the values “reign”, “language”, or “the reign of Charlemagne” may belong to Δ_2 . Their presence in Δ_2 does not change the category because $\text{FullCore}(D_2^{(3)})$ is true.

[Reason]. This passage states that the Latin used during the reign of Charlemagne was later called Medieval Latin. Another retrieved passage states that the name Sylvester is derived from the Latin adjective *silvestris*. Together, these statements provide the information requested about Medieval Latin as the answer.

(3) $D_3^{(3)}$: *Useless*. For this computation, let v_R be a document vertex whose value is the “imperial period of Rome”. The final result contains $(u_e, v_R) \in \Pi_3$, so $\Pi_3 \neq \emptyset$. It also gives $\mathcal{R}_3 = \emptyset$. Hence, $\text{FullCore}(D_3^{(3)})$ is false. The stored records contain neither a Condition (2)(a) record nor a Condition (2)(b) record satisfying the definition of $\text{ConflictCert}(D_3^{(3)})$. Therefore, $\neg \text{FullCore}(D_3^{(3)})$, $\neg \text{ConflictCert}(D_3^{(3)})$, $\Pi_3 \neq \emptyset$, and $D_3^{(3)}$ is marked as *useless*.

[Reason]. This passage discusses the imperial period of Rome and several Roman emperors. The query instead asks about Sylvester, Charlemagne, Latin, which this passage does not state. $\square$

Case 4: The final HotpotQA case asks which organization first used the Lockheed P-80 Shooting Star operationally as a jet fighter. The answer is *the United States Army Air Forces (USAAF)*.

$Q^{(4)}$: “Which organization first used the Lockheed P-80 Shooting Star operationally as a jet fighter?”

$D_1^{(4)}$: The Lockheed P-80 Shooting Star was the first jet fighter used operationally by the United States Army Air Forces (USAAF).

$D_2^{(4)}$: The USAAF passed on the proposed L-133; that effort accelerated development of the USAAF’s first successful operational jet fighter, the P-80 Shooting Star.

$D_3^{(4)}$: During the Korean War, the Americans rushed the North American F-86 Sabre into battle against the MiGs after American F-80 pilots encountered them.

The query asks for the organization in the statement that “the Lockheed P-80 Shooting Star was the first jet fighter it used operationally”. We denote the corresponding query hyperedge cluster by C_Q . Let u_o , u_a , and u_s denote the query vertices for the requested organization, the “Lockheed P-80 Shooting Star”, and its status as the “first jet fighter used operationally”, respectively. In C_Q , u_o corresponds to “Which organization,” u_a corresponds to “the Lockheed P-80 Shooting Star,” and u_s corresponds to “first used ... operationally as a jet fighter.” Thus, C_Q expresses that the “requested organization”, the “specified aircraft”, and “this operational status” must occur in the same relation. Among the displayed documents, $D_1^{(4)}$ directly states this relation, $D_2^{(4)}$ states a related relation between the “USAAF” and the “P-80”, and $D_3^{(4)}$ describes the operational use of a different aircraft, the “F-86 Sabre”.

(1) $D_1^{(4)}$: *Consistent*. Let the values of $v_o^{(1)}$, $v_a^{(1)}$, and $v_s^{(1)}$ be “USAAF”, the “P-80 Shooting Star”, and “first jet fighter used operationally”. For the cluster pair (C_Q, C_1) returned by HC,

$$\mathcal{D}^\delta(C_Q, C_1) = \{(u_o, v_o^{(1)}), (u_a, v_a^{(1)}), (u_s, v_s^{(1)})\} \subseteq \Pi.$$

Since $C_1.E \cap H_1.E \neq \emptyset$, $(C_Q, C_1) \in \mathcal{R}_1$. Set $\mathcal{K}^{(4)} = \{(C_Q, C_1)\}$. The query hyperedge is the left-hand side of the first equality, and $\{u_o, u_a, u_s\} = H_Q.V$ is the left-hand side of the second equality.

The only proper subset of $\mathcal{K}^{(4)}$ is the empty set, which satisfies neither equality. Hence, $\text{FullCore}(D_1^{(4)})$ is true, and the document is marked as *consistent*.

[Reason]. This passage states that the Lockheed P-80 Shooting Star was the first jet fighter used operationally by the United States Army Air Forces. This directly provides the information requested about organization. $\square$

(2) $D_2^{(4)}$: *Useless*. For $D_2^{(4)}$, HC returns (C_Q, C_2) , where $C_2.E \cap H_2.E \neq \emptyset$ and

$$\mathcal{D}^\delta(C_Q, C_2) = \{(u_o, v_o^{(2)}), (u_a, v_a^{(2)})\} \subseteq \Pi.$$

The values of $v_o^{(2)}$ and $v_a^{(2)}$ are “USAAF” and the “P-80 Shooting Star”. Thus, $(C_Q, C_2) \in \mathcal{R}_2$ and $\Pi_2 \neq \emptyset$. No pair in this $\mathcal{D}$ -match has u_s as its first component.

Among the cluster pairs returned in this case, (C_Q, C_1) is the only element of $\bigcup_i \mathcal{R}_i$ whose $\mathcal{D}$ -match contains a pair with first component u_s . Therefore, every set satisfying the second equality contains (C_Q, C_1) . Once (C_Q, C_1) is present, removing (C_Q, C_2) leaves both equalities true. Hence, no set satisfying the proper-subset condition intersects $\mathcal{R}_2$, and $\text{FullCore}(D_2^{(4)})$ is false.

The stored records contain neither a Condition (2)(a) record nor a Condition (2)(b) record that satisfies the definition of $\text{ConflictCert}(D_2^{(4)})$. Thus, $\text{ConflictCert}(D_2^{(4)})$ is false. Together with $\Pi_2 \neq \emptyset$, these facts make $D_2^{(4)}$ *useless*.

[Reason]. This passage states that the USAAF passed on the proposed L-133, but it does not state Which organization first used the Lockheed P-80 Shooting Star. It therefore does not establish organization. $\square$

(3) $D_3^{(4)}$: *Conflict*. For this document, HC returns (C_Q, C_3) with $C_3.E \cap H_3.E \neq \emptyset$. Its $\mathcal{D}$ -match contains $(u_o, v_o^{(3)})$ and $(u_a, v_a^{(3)})$, where $\llbracket v_o^{(3)} \rrbracket = \text{“the Americans”}$, $\llbracket v_a^{(3)} \rrbracket = \text{“North American F-86 Sabre”}$. The computation gives $h_o(u_a, v_a^{(3)}) = \text{false}$, $(u_a, v_a^{(3)}) \in \mathcal{D}^\delta(C_Q, C_3) \setminus \Pi$. It stores the Condition (2)(b) record

$$\langle (u_o, v_o^{(3)}), (C_Q, C_3), (u_a, v_a^{(3)}), 2(b) \rangle,$$

where $(u_o, v_o^{(3)}) \in \Pi^{(0)} \setminus \Pi$. Hence, $\mathcal{D}^\delta(C_Q, C_3) \not\subseteq \Pi$ and $(C_Q, C_3) \notin \mathcal{R}_3$. No other pair returned for this document belongs to $\mathcal{R}_3$, so $\text{FullCore}(D_3^{(4)})$ is false. The displayed record makes $\text{ConflictCert}(D_3^{(4)})$ true. Therefore, $D_3^{(4)}$ is marked as *conflict*.

[Reason]. The query asks about the Lockheed P-80 Shooting Star, whereas this passage states the North American F-86 Sabre. The information in this passage therefore does not satisfy organization. $\square$

For $D_1^{(4)}$, FullCore is true. For $D_2^{(4)}$, $\Pi_2 \neq \emptyset$ while both predicates are false. For $D_3^{(4)}$, FullCore is false and ConflictCert is true. These three truth values directly give the three displayed categories.

Summary. Across the four cases, a document is marked as *consistent* exactly when $\text{FullCore}(D_i)$ is true. This means that some set $\mathcal{K}$ satisfies the two displayed equalities, every proper subset of $\mathcal{K}$ fails at least one of them, and $\mathcal{K} \cap \mathcal{R}_i \neq \emptyset$. When $\text{FullCore}(D_i)$ is false, the document is marked as *conflict* exactly when $\text{ConflictCert}(D_i)$ is true. This predicate is true only when the stored records contain a Condition (2)(a) or Condition (2)(b) record satisfying its formal definition. When both predicates are false, $\Pi_i \neq \emptyset$ gives the *useless* category, whereas $\Pi_i = \emptyset$ gives the *irrelevant* category. A pair may

belong to Δ_i even when $\text{FullCore}(D_i)$ is true; hence, $\Delta_i \neq \emptyset$ alone does not determine the category. None of the documents in these four cases has

$$\neg \text{FullCore}(D_i) \wedge \neg \text{ConflictCert}(D_i) \wedge \Pi_i = \emptyset;$$

therefore, the *irrelevant* category is defined above but is not instantiated by these four cases. The resulting category and reason are passed to the downstream LLM as auxiliary suggestions. The original retrieved text is retained unchanged and is the text used by the LLM to generate the answer.